

Lab Manual

AIC 262

Introduction to Artificial Intelligence



CUI

Department of Computer Science
Islamabad Campus

Lab Contents:

The topics include: Basics of Python for AI , Different Searching techniques categorized into heuristical and blind , Analytical problems for testing AI approaches like N-Queens, Introuction to Genetic Algorithms , Introduction to Game Theory including min-max and alpha-beta pruning , and Game Modules Development.

Student Outcomes (SO)

S.#	Description
2	Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines
3	Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations
4	Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations
5	Function effectively as an individual and as a member or leader in diverse teams and in multi-disciplinary settings.

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -5	Implement various searching technique and expert system to solve an AI problem.	<i>Applying</i>	2-4
CLO-6	Develop a medium size project using AI techniques in a team environment.	<i>Applying</i>	2-5

Lab Assessment Policy

The lab work done by the student is evaluated using Psycho-motor rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

Lab #	Main Topic	Page #
Lab 01	Python basics for Artificial Intelligence	4
Lab 02	Understanding Data and Memory usage for AI	15
Lab 03	Uninformed Searching Algorithms –Breadth First Technique	30
Lab 04	Uninformed Searching Algorithms –Depth First Technique	40
Lab 05	Uninformed Searching Algorithms –Uniform Cost Technique	46
Lab 06	Greedy Search Algorithms: Shortest Path	56
Lab 07	Greedy Algorithms: Cyclic Path Problems	66
	Mid Term Exam	
Lab 08	Informed Searching Algorithms: A* Search	85
Lab 9	Informed Searching Algorithms: Hill Climbing & Best First Search	97
Lab 10	Introduction to Genetic Algorithms	109
Lab 11	Introduction to Game Theory: Min-Max and Alpha-Beta Pruning	120
Lab 12	Linear Regression: Experimenting with different Datasets	132
Lab 13	Introduction to game engines using Python	139
Lab 14	Introduction to Expert Systems using Prolog	151
Lab 15	Creating Complex knowledge bases in Expert Systems*	
Lab 16	Introduction to COLAB: Implementing a basic prediction system*	
	Final Term Exam	
	*Can be added as per Committee Recommendation	

Lab 01

Python Basics for Artificial Intelligence

Objective:

This lab is an introductory session on **Python**. It is a powerful object-oriented programming language, comparable to Perl, Ruby, Scheme and Java. This lab will enable the students to learn the core building blocks used in AI based approaches.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understanding IDE's
- Basic use of conditionals
- Basic use of loops
- Basic use of math libraries
- Developing a basic logic inference engine

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts

Programming is simply the act of entering instructions for the computer to perform. These instructions might crunch some numbers, modify text, look up information in files, or communicate with other computers over the Internet. All programs use basic instructions as building blocks.

You can combine these building blocks to implement more intricate decisions, too. Programming is a creative task, somewhat like constructing a castle out of LEGO bricks. You start with a basic idea of what you want your castle to look like and inventory your available blocks. Then you start building. Once you've finished building your program, you can pretty up your code just like you would your castle.

Python refers to the Python programming language (with syntax rules for writing what is considered valid Python code) and the Python interpreter software that reads source code (written in the Python language) and performs its instructions. The name Python comes from the surreal British comedy group Monty Python, not from the snake. Python programmers are affectionately called Pythonistas.

In regards to setting up the python environment, if you're new to Python and focused primarily on learning the language rather than building professional software, then you should install from the Microsoft Store package. This offers the shortest and easiest path to getting started with minimal hassle.

On the other hand, if you're an experienced developer looking to develop professional software in a Windows environment, then the official Python.org installer is the right choice. Your installation won't be limited by Microsoft Store policies, and you can control where the executable is installed and even add Python to PATH if necessary.

In Python, a namespace is a collection of names and the details of the objects referenced by the names. We can consider a namespace as a python dictionary which maps object names to objects. The keys of the dictionary correspond to the names and the values correspond to the objects in python. In python, there are four types of namespaces, namely built-in namespaces, global namespaces, local namespaces and enclosing namespaces.

Most recent releases of the python build can be downloaded from:

<https://www.python.org/downloads/windows/>

2) Solved Lab Activities (Allocated Time 1 Hour)

Activity 1:

Display numbers on screen using Python IDLE.

Solution:

1. Run Python IDLE
2. Type in any number, say **24** and press **Enter**.
3. **24** should be printed on the screen
4. Now type **4.2**, press **Enter**.
5. **4.2** should be displayed on screen as an output.
6. Now type **print(234)**. Press **Enter**. **234** will be the output.
7. Type **print(45.90)** and press **Enter**. The output will show **45.90** on screen.

Activity 2:

Display strings on screen.

Solution:

1. Python recognizes strings through quotes (single, double). Anything inside quotes is a string for Python interpreter.
2. Type **hello**, press **Enter**. An error message will be displayed as Python interpreter does not understand this as a string.
3. Type **'hello'** and press **Enter**. Hello will be displayed.
4. Type **'Quote me on this!'** and press **Enter**. Same string will be displayed.
5. Type **"What's your name?"** and press **Enter**. **What's your name?** will be printed on the screen.
6. You can specify multi-line strings using triple quotes – **"""** or **'''**. Type following text and press Enter

```
"""This is a multi-line string. This is the first line.  
This is the second line.  
"What's your name?," I asked.  
He said "Bond, James Bond." """
```

Activity 3:

Use Python as a calculator.

Solution:

1. The interpreter acts as a simple calculator: you can type an expression at it and it will write the value. Expression syntax is straightforward: the operators + (addition), - (subtraction), * (multiplication) and / (division) work just like in most other languages. Parentheses () can be used for grouping.
2. Type in `2 + 2` and press **Enter**. Python will output its result i.e. `4`.
3. Try following expressions in the shell.
 - a. `50 - 4`
 - b. `23.5 - 2.0`
 - c. `23 - 18.5`
 - d. `5 * 6`
 - e. `2.5 * 10`
 - f. `2.5 * 2.5`
 - g. `28 / 4` (note: division returns a floating point number)
 - h. `26 / 4`

Activity 4:

Get an integer answer from division operation. Also get remainder of a division operation in the output.

Solution:

1. Division (/) always gives a float answer. There are different ways to get a whole as an answer.
2. Use // operator:
 - a. Type `28 // 4` and press **Enter**. The answer is a whole number.
 - b. Type `26 // 4` and press **Enter**.
3. Use (int) cast operator: this operator changes the interchangeable types.
 - a. Type `(int)26 / 4` and press **Enter**. The answer is a whole number.
 - b. Type `(int) 28/4`; press **Enter**.
4. The modulus (or mod) % operator is used to get the remainder as an output (division / operator returns the quotient).

-
- a. Type **28 % 4**. Press **Enter**. **0** will be the result.
 - b. Type **26 % 4**. Press **Enter**. **2** will be the result.

Activity 5:

Calculate 4^3 , 4^{10} , 4^{29} , 4^{150} , 4^{1000}

Solution:

1. The multiplication (*) operator can be used for calculating powers of a number. However, if the power is big, the task will be tedious. For calculating powers of a number, Python uses ** operator.
2. Type following and obtain the results of above expressions.

a

. `4 ** 3` b. `4 ** 10`

c. `4 ** 29`

d. `4 ** 150`

e. `4 ** 1000`

Activity 6:

Write following math expressions. Solve them by hand using operators' precedence. Calculate their answers using Python. Match the results.

Solution:

The table below shows the operator precedence in Python (from Highest to Lowest).

Operator	Operation	Example
**	Exponent	<code>2 ** 3 = 8</code>
%	Modulus/remainder	<code>22 % 8 = 6</code>
//	Integer division/floored quotient	<code>22//8 = 2</code>
/	Division	<code>22/8 = 2.75</code>
*	Multiplication	<code>3*5 = 15</code>
-	Subtraction	<code>5 - 2 = 3</code>
+	Addition	<code>2 + 2 = 4</code>

Calculate following expressions:

-
1. $2+3*6$
 2. $(2+3)*6$
 3. $48565878 * 578453$
 4. $2 + \quad \quad 2$ (note the spaces after +)
 5. $(5 - 1) * ((7 + 1) / (3 - 1))$
 6. $5 +$
 7. $42 + 5 + * 2$

Activity 7:

Combine numbers and text.

Solution:

Type the following code. Run it.

```
# Text x =  
"Nancy"  
print(x)  
  
# Combine numbers and text s = "My lucky number is  
%d, what is yours?" % 7 print(s)  
  
# alternative method of combining numbers and text s = "My lucky  
number is " + str(7) + ", what is yours?" print(s)
```

Activity 8:

Take input from the keyboard and use it in your program.

Solution:

In Python and many other programming languages you can get user input. In Python the `input()` function will ask keyboard input from the user. The `input` function prompts text if a parameter is given. The function reads input from the keyboard, converts it to a string and removes the newline (Enter). Type and experiment with the script below.

```
#!/usr/bin/env python3

name = input("What is your name? ")
print('Hello ' + name)

job = input("What is your job? ")
print('Your job is ' + job)

num = input('Give me a number? ')
print('You said: ' + str(num))
```

Activity 9:

Let us take an integer from user as input and check whether the given value is even or not.

Solution:

- A. Create a new Python file from Python Shell and type the following code.
- B. Run the code by pressing F5.

```
n=input("Enter a number ")
if int(n)%2==0:
    print("The given number is an even number")
```

You will get the following output.

```
Enter a number 12
The given number is an even number
>>>
```

Activity 10:

Let us modify the code to take an integer from user as input and check whether the given value is even or odd. If the given value is not even then it means that it will be odd. So here we need to use if-else statement as demonstrated below.

Solution:

- A. Create a new Python file from Python Shell and type the following code.
- B. Run the code by pressing F5.

```
n=input("Enter a number ")
if int(n)%2==0:
    print("The given number is an even number")
else:
    print("The given number is an odd number")
```

You will get the following output.

```
Enter a number 11
The given number is an odd number
>>>
```

Activity 11:

Calculate the sum of all the values between 0-10 using while loop.

Solution:

- Create a new Python file from Python Shell and type the following code.
- Run the code by pressing F5.

```
summation = 0
i=1
while i<=10:
    summation=summation+i
    i=i+1

print("sum is ",summation)
```

You will get the following output.

```
sum is 55
>>>
```

Activity 12:

Accept 5 integer values from user and display their sum. Draw flowchart before coding in python.

Solution:

- Create a new Python file from Python Shell and type the following code.
- Run the code by pressing F5.

```
summ = 0
i=0
while i<=4:
    s=input("enter a number")
    n=int(s)
    summ=summ+n
    i=i+1

print("sum is ",summ)
```

You will get the following output.

```
enter a number1
enter a number2
enter a number3
enter a number4
enter a number5
sum is 15
>>>
```

Activity 13:

Write a Python code to keep accepting integer values from user until 0 is entered. Display sum of the given values.

Solution:

- Create a new Python file from Python Shell and type the following code.
- Run the code by pressing F5.

```
sum=0
s=input("Enter an integer value...")
n=int(s)
while n!=0:
    sum=sum+n
    s=input("Enter an integer value...")
    n=int(s)
print("Sum of given values is ",sum)
```

You will get the following output.

```
Enter an integer value...10
Enter an integer value...521
Enter an integer value...5
Enter an integer value...22
Enter an integer value...0
Sum of given values is 558
>>>
```

Activity 14:

Write a Python code to accept an integer value from user and check that whether the given value is prime number or not.

Solution:

- Create a new Python file from Python Shell and type the following code.
- Run the code by pressing F5.

```
isPrime = True
i=2
n=int(input("enter a number"))
while i<n:
    remainder=n%i
    if remainder==0:
        isPrime=False
        break
    else:
        i=i+1

if isPrime:
    print("Number is Prime")
else:
    print("Number is not Prime")
```

You will get the following output(s).

```
enter a number17
Number is Prime
>>>

enter a number49
Number is not Prime
>>> |
```

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

Lab Task 1:

Write a Python code to accept marks of a student from 1-100 and display the grade according to the following formula.

Grade F if marks are less than 50 Grade E if marks are between 50 to 60 Grade D if marks are between 61 to 70 Grade C if marks are between 71 to 80 Grade B if marks are between 81 to 90

Grade A if marks are between 91 to 100

Lab Task 2:

Write a program that takes a number from user and calculate the factorial of that number.

Lab Task 3:

Fibonacci series is that when you add the previous two numbers the next number is formed.

You have to start from 0 and 1.

E.g. $0+1=1 \rightarrow 1+1=2 \rightarrow 1+2=3 \rightarrow 2+3=5 \rightarrow 3+5=8 \rightarrow 5+8=13$

So the series becomes

0 1 1 2 3 5 8 13 21 34 55

Steps: You have to take an input number that shows how many terms to be displayed. Then use loops for displaying the Fibonacci series up to that term e.g. input no is =6 the output should be 0 1 1 2 3 5.

Lab 02

Understanding Data and Memory usage for AI

Objective:

This lab will give you practical implementation of different types of sequences including lists, tuples, sets and dictionaries. We will use lists alongside loops in order to know about indexing individual items of these containers. This lab will also allow students to write their own functions.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Develop a basic understanding of data structures commonly used for AI
- Understanding Lists and Tuples
- Solving complex mathematical problems
- Using loops with lists
- Developing customized functions for logical inference
- Basic understanding of Object Oriented Concepts like Class definitions etc.

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts

In Python the concept of Lists is used which is just like arrays, declared in other languages which is an ordered collection of data. It is very flexible as the items in a list do not need to be of the same type.

The implementation of Python List is similar to Vectors in C++ or ArrayList in JAVA. The costly operation is inserting or deleting the element from the beginning of the List as all the elements are needed to be shifted. Insertion and deletion at the end of the list can also become costly in the case where the pre-allocated memory becomes full.

Python provides different types of data structures as sequences. In a sequence, there are more than one values and each value has its own index as can be seen in Figure 1 below:

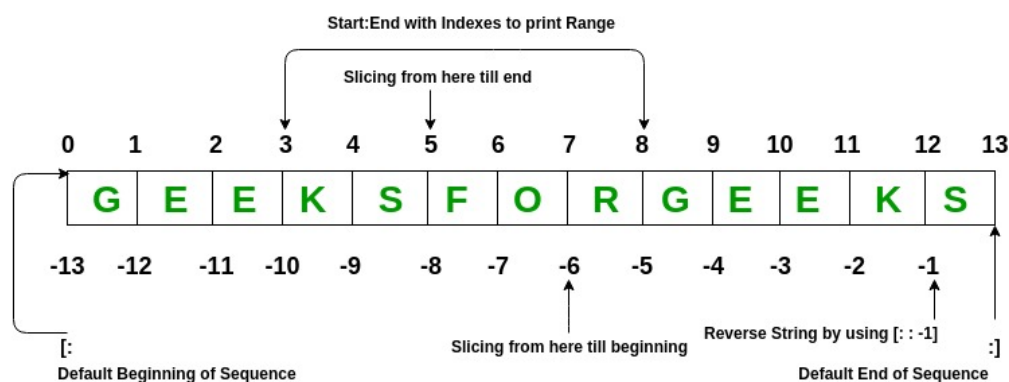


Figure 1: Assigning values in a List (Image Courtesy: <https://www.geeksforgeeks.org/python-data-structures/>)

The first value will have an index 0 in python, the second value will have index 1 and so on. These indices are used to access a particular value in the sequence.

Python offers different types of sequences. Lists are the most important type of sequence being used in Python. It is a collection of same or different type of objects. These objects are separated by commas to distinguish from each other enclosed in square brackets. The following activities show that how lists are used in Python.

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets. The data type "set", which is a collection type, has been part of Python since version 2.4. A set contains an unordered collection of unique and immutable objects.

The set data type is, as the name implies, a Python implementation of the sets as they are known from mathematics. This explains, why sets unlike lists or tuples can't have multiple occurrences of the same element.

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values.

2) Solved Lab Activites (Allocated Time 1 Hour)

Activity 1:

For developing un understanding, use loops to accept 5 values from user and store them in a list. Display all the values (objects) of the list.

```
myList=[]
for i in range(5):
    val=input("Enter a value:")
    myList.append(val)

print("The given list is")
print(myList)
```

The output should be similar to the following:

```
Enter a value:1
Enter a value:Mango
Enter a value:Fruit
Enter a value:Pakistan
Enter a value:1947
The given list is
['1', 'Mango', 'Fruit', 'Pakistan', '1947']
>>>
```

Activity 2:

Accept 5 integer values from user. Store these values in a list and display the list in ascending order.

```
myList=[]
for i in range(5):
    val=input("Enter a value:")
    n=int(val)
    myList.append(n)

myList.sort()
print(myList)
```

Notice the use of sort

The output in this case would be:

```
Enter a value:1
Enter a value:5
Enter a value:2
Enter a value:4
Enter a value:3
[1, 2, 3, 4, 5]
>>>
```

Activity 3:

Accept two lists from user and display their join.

```
myList1=[]
print("Enter objects of first list...")
for i in range(5):
    val=input("Enter a value:")
    n=int(val)
    myList1.append(n)

myList2=[]
print("Enter objects of second list...")
for i in range(5):
    val=input("Enter a value:")
    n=int(val)
    myList2.append(n)

list3=myList1+myList2;
print(list3)
```

The output should be:

```
Enter objects of first list...
Enter a value:1
Enter a value:2
Enter a value:3
Enter a value:4
Enter a value:5
Enter objects of second list...
Enter a value:6
Enter a value:7
Enter a value:8
Enter a value:9
Enter a value:0
[1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
>>>
```

Activity 4:

Nesting a list inside of a list and accessing elements.

```
# Creating a List with
# the use of multiple values
List = ["Geeks", "For", "Geeks"]
print("\nList containing multiple values: ")
print(List)

# Creating a Multi-Dimensional List
# (By Nesting a list inside a List)
List2 = [['Geeks', 'For'], ['Geeks']]
print("\nMulti-Dimensional List: ")
print(List2)

# accessing a element from the
# list using index number
print("Accessing element from the list")
print(List[0])
print(List[2])

# accessing a element using
# negative indexing
print("Accessing element using negative indexing")

# print the last element of list
print(List[-1])

# print the third last element of list
print(List[-3])
```

Output should be:

```
List containing multiple values:
['Geeks', 'For', 'Geeks']
```

```
Multi-Dimensional List:
```

```
[['Geeks', 'For'], ['Geeks']]
```

```
Accessing element from the list
```

```
Geeks
```

Geeks

Accessing element using negative indexing

Geeks

Geeks

Activity 5:

Creating a dictionary in Python, basically Python dictionary is like hash tables in any other language. It is an unordered collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element, Dictionary holds the key:value pair. Key-value is provided in the dictionary to make it more optimized.

Indexing of Python Dictionary is done with the help of keys. These are of any hashable type i.e. an object whose can never change like strings, numbers, tuples, etc. We can create a dictionary by using curly braces ({}) or dictionary comprehension.

```
# Creating a Dictionary
Dict = {'Name': 'Geeks', 1: [1, 2, 3, 4]}
print("Creating Dictionary: ")
print(Dict)

# accessing a element using key
print("Accessing a element using key:")
print(Dict['Name'])

# accessing a element using get()
# method
print("Accessing a element using get:")
print(Dict.get(1))

# creation using Dictionary comprehension
myDict = {x: x**2 for x in [1,2,3,4,5]}
print(myDict)
```

The output should be:

Creating Dictionary:

```
{'Name': 'Geeks', 1: [1, 2, 3, 4]}
```

Accessing a element using key:

Geeks

Accessing a element using get:

```
[1, 2, 3, 4]
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

The dictionary can also be used with some added in-built features like OrderedDict. An OrderedDict is also a sub-class of dictionary but unlike a dictionary, it remembers the order in which the keys were inserted.

```
from collections import OrderedDict
```

```
print("Before deleting:\n")
```

```
od = OrderedDict()
```

```
od['a'] = 1
```

```
od['b'] = 2
```

```
od['c'] = 3
```

```
od['d'] = 4
```

```
for key, value in od.items():
```

```
    print(key, value)
```

```
print("\nAfter deleting:\n")
```

```
od.pop('c')
```

```
for key, value in od.items():
```

```
    print(key, value)
```

```
print("\nAfter re-inserting:\n")
```

```
od['c'] = 3
```

```
for key, value in od.items():
```

```
    print(key, value)
```

In this case the output would be:

Before deleting:

a 1

b 2

c 3

d 4

After deleting:

a 1

b 2

d 4

After re-inserting:

a 1

b 2

d 4

c 3

Similarly for combining many dictionaries we can use a Chainmap. A ChainMap encapsulates many dictionaries into a single unit and returns a list of dictionaries. When a key is needed to be found then all the dictionaries are searched one by one until the key is found.

```
from collections import ChainMap
```

```
d1 = {'a': 1, 'b': 2}
```

```
d2 = {'c': 3, 'd': 4}
```

```
d3 = {'e': 5, 'f': 6}
```

```
# Defining the chainmap
```

```
c = ChainMap(d1, d2, d3)
```

```
print(c)
```

```
print(c['a'])
```

```
print(c['g'])
```

In this case the output would be:

```
ChainMap({'a': 1, 'b': 2}, {'c': 3, 'd': 4}, {'e': 5, 'f': 6})
```

```
1
```

```
KeyError: 'g'
```

Note the missing key returned an error as it was not able to find it in dictionaries.

Activity 6:

The Python Tuple is a collection of Python objects much like a list but Tuples are immutable in nature i.e. the elements in the tuple cannot be added or removed once created. Just like a List, a Tuple can also contain elements of various types.

In Python, tuples are created by placing a sequence of values separated by ‘comma’ with or without the use of parentheses for grouping of the data sequence.

```
# Creating a Tuple with
# the use of Strings
Tuple = ('Geeks', 'For')
print("\nTuple with the use of String: ")
print(Tuple)
```

```
# Creating a Tuple with
# the use of list
list1 = [1, 2, 4, 5, 6]
print("\nTuple using List: ")
Tuple = tuple(list1)
```

```
# Accessing element using indexing
print("First element of tuple")
print(Tuple[0])
```

```
# Accessing element from last
# negative indexing
print("\nLast element of tuple")
print(Tuple[-1])
```

```
print("\nThird last element of tuple")
print(Tuple[-3])
```

The output should be:

Tuple with the use of String:

```
('Geeks', 'For')
```

```
Tuple using List:
```

```
First element of tuple
```

```
1
```

```
Last element of tuple
```

```
6
```

```
Third last element of tuple
```

```
4
```

The Tuple also has an enhanced version called NamedTuple, which returns a tuple object with names for each position which the ordinary tuples lack.

For example, consider a tuple names student where the first element represents fname, second represents lname and the third element represents the DOB. Suppose for calling fname instead of remembering the index position you can actually call the element by using the fname argument, then it will be really easy for accessing tuples element. This functionality is provided by the NamedTuple.

```
from collections import namedtuple
# Declaring namedtuple()
Student = namedtuple('Student',['name','age','DOB'])
# Adding values
S = Student('Akber','19','2541997')
# Access using index
print ("The Student age using index is : ",end = "")
print (S[1])
# Access using name
print ("The Student name using keyname is : ",end = "")
print (S.name)
```

The output would be:

```
The Student age using index is : 19
```

```
The Student name using keyname is : Akber
```

Activity 7:

The Python Set is an unordered collection of data that is mutable and does not allow any duplicate element. Sets are basically used to include membership testing and eliminating duplicate entries. The data structure used in this is Hashing, a popular technique to perform insertion, deletion, and traversal.

If Multiple values are present at the same index position, then the value is appended to that index position, to form a Linked List.

```
# Creating a Set with
# a mixed type of values
# (Having numbers and strings)
Set = set([1, 2, 'Geeks', 4, 'For', 6, 'Geeks'])
print("\nSet with the use of Mixed Values")
print(Set)

# Accessing element using
# for loop
print("\nElements of set: ")
for i in Set:
    print(i, end = " ")
print()

# Checking the element
# using in keyword
print("Geeks" in Set)
```

The output should be:

Set with the use of Mixed Values

{1, 2, 'Geeks', 4, 6, 'For'}

Elements of set:

1 2 Geeks 4 6 For

True

Try also experimenting with frozen sets.

Activity 8:

The Python Strings are arrays of bytes representing Unicode characters. In simpler terms, a string is an immutable array of characters. Python does not have a character data type, a single character is simply a string with a length of 1. As strings are immutable, modifying a string will result in creating a new copy.

```
String = "Welcome to GeeksForGeeks"
print("Creating String: ")
print(String)

# Printing First character
print("\nFirst character of String is: ")
print(String[0])

# Printing Last character
print("\nLast character of String is: ")
print(String[-1])
```

The output should be:

```
Creating String:
Welcome to GeeksForGeeks
First character of String is:
W
Last character of String is:
s
```

Try also experimenting with Bytearray.

Activity 9:

If you are a beginner in python programming, you must be knowing about primitive data types like integers, floating point numbers, strings and complex numbers. Also, till now we have built data structures like python dictionary, list, tuple and set. Now we will learn to create complex data types to store information about real world objects using classes in python.

```
class Person:

    def __init__(self, name, age):

        self.name = name

        self.age = age
```

```
def myfunc(self):

    print("Hello my name is " + self.name)

    #print("Hello my age is " + self.age)

p1 = Person("akber", 36)

p1.myfunc()
```

Activity 10:

It is especially important to understand memory allocations in Python. As python is used in ML and AI where large datasets are used. Memory leaks,i.e, the program is out of memory after running for several hours, is a common problem. To manage these memory leaks memory monitoring is essential. Monitoring memory is also called profiling. As a developer, it's a necessity that we profile our program and use less memory allocation as much as possible.

For this purpose one method is Tracemalloc which is a library module that traces every memory block in python. The tracing starts by using the start() during runtime. This library module can also give information about the total size, number, and average size of allocated memory blocks.

In the first instance we can use the tracemalloc library for a reference problem.

```
# importing the module
import tracemalloc

# code or function for which memory
# has to be monitored
def app():
    lt = []
    for i in range(0, 100000):
        lt.append(i)

# starting the monitoring
tracemalloc.start()

# function call
app()

# displaying the memory
```

```
print(tracemalloc.get_traced_memory())
```

```
# stopping the library
```

```
tracemalloc.stop()
```

The output is given in form of (current, peak), i.e, current memory is the memory the code is currently using and peak memory is the maximum space the program used while executing.

```
(0, 3617252)
```

You can also experiment with other Python libraries like PsUtil and Memory Profiler. But they need to be installed separately using PiP INSTALL Command.

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

Lab Task 1:

Let us put our learning to test now, and solve the following given problems:

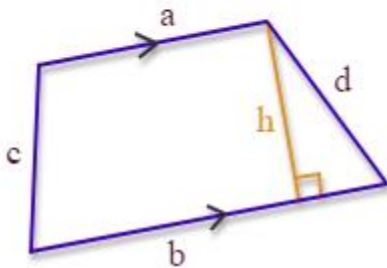
- i. Write a Python program to create an instance of a specified class and display the namespace of the said instance.
- ii. Write a Python function `student_data ()` which will print the id of a student (`student_id`). If the user passes an argument `student_name` or `student_class` the function will print the student name and class
- iii. Write a simple Python class named `Student` and display its type. Also, display the `__dict__` attribute keys and the value of the `__module__` attribute of the `Student` class.
- iv. Write a Python program to create two empty classes, `Student` and `Marks`. Now create some instances and check whether they are instances of the said classes or not. Also, check whether the said classes are subclasses of the built-in object class or not
- v. Write a Python class named `Student` with two attributes `student_name`, `marks`. Modify the attribute values of the said class and print the original and modified values of the said attributes.
- vi. Write a Python class named `Student` with two attributes `student_id`, `student_name`. Add a new attribute `student_class` and display the entire attribute and their values of the said class. Now remove the `student_name` attribute and display the entire attribute with values.
- vii. Write a Python class named `Student` with two attributes `student_id`, `student_name`. Add a new attribute `student_class`. Create a function to display the entire attribute and their values in `Student` class.
- viii. Write a Python class named `Student` with two instances `student1`, `student2` and assign given values to the said instances attributes. Print all the attributes of `student1`, `student2` instances with their values in the given format.
- ix. Run Memory Allocation tests on the created Python class.
- x. Try reducing the memory allocation using different techniques.

Lab Task 2:

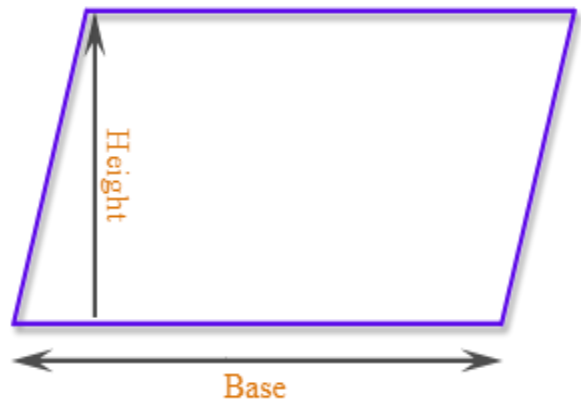
- i. Make two classes for trapezoid and parallelogram and write a code to calculate their areas.
- ii. Also write a comparison class to see which shape has the largest area, store results in a dictionary, iterate for different dimensions and use dictionary functions to search the shapes with largest areas.

Area of a Trapezoid

$$A = \frac{1}{2} (a + b) h$$



Parallelogram



Area of parallelogram = **Base** x **Height**

Lab 03

Uninformed Searching Algorithms

Breadth First Technique

Objective:

This lab will introduce students to search problems. We will first start by representing problems in terms of state space graph. Given a state space graph, starting state and a goal state, students will then perform a basic Breadth First Search solution within that graph. The output will be a set of actions or a path that will begin from initial state/node and end in the goal node.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Develop a basic understanding of uninformed search techniques
- Understand breadth first search technique
- Understand how to create a state space from a tree

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 30 Minutes)

Blind search, also called uninformed search, works with no information about the search space, other than to distinguish the goal state from all the others.

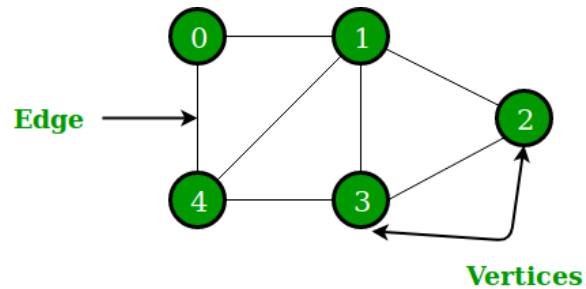
A search strategy is defined by the order of node expansion where the strategies are evaluated along the following dimensions:

- completeness: does it always find a solution if one exists?
- time complexity: number of nodes generated
- space complexity: maximum number of nodes in memory
- optimality: does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b: maximum branching factor of the search tree
 - d: depth of the least-cost solution
 - m: maximum depth of the state space (may be ∞)

A graph is collection of two sets V and E where V is a finite non-empty set of vertices and E is a finite non-empty set of edges.

- Vertices are nothing but the nodes in the graph.
- Two adjacent vertices are joined by edges.
- Any graph is denoted as $G = \{V, E\}$.

For Example:

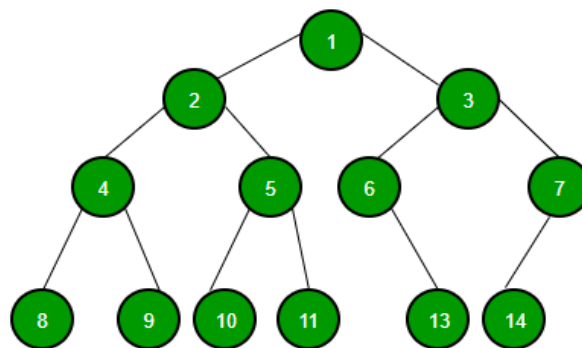


$$G = \{\{V_1, V_2, V_3, V_4, V_5, V_6\}, \{E_1, E_2, E_3, E_4, E_5, E_6, E_7\}\}$$

Whereas, a tree is a finite set of one or more nodes such that:

1. There is a specially designated node called root.
2. The remaining nodes are partitioned into $n \geq 0$ disjoint sets $T_1, T_2, T_3, \dots, T_n$ where $T_1, T_2, T_3, \dots, T_n$ is called the subtrees of the root.

The concept of tree is represented by following Fig.



There are different types of blind or uninformed searches that can be used to solve a certain problem. Commonly used search techniques are:

- Breadth First Search
- Depth First Search
- Uniform Cost Search

We will divulge upon each search techniques as we progress. In this lab session we will discuss Breadth-First Search (BFS). The BFS is an algorithm used for traversing graphs or trees. Traversing means visiting each node of the graph. Breadth-First Search is a recursive algorithm to search all the vertices of a graph or a tree. BFS in python can be implemented by using data structures like a dictionary and lists. Breadth-First Search in tree and graph is almost the same. The only difference is that the graph may contain cycles, so we may traverse to the same node again.

Before learning the python code for Breadth-First and its output, let us go through the algorithm it follows for the same. We can take the example of Rubik's Cube for the instance. Rubik's Cube is seen as searching for a path to convert it from a full mess of colors to a single color. So comparing the Rubik's Cube to the graph, we can say that the possible state of the cube is corresponding to the nodes of the graph and the possible actions of the cube is corresponding to the edges of the graph.

As breadth-first search is the process of traversing each node of the graph, a standard BFS algorithm traverses each vertex of the graph into two parts: 1) Visited 2) Not Visited. So, the purpose of the algorithm is to visit all the vertex while avoiding cycles. The first value will have an index 0 in python, the second value will have index 1 and so on. These indices are used to access a particular value in the sequence.

Breadth first search is quite useful in the field of AI having varied applications like:

- Shortest Path and Minimum Spanning Tree for unweighted graph In an unweighted graph, the shortest path is the path with least number of edges. With Breadth First, we always reach a vertex from given source using the minimum number of edges.
- Peer to Peer Networks: In Peer to Peer Networks like BitTorrent, Breadth First Search is used to find all neighbor nodes.
- Crawlers in Search Engines: Crawlers build index using Breadth First. The idea is to start from source page and follow all links from source and keep doing same. Depth First Traversal can also be used for crawlers, but the advantage with Breadth First Traversal is, depth or levels of the built tree can be limited.
- Social Networking Websites: In social networks, we can find people within a given distance 'k' from a person using Breadth First Search till 'k' levels.
- GPS Navigation systems: Breadth First Search is used to find all neighboring locations.
- Broadcasting in Network: In networks, a broadcasted packet follows Breadth First Search to reach all nodes.

2) Solved Lab Activities (Allocated Time 1 Hour)

Activity 1:

The BFS search starts from a node, then it checks all the nodes at distance one from the beginning node, then it checks all the nodes at distance two, and so on. So as to recollect the nodes to be visited, BFS uses a queue.

The steps of the algorithm work as follow:

1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.

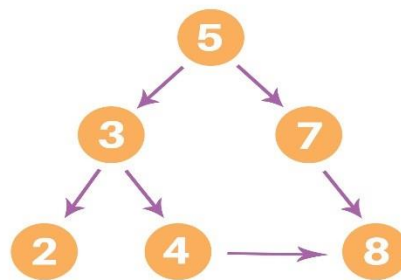


Figure: Graph for BFS Implementation

Many times, a graph may contain two different disconnected parts and therefore to make sure that we have visited every vertex, we can also run the BFS algorithm at every node.

```
graph = {
    '5' : ['3', '7'],
    '3' : ['2', '4'],
    '7' : ['8'],
    '2' : [],
    '4' : ['8'],
    '8' : []
}
```

##The above code will create a graph with the entries given in the figure above.

```

visited = [] # List for visited nodes.
queue = []   #Initialize a queue

def bfs(visited, graph, node): #function for BFS
    visited.append(node)
    queue.append(node)

    while queue:          # Creating loop to visit each node
        m = queue.pop(0)
        print (m, end = " ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, '5')    # function calling

```

Activity 2:

Try implementing BFS for a tree now, using the following tree structure:

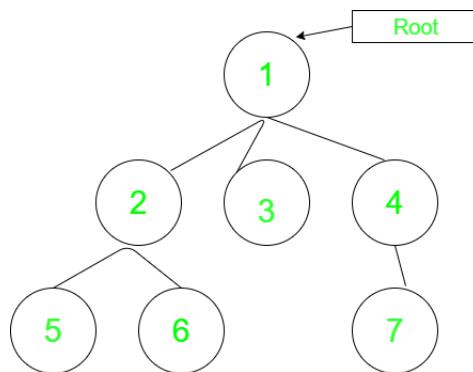


Figure: Tree for BFS Implementation

The code to make the tree is given as

```

# python program to demonstrate some of the above
# terminologies

# Function to add an edge between vertices x and y

# Function to print the parent of each node

```

```
def printParents(node, adj, parent):

    # current node is Root, thus, has no parent
    if (parent == 0):
        print(node, "->Root")
    else:
        print(node, "->", parent)

    # Using DFS
    for cur in adj[node]:
        if (cur != parent):
            printParents(cur, adj, node)

# Function to print the children of each node
def printChildren(Root, adj):

    # Queue for the BFS
    q = []

    # pushing the root
    q.append(Root)

    # visit array to keep track of nodes that have been
    # visited
    vis = [0]*len(adj)

    # BFS
    while (len(q) > 0):
        node = q[0]
        q.pop(0)
        vis[node] = 1
        print(node, "-> ", end=" ")

        for cur in adj[node]:
            if (vis[cur] == 0):
                print(cur, " ", end=" ")
                q.append(cur)
        print("\n")

# Function to print the leaf nodes
def printLeafNodes(Root, adj):
```

```

# Leaf nodes have only one edge and are not the root
for i in range(0, len(adj)):
    if (len(adj[i]) == 1 and i != Root):
        print(i, end=" ")
print("\n")

# Function to print the degrees of each node
def printDegrees(Root, adj):

    for i in range(1, len(adj)):
        print(i, ": ", end=" ")

        # Root has no parent, thus, its degree is equal to
        # the edges it is connected to
        if (i == Root):
            print(len(adj[i]))
        else:
            print(len(adj[i])-1)

# Driver code

# Number of nodes
N = 7
Root = 1

# Adjacency list to store the tree
adj = []
for i in range(0, N+1):
    adj.append([])

# Creating the tree
adj[1].append(2)
adj[2].append(1)

adj[1].append(3)
adj[3].append(1)

adj[1].append(4)
adj[4].append(1)

adj[2].append(5)
adj[5].append(2)

```

```
adj[2].append(6)
adj[6].append(2)

adj[4].append(7)
adj[7].append(4)

# Printing the parents of each node
print("The parents of each node are:")
printParents(Root, adj, 0)

# Printing the children of each node
print("The children of each node are:")
printChildren(Root, adj)

# Printing the leaf nodes in the tree
print("The leaf nodes of the tree are:")
printLeafNodes(Root, adj)

# Printing the degrees of each node
print("The degrees of each node are:")
printDegrees(Root, adj)
```

Activity 3:

Using the code in ‘activity 1’ apply the same procedure to the graph give in the Figure below:

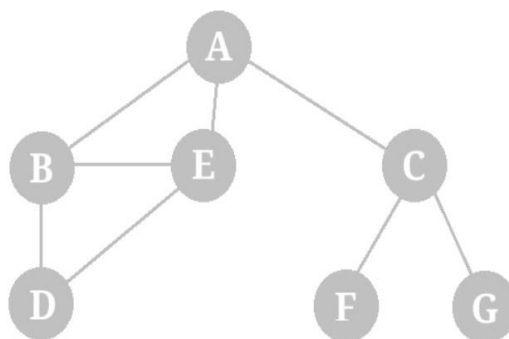


Figure: Graph for Activity 3

Activity 4:

Try changing the start and goal states for the Activity 3 implementation and observe what happens.

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

Imagine the same graph we discussed above but this time we also mention the cost of each edge.

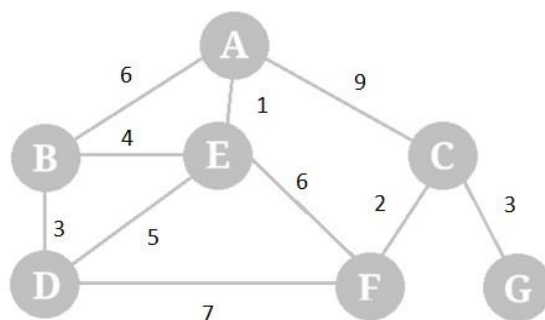


Figure: Weighted Graph for Activity Lab Task 1

First we modify our graph structure so that in Node.actions is an array of tuples where each tuple contains a vertex and its associated weight.

```
graph = { 'A': Node('A', None, [('B', 6), ('C', 9), ('E', 1)], 0),
          'B': Node('B', None, [('A', 6), ('D', 3), ('E', 4)], 0),
          'C': Node('C', None, [('A', 9), ('F', 2), ('G', 3)], 0),
          'D': Node('D', None, [('B', 3), ('E', 5), ('F', 7)], 0),
          'E': Node('E', None, [('A', 1), ('B', 4), ('D', 5), ('F', 6)], 0),
          'F': Node('F', None, [('C', 2), ('E', 6), ('D', 7)], 0),
          'G': Node('G', None, [('C', 3)], 0) }
```

Once the graph is updated, try applying BFS algorithm and calculate the total cost from start to goal state (A to G).

Try different variations of start and goal states and see what happens to path costs.

Lab Task 2:

In order to move onto more complex implementations of BFS let us consider a concise version of Travelling Salesman Problem given in Figure below:

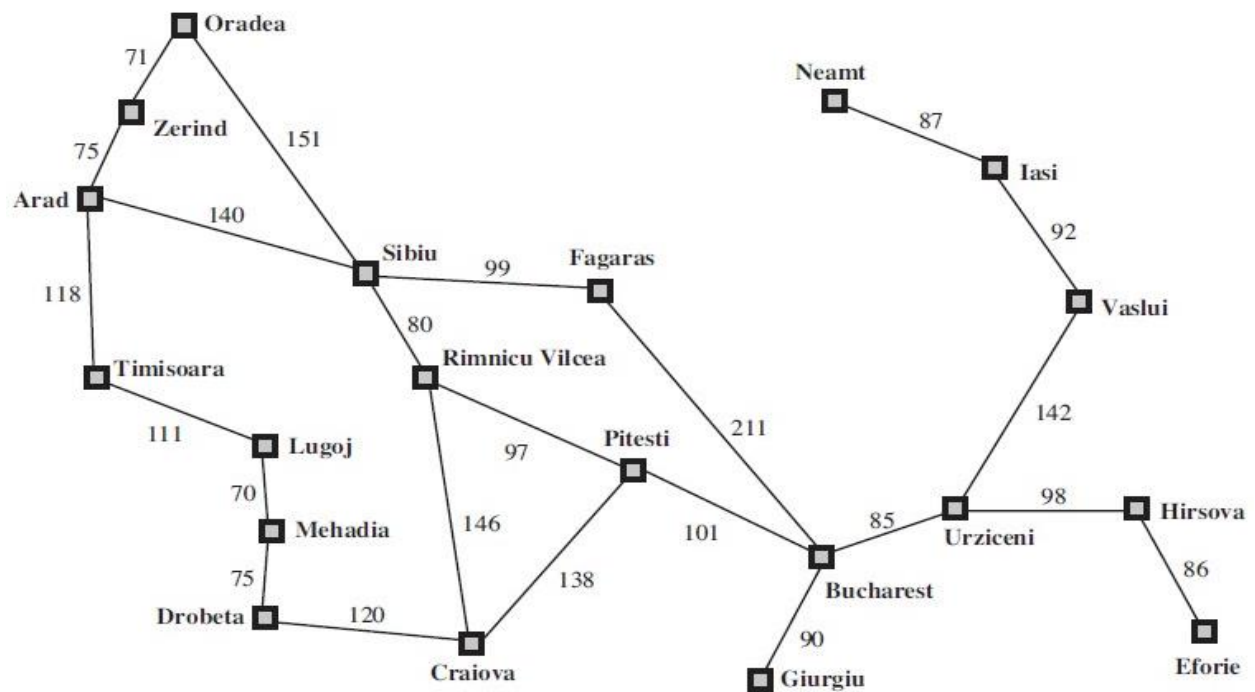


Figure: Travelling Salesman Problem

- Draw a corresponding graph from the above Figure from Arad to Bucharest containing all possible nodes. (Arad as Start Node and Bucharest as Goal Node)
- Apply breadth first search to reach the goal state, your output should have the path and the total distance to reach the path.
- Save your code and results as BFS_REG#.py as it would be used in later exercises.

Lab 04

Uninformed Searching Algorithms

Depth First Technique

Objective:

This lab will introduce students to search problems. We will first start by representing problems in terms of state space graph. Given a state space graph, starting state and a goal state, students will then perform a basic Depth First Search solution within that graph. The output will be a set of actions or a path that will begin from initial state/node and end in the goal node.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand depth first search technique
- Understand how to solve different path problems using depth first technique

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 15 Minutes)

Recall the concepts discussed in the previous lab activity regarding blind search techniques. We observed that there are several types of blind or uninformed searches that can be used to solve a certain problem. Commonly used search techniques are:

- Breadth First Search
- Depth First Search
- Uniform Cost Search

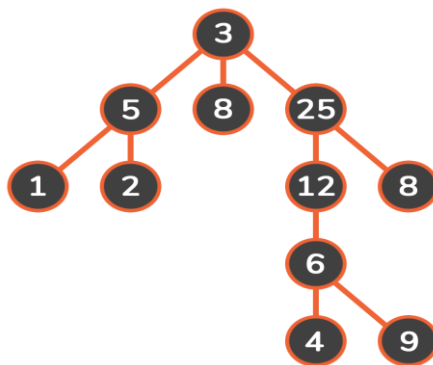


Figure: DFS Path [3-5-1-2-8-25-12-6-4-9-8]

The Depth-First Search (DFS) is an algorithm used to traverse or locate a target node in a graph or tree data structure. It prioritizes depth and searches along one branch, as far as it can go - until the end of that branch. Once there, it backtracks to the first possible divergence from that branch, and searches until the end of that branch, repeating the process.

Given the nature of the algorithm, you can easily implement it recursively - and you can always implement a recursive algorithm iteratively as well.

The start node is the root node for tree data structures, while with more generic graphs it can be any node.

The DFS is widely used as a part of many other algorithms that resolve graph-represented problems. From cycle searches, path finding, topological sorting, to finding articulation points and strongly connected components. The reason behind this widespread use of the DFS algorithm lies in its overall simplicity and easy recursive implementation.

2) Solved Lab Activities (Allocated Time 1 Hour)

Activity 1:

The Depth-First Search is a recursive algorithm that uses the concept of backtracking. It involves thorough searches of all the nodes by going ahead if potential, else by backtracking. Here, the word backtrack means once you are moving forward and there are not any more nodes along the present path, you progress backward on an equivalent path to seek out nodes to traverse. All the nodes are progressing to be visited on the current path until all the unvisited nodes are traversed after which subsequent paths are going to be selected.

Before learning the python code for Depth-First and its output, let us go through the algorithm it follows for the same. The recursive method of the Depth-First Search algorithm is implemented using stack.

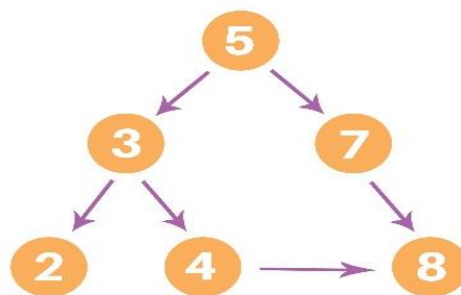


Figure: Graph for DFS implementation

The DSF algorithm follows as:

We will start by putting any one of the graph's vertex on top of the stack.

After that take the top item of the stack and add it to the visited list of the vertex.

Next, create a list of that adjacent node of the vertex. Add the ones which aren't in the visited list of vertexes to the top of the stack.

Lastly, keep repeating steps 2 and 3 until the stack is empty.

```
# Using a Python dictionary to act as an adjacency list
graph = {
    '5' : ['3', '7'],
    '3' : ['2', '4'],
    '7' : ['8'],
    '2' : [],
    '4' : ['8'],
    '8' : []
}

visited = set() # Set to keep track of visited nodes of graph.

def dfs(visited, graph, node): #function for dfs
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Depth-First Search")
dfs(visited, graph, '5')
```

Th output in this case would be: 5 3 2 4 8 7

Activity 2:

The Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree. The only difference is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a Boolean visited array.

For example, in the graph given in the Figure below, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. A Depth First Traversal of the following graph is 2, 0, 1, 3.

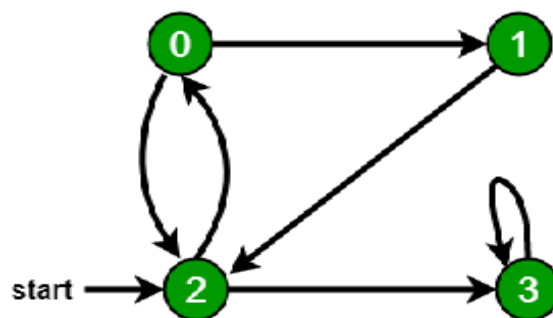


Figure: DFS Graph for implementation

```
# Python program to print DFS traversal for complete graph
from collections import defaultdict
```

```
# This class represents a directed graph using adjacency
# list representation
```

```
class Graph:
```

```
    # Constructor
```

```
    def __init__(self):
```

```
        # default dictionary to store graph
```

```
        self.graph = defaultdict(list)
```

```
    # function to add an edge to graph
```

```
    def addEdge(self,u,v):
```

```
        self.graph[u].append(v)
```

```
    # A function used by DFS
```

```
    def DFSUtil(self, v, visited):
```

```
        # Mark the current node as visited and print it
```

```

        visited[v]= True
        print v,

        # Recur for all the vertices adjacent to
        # this vertex
        for i in self.graph[v]:
            if visited[i] == False:
                self.DFSUtil(i, visited)

# The function to do DFS traversal. It uses
# recursive DFSUtil()
def DFS(self):
    V = len(self.graph) #total vertices

    # Mark all the vertices as not visited
    visited =[False]*(V)

    # Call the recursive helper function to print
    # DFS traversal starting from all vertices one
    # by one
    for i in range(V):
        if visited[i] == False:
            self.DFSUtil(i, visited)

```

```

# Driver code
# Create a graph given in the above diagram
g = Graph()
g.addEdge(0, 1)
g.addEdge(0, 2)
g.addEdge(1, 2)
g.addEdge(2, 0)
g.addEdge(2, 3)
g.addEdge(3, 3)

print "Following is Depth First Traversal"
g.DFS()

```

The output of the code would be

Following is Depth First Traversal

0 1 2 3

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

In order to move onto more complex implementations, like previous implementation of BFS, solve the same Travelling Salesman Problem (TSP) given in Figure below using DFS from Arad to Bucharest:

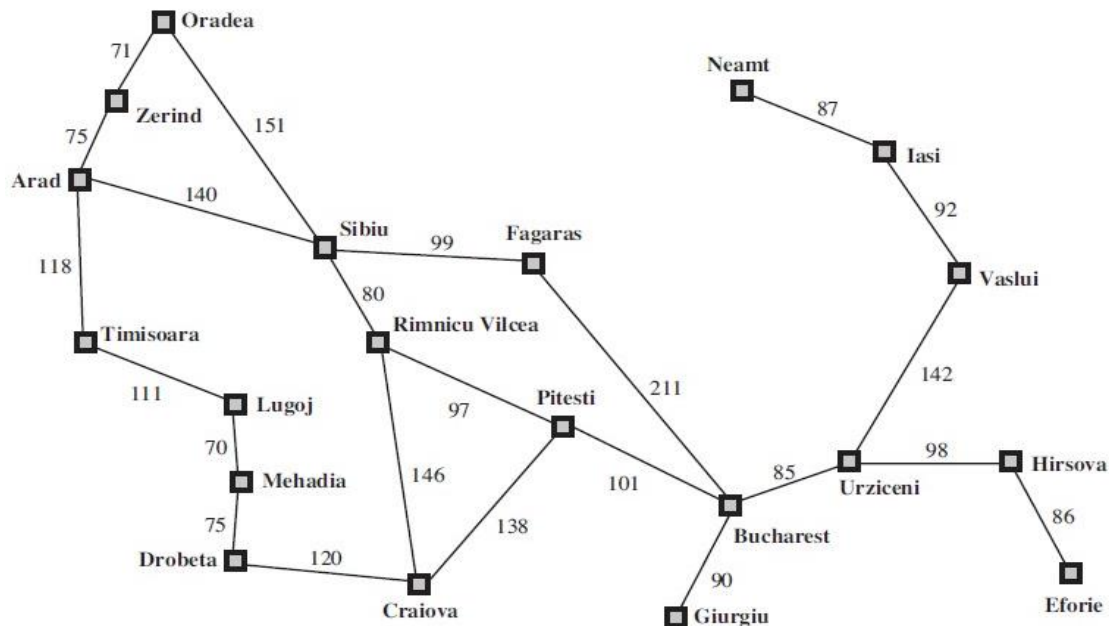


Figure: Graph for Travelling Salesman Problem

Figure: Travelling Salesman Problem

- Draw a corresponding graph from the above Figure from Arad to Bucharest containing all possible nodes. (Arad as Start Node and Bucharest as Goal Node)
- Apply depth first search to reach the goal state, your output should have the path and the total distance to reach the path.
- Save your code and results as DFS_REG#.py as it would be used in later exercises.

Lab Task 2:

Create a graph of the Figure for TSP given above and then set initial and goal states such that the number of nodes visited for BFS is smaller than that in DFS. Now modify the initial and goal state such that the number of nodes visited for BFS is larger than that in DFS.

Lab 05

Uninformed Searching Algorithms

Uniform Cost Search

Objective:

This lab will strengthen the concepts of search techniques being practiced in the previous sessions. In this session focus would be on understanding the Uniform Cost Search and overall comparison of approaches like BFS and DFS.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand uniform cost search technique
- Understand how to solve different path problems using uniform cost technique

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 15 Minutes)

In the previous lab sessions, we have discussed search algorithms like *Breadth-First Search*, and *Depth First Search* to find a solution in a *graph*. The graph represents the given problem, in which each vertex (node) of the graph represents a state of the problem and each edge represents a valid action that takes us from one state (vertex) to the other.

The UCS algorithm uses the path's cost from the initial node to the current node as the extension criterion. Starting from the initial state (starting node), the UCS algorithm, in each step chooses the node that is closer to the initial node. When the algorithm finds the solution, returns the path from the initial state to the final state. The UCS algorithm is characterized as complete, as it always returns a solution if exists. Moreover, the UCS algorithm guarantees the optimum solution.

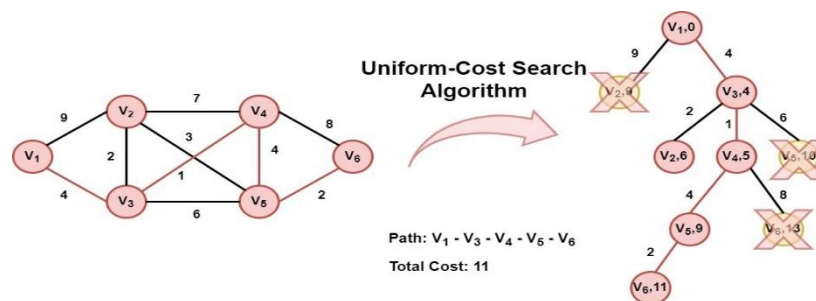


Figure: UCS Algorithm

2) Solved Lab Activities (Allocated Time 1 Hour)

The UCS algorithm takes as inputs the graph along with the starting and the destination nodes and returns the optimum path between these nodes if exists. Similar to the Greedy algorithm, the UCS algorithm uses two lists, the *opened* and the *closed* list. The first list contains the nodes that are possible to be selected and the closed list contained the nodes that have already been selected. Firstly, the first node (initial state) is appended to the opened list (initialization phase). In each step, the node (selected node) with the smallest distance value is removed from the opened list and is appended to the closed list. For each child of the selected node, the algorithm calculates the distance from the first node to this. If the child does not exist in both lists or is in the opened list but with a bigger distance value from the initial node, then the child is appended in the opened list in the position of the corresponding node. The whole process is terminated when a solution is found, or the opened list be empty. The latter situation means that there is not a possible solution to the related problem. The pseudocode of the UCS algorithm is the following:

```
1. function UCS(Graph, start, target):
2.   Add the starting node to the opened list. The node has
3.   has zero distance value from itself
4.   while True:
5.     if opened is empty:
6.       break # No solution found
7.     selecte_node = remove from opened list, the node with
8.       the minimum distance value
9.     if selected_node == target:
10.      calculate path
11.      return path
12.    add selected_node to closed list
13.    new_nodes = get the children of selected_node
14.    if the selected node has children:
15.      for each child in children:
16.        calculate the distance value of child
17.        if child not in closed and opened lists:
18.          child.parent = selected_node
19.          add the child to opened list
20.        else if child in opened list:
21.          if the distance value of child is lower than
22.            the corresponding node in opened list:
23.              child.parent = selected_node
24.              add the child to opened list
```

Activity 1:

In order to understand the concept of UCS it is imperative to understand its working. So let us look at the graph given in Figure below:

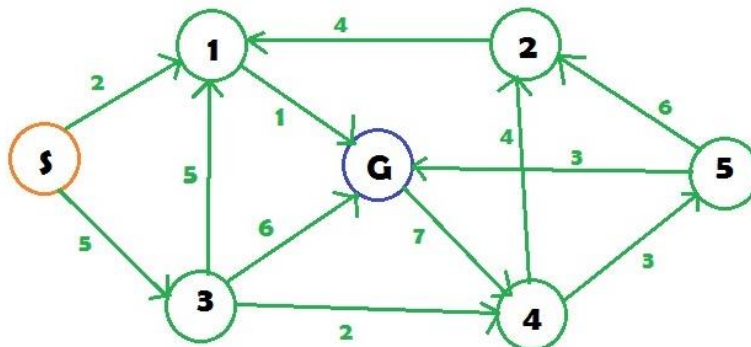


Figure: Graph for UCS Implementation

This version of UCS is useful for graphs which are too large to represent in the memory. The Uniform-Cost Search is mainly used in Artificial Intelligence problems for matching.

In this algorithm from the starting state we will visit the adjacent states and will choose the least costly state then we will choose the next least costly state from the all un-visited and adjacent states of the visited states, in this way we will try to reach the goal state (note we wont continue the path through a goal state), even if we reach the goal state we will continue searching for other possible paths(if there are multiple goals) . We will keep a priority queue which will give the least costliest next state from all the adjacent states of visited states.

Python3 implementation of above approach

```
# returns the minimum cost in a vector( if  
# there are multiple goal states)  
def uniform_cost_search(goal, start):
```

```
    # minimum cost upto  
    # goal state from starting  
    global graph, cost  
    answer = []
```

```
    # create a priority queue  
    queue = []
```

```
    # set the answer vector to max value  
    for i in range(len(goal)):  
        answer.append(10**8)
```

```
# insert the starting index
queue.append([0, start])

# map to store visited node
visited = {}

# count
count = 0

# while the queue is not empty
while (len(queue) > 0):

    # get the top element of the
    queue = sorted(queue)
    p = queue[-1]

    # pop the element
    del queue[-1]

    # get the original value
    p[0] *= -1

    # check if the element is part of
    # the goal list
    if (p[1] in goal):

        # get the position
        index = goal.index(p[1])

        # if a new goal is reached
        if (answer[index] == 10**8):
            count += 1

        # if the cost is less
        if (answer[index] > p[0]):
            answer[index] = p[0]

    # pop the element
    del queue[-1]

    queue = sorted(queue)
```

```

        if (count == len(goal)):
            return answer

    # check for the non visited nodes
    # which are adjacent to present node
    if (p[1] not in visited):
        for i in range(len(graph[p[1]])):

            # value is multiplied by -1 so that
            # least priority is at the top
            queue.append( [(p[0] + cost[(p[1], graph[p[1]][i])])* -1,
graph[p[1]][i]])

            # mark as visited
            visited[p[1]] = 1

    return answer

# main function
if __name__ == '__main__':

    # create the graph
    graph, cost = [[] for i in range(8)], {}

    # add edge
    graph[0].append(1)
    graph[0].append(3)
    graph[3].append(1)
    graph[3].append(6)
    graph[3].append(4)
    graph[1].append(6)
    graph[4].append(2)
    graph[4].append(5)
    graph[2].append(1)
    graph[5].append(2)
    graph[5].append(6)
    graph[6].append(4)

    # add the cost
    cost[(0, 1)] = 2
    cost[(0, 3)] = 5
    cost[(1, 6)] = 1

```

```

cost[(3, 1)] = 5
cost[(3, 6)] = 6
cost[(3, 4)] = 2
cost[(2, 1)] = 4
cost[(4, 2)] = 4
cost[(4, 5)] = 3
cost[(5, 2)] = 6
cost[(5, 6)] = 3
cost[(6, 4)] = 7

# goal state
goal = []

# set the goal
# there can be multiple goal states
goal.append(6)

# get the answer
answer = uniform_cost_search(goal, 0)

# print the answer
print("Minimum cost from 0 to 6 is = ",answer[0])

```

The output in this case would be:

```

Python 3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\hanzalah\Downloads\ASTAR-REV (3).py =====
Path found: ['A', 'F', 'G', 'I', 'E']
>>>
===== RESTART: C:/Users/hanzalah/Downloads/djiskatra.py =====
Vertex Distance from Source
0         0
1         4
2        12
3        19
4        21
5        11
6         9
7         8
8        14
>>>
===== RESTART: C:/Users/hanzalah/Downloads/UCS2.py =====
Minimum cost from 0 to 6 is = 3
>>>

```

Activity 2:

In this case we were given the start and goal states, however most of the times when we are dealing with unknown datasets it is difficult to ascertain the start node. In this case we use different type of data searches like linear , binary , and exponential depending on the complexity of the data.

The linear search is the most used for small datasets.

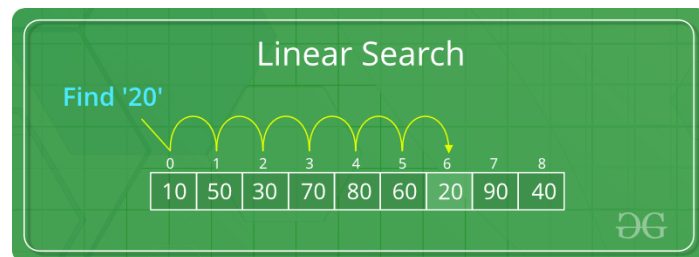


Figure: Linear Search

```
# Python3 code to linearly search x in arr[].  
# If x is present then return its location,  
# otherwise return -1
```

```
def search(arr, n, x):
```

```
    for i in range(0, n):  
        if (arr[i] == x):  
            return i  
    return -1
```

```
# Driver Code
```

```
arr = [2, 3, 4, 10, 40]
```

```
x = 10
```

```
n = len(arr)
```

```
# Function call
```

```
result = search(arr, n, x)
```

```
if(result == -1):
```

```
    print("Element is not present in array")
```

```
else:
```

```
    print("Element is present at index", result)
```

For The above search, if we are to find 10 from the array, it would give the following output.

Element is present at index 3

However, the time complexity would increase in case the element was present at the last column of the array. For this reason certain optimizations functions need to be applied.

Activity 3:

In order to improve the time complexity let us apply the following optimization.

In case the element Found at last $O(n)$ to $O(1)$

It is the same as previous method because here we are performing two 'if' operations in one iteration of the loop and in previous method we performed only 1 'if' operation. This makes both the time complexities same.

Below is the implementation:

```
# Python3 program for linear search
def search(arr, search_Element):
    left = 0
    length = len(arr)
    position = -1
    right = length - 1

    # Run loop from 0 to right
    for left in range(0, right, 1):

        # If search_element is found with
        # left variable
        if (arr[left] == search_Element):
            position = left
            print("Element found in Array at ", position +
                  1, " Position with ", left + 1, " Attempt")
            break

        # If search_element is found with
        # right variable
        if (arr[right] == search_Element):
            position = right
            print("Element found in Array at ", position + 1,
                  " Position with ", length - right, " Attempt")
            break
    left += 1
```

```

        right -= 1

    # If element not found
    if (position == -1):
        print("Not found in Array with ", left, " Attempt")

# Driver code
arr = [1, 2, 3, 4, 5]
search_element = 5

# Function call
search(arr, search_element)

```

The output in this case would be:

```

Python 3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\hanzalah\Downloads\ASTAR-REV (3).py =====
Path found: ['A', 'F', 'G', 'I', 'E']
>>>
===== RESTART: C:/Users/hanzalah/Downloads/djiskatra.py =====
Vertex  Distance from Source
0        0
1        4
2       12
3       19
4       21
5       11
6        9
7        8
8       14
>>>
===== RESTART: C:/Users/hanzalah/Downloads/UCS2.py =====
Minimum cost from 0 to 6 is = 3
>>>
===== RESTART: C:/Users/hanzalah/Downloads/expsearch.py =====
Element is present at index 3
>>>
===== RESTART: C:/Users/hanzalah/Downloads/linear-opt.py =====
Element found in Array at 5 Position with 1 Attempt
>>>

```

The above showcases some optimizations and in-built search operations that can be applied as a pre-processing step for small to large dataset problems.

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

In order to move onto more complex implementations, like previous implementation of BFS and DFS, solve the same Travelling Salesman Problem (TSP) given in Figure below using UCS from Arad to Bucharest:

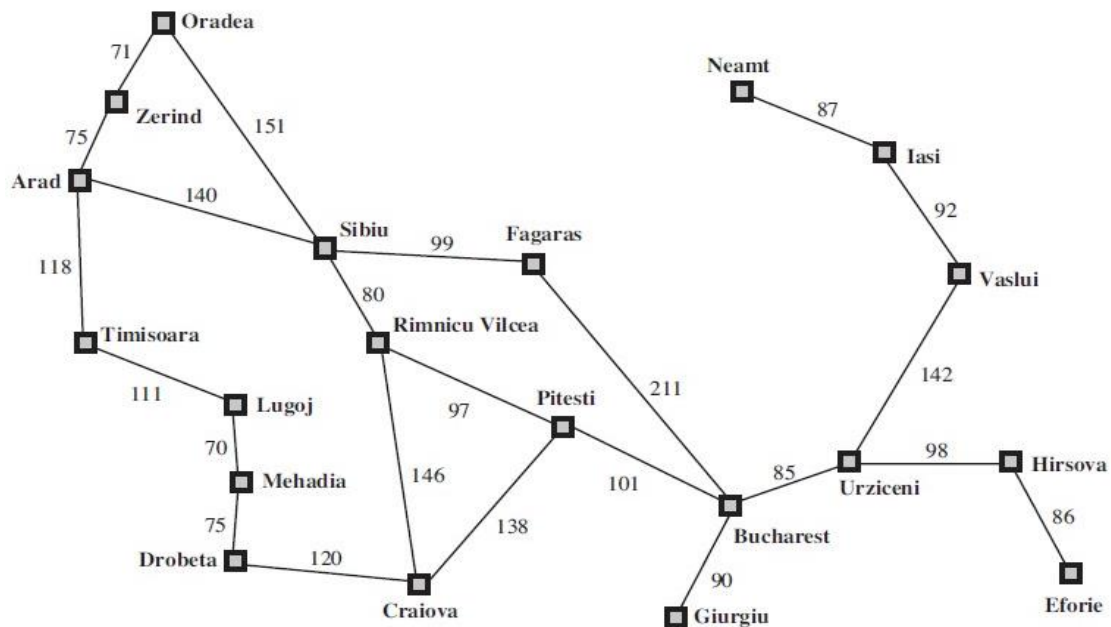


Figure: Graph for Travelling Salesman Problem

Lab Task 2:

Run memory allocation tests to see which one of the implementations from BFS , DFS and UCS takes the lowest amount of memory.

Lab 06

Greedy Search Algorithms

Objective:

This lab will focus on understanding Greedy approaches. The greedy searches are an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit. In this case the problems where choosing the locally optimal also lead to global solution are best fit for Greedy methodology.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand different types of Greedy search approaches
- Understand how to solve different path problems using greedy search techniques

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 15 Minutes)

A greedy algorithm selects a candidate greedily (local optimum) and adds it to the current solution provided that it doesn't corrupt the feasibility. If the solution obtained by above step is not final, repeat till global optimum or the final solution is obtained.

Although there are several mathematical strategies available to proof the correctness of Greedy Algorithms, we will try to proof it intuitively and use method of contradiction.

Greedy Algorithm usually involves a sequence of choices. The Greedy algorithms can't backtrack, hence once they make a choice, they're committed to it. So, it's critical that they never make a bad choice.

Suppose S be the solution obtained by applying greedy algorithm to a problem and O be the optimum solution to the problem. If both S and O are same, then our algorithm is by default correct.

If S and O are different then clearly while stacking up various local solutions for the problem we made a mistake and chose a less efficient solution which resulted in S rather than O as a solution.

But according to the definition of greedy algorithms we always choose the local optimum solution. Hence using proof by contradiction, it can be said that greedy algorithm gives the correct solution.

2) Solved Lab Activities (Allocated Time 1 Hour)

For example, consider the Fractional Knapsack Problem. The local optimal strategy is to choose the item that has maximum value vs weight ratio. This strategy also leads to global optimal solution because we are allowed to take fractions of an item.

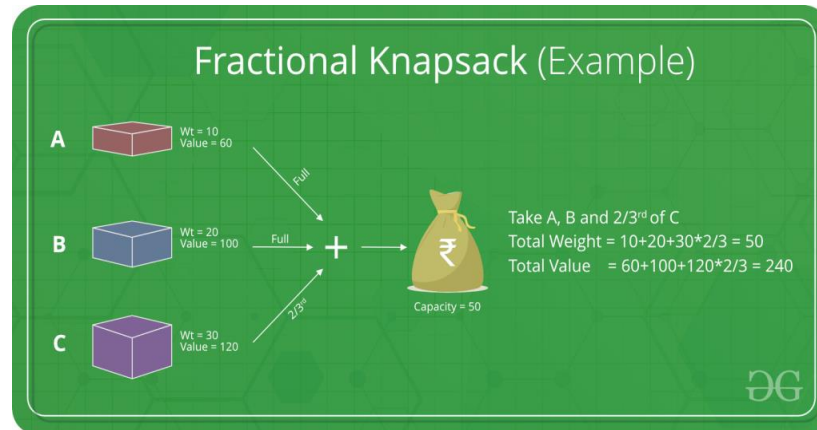


Figure: Fractional knapsack Problem

Activity 1:

In order to understand the greedy approaches, let us take the fractional knapsack example given in the figure above. Given the weights and values of n items, we need to put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

In the 0-1 Knapsack problem, we are not allowed to break items. We either take the whole item or don't take it. However, in Fractional Knapsack, we can break items for maximizing the total value of knapsack. This problem in which we can break an item is also called the fractional knapsack problem.

An efficient solution is to use the Greedy approach. The basic idea of the greedy approach is to calculate the ratio value/weight for each item and sort the item on basis of this ratio. Then take the item with the highest ratio and add them until we can't add the next item as a whole and at the end add the next item as much as we can. Which will always be the optimal solution to this problem.

A simple code with our own comparison function can be written as follows, please see the sorting function more closely, the third argument to sort function is our comparison function which sorts the item according to value/weight ratio in non-decreasing order.

After sorting we need to loop over these items and add them in our knapsack satisfying above-mentioned criteria.

```
# Python3 program to solve fractional  
# Knapsack Problem
```

```
class ItemValue:
```

```
    """Item Value DataClass"""
```

```
    def __init__(self, wt, val, ind):  
        self.wt = wt  
        self.val = val  
        self.ind = ind  
        self.cost = val // wt
```

```
    def __lt__(self, other):  
        return self.cost < other.cost
```

```
# Greedy Approach
```

```
class FractionalKnapSack:
```

```
    """Time Complexity  $O(n \log n)$ """
```

```
    @staticmethod
```

```
    def getMaxValue(wt, val, capacity):
```

```
        """function to get maximum value """
```

```
        iVal = []
```

```
        for i in range(len(wt)):
```

```
            iVal.append(ItemValue(wt[i], val[i], i))
```

```
        # sorting items by value
```

```
        iVal.sort(reverse=True)
```

```
        totalValue = 0
```

```
        for i in iVal:
```

```
            curWt = int(i.wt)
```

```
            curVal = int(i.val)
```

```
            if capacity - curWt >= 0:
```

```
                capacity -= curWt
```

```
                totalValue += curVal
```

```
            else:
```

```

        fraction = capacity / curWt
        totalValue += curVal * fraction
        capacity = int(capacity - (curWt * fraction))
        break
    return totalValue

# Driver Code
if __name__ == "__main__":
    wt = [10, 40, 20, 30]
    val = [60, 40, 100, 120]
    capacity = 50

    # Function call
    maxValue = FractionalKnapsack.getMaxValue(wt, val, capacity)
    print("Maximum value in Knapsack =", maxValue)

```

The output in this case would be:

Maximum possible value = 240

By taking full items of 10 kg, 20 kg and
2/3rd of last item of 30 kg

Activity 2:

Let us try experimenting with another greedy approach using the “Policeman catches thieves” problem.

Where a Given an array of size n that has the following specifications:

- Each element in the array contains either a policeman or a thief.
- Each policeman can catch only one thief.
- A policeman cannot catch a thief who is more than K units away from the policeman.
- We need to find the maximum number of thieves that can be caught.

For example if Input : arr[] = {'P', 'T', 'T', 'P', 'T'},

k = 1.

Output : 2.

Here maximum 2 thieves can be caught, the first policeman catches the first thief and second policeman can catch either second or third thief.

A brute force approach would be to check all feasible sets of combinations of police and thief and return the maximum size set among them. Its time complexity is exponential, and it can be optimized if we observe an important property.

An efficient solution is to use a greedy algorithm. But which greedy property to use can be tricky. We can try using: “For each policeman from the left catch the nearest possible thief.” This works for example three given above but fails for example two as it outputs 2 which is incorrect.

We may also try: “For each policeman from the left catch the farthest possible thief”. This works for example two given above but fails for example three as it outputs 2 which is incorrect. A symmetric argument can be applied to show that traversing for these from the right side of the array also fails. We can observe that thinking irrespective of the policeman and focusing on just the allotment works:

- Get the lowest index of policeman p and thief t. Make an allotment
- if $|p-t| \leq k$ and increment to the next p and t found.
- Otherwise increment $\min(p, t)$ to the next p or t found.
- Repeat above two steps until next p and t are found.
- Return the number of allotments made.

Below is the implementation of the above algorithm. It uses vectors to store the indices of police and thief in the array and processes them.

```
# Python3 program to find maximum
# number of thieves caught

# Returns maximum number of thieves
# that can be caught.
def policeThief(arr, n, k):
    i = 0
    l = 0
    r = 0
    res = 0
    thi = []
    pol = []

    # store indices in list
    while i < n:
        if arr[i] == 'P':
            pol.append(i)
        elif arr[i] == 'T':
            thi.append(i)
        i += 1

    # track lowest current indices of
    # thief: thi[l], police: pol[r]
```

```

while l < len(thi) and r < len(pol):

    # can be caught
    if (abs( thi[l] - pol[r] ) <= k):
        res += 1
        l += 1
        r += 1

    # increment the minimum index
    elif thi[l] < pol[r]:
        l += 1
    else:
        r += 1

return res

# Driver program
if __name__ == '__main__':
    arr1 = ['P', 'T', 'T', 'P', 'T']
    k = 2
    n = len(arr1)
    print(("Maximum thieves caught: {}".
          format(policeThief(arr1, n, k))))

    arr2 = ['T', 'T', 'P', 'P', 'T', 'P']
    k = 2
    n = len(arr2)
    print(("Maximum thieves caught: {}".
          format(policeThief(arr2, n, k))))

    arr3 = ['P', 'T', 'P', 'T', 'T', 'P']
    k = 3
    n = len(arr3)
    print(("Maximum thieves caught: {}".
          format(policeThief(arr3, n, k))))

```

The output in this case would be:

Maximum thieves caught: 2

Maximum thieves caught: 3

Maximum thieves caught: 3

Activity 3

Finding elements in mixed datasets is a complicated problem in AI. In the previous session we looked at some linear implementations to find elements. Since we are now experimenting with more complex search problems in the greedy paradigm, it is worth exploring the binary and exponential search as well.

In binary search we basically ignore half of the elements just after one comparison.

- Compare x with the middle element.
- If x matches with the middle element, we return the mid index.
- Else If x is greater than the mid element, then x can only lie in the right half subarray after the mid element. So we recur for the right half.
- Else (x is smaller) recur for the left half.



Figure: Binary Search Algorithm

A basic implementation of the binary search is presented below:

```
# Python3 Program for recursive binary search.

# Returns index of x in arr if present, else -1

def binarySearch(arr, l, r, x):
    # Check base case
    if r >= l:
        mid = l + (r - l) // 2

        # If element is present at the middle itself
        if arr[mid] == x:
            return mid

        # If element is smaller than mid, then it
        # can only be present in left subarray
        elif arr[mid] > x:
            return binarySearch(arr, l, mid-1, x)
```

```
        # Else the element can only be present # in right subarray
    else:
        return binarySearch(arr, mid + 1, r, x)

    else:
        # Element is not present in the array
        return -1

# Driver Code
arr = [2, 3, 4, 10, 40]
x = 10

# Function call
result = binarySearch(arr, 0, len(arr)-1, x)

if result != -1:
    print("Element is present at index % d" % result)
else:
    print("Element is not present in array")
```

The output in this case would be:

Element is present at index 3

Activity 4

Every positive fraction can be represented as sum of unique unit fractions. A fraction is unit fraction if numerator is 1 and denominator is a positive integer, for example $1/3$ is a unit fraction. Such a representation is called Egyptian Fraction as it was used by ancient Egyptians.

Egyptian Fraction Representation of $2/3$ is $1/2 + 1/6$

Egyptian Fraction Representation of $6/14$ is $1/3 + 1/11 + 1/231$

Egyptian Fraction Representation of $12/13$ is $1/2 + 1/3 + 1/12 + 1/156$

We can generate Egyptian Fractions using Greedy Algorithm. For a given number of the form ' nr/dr ' where $dr > nr$, first find the greatest possible unit fraction, then recur for the remaining part. For example, consider $6/14$, we first find ceiling of $14/6$, i.e., 3. So the first unit fraction becomes $1/3$, then recur for $(6/14 - 1/3)$ i.e., $4/42$.

```
# Python3 program to print a fraction
# in Egyptian Form using Greedy
# Algorithm

# import math package to use
# ceiling function

import math

def getEgyptianFraction(numerator,denominator):
    str = ""
    output = getEgyptianFractionUtil(numerator,denominator,[])
    for denom in output:
        str += "1/{0} + ".format(denom);
    strCopy = str[:-3] #removing the last + sign
    return strCopy

def getEgyptianFractionUtil(numerator,denominator,listOfDenoms):
    if numerator == 0:
        return listOfDenoms
    newDenom = math.ceil(denominator/numerator);
    #append in output list
    listOfDenoms.append(newDenom);
    listOfDenoms = getEgyptianFractionUtil(numerator*newDenom - denominator,
                                           newDenom*denominator,listOfDenoms);

    return listOfDenoms

print(getEgyptianFraction(6,14))
```

The output in this case would be:

Egyptian Fraction Representation of $6/14$ is

$1/3 + 1/11 + 1/231$

The Greedy algorithm works because a fraction is always reduced to a form where denominator is greater than numerator and numerator doesn't divide denominator. For such reduced forms, the highlighted recursive call is made for reduced numerator. So, the recursive calls keep on reducing the numerator till it reaches 1.

4) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

We have discussed a scalar implementation of the binary search in this lab exercise, implement a 2D version of the binary search to find elements:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

Figure: 2D Binary Search

Lab Task 2:

Implement a 2D version of the Policeman catches thieves' problem discussed above.

Lab Task 3:

Prove that the fractional knapsack problem offers better time complexity than other searching methods.

Lab 07

Greedy Algorithms: Cyclic Path Problems

Objective:

This lab will focus on divulging further into Greedy approaches looking into minimum spanning trees and finding cycles in undirected graphs. Given a connected and undirected graph, a spanning tree of that graph is a subgraph that is a tree and connects all the vertices together. A single graph can have many different spanning trees. A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected, undirected graph is a spanning tree with a weight less than or equal to the weight of every other spanning tree. The weight of a spanning tree is the sum of weights given to each edge of the spanning tree. We will use the concept of MSTs to understand their applications in the field of AI.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand different types of Greedy algorithms for minimum spanning trees
- Understand how to solve different path problems using minimum spanning trees
- Understand how to solve shortest path problems

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 15 Minutes)

Minimum Spanning Trees (MST) is a fundamental problem with diverse applications in AI based systems. Although we would be covering the basic implementation of MSTs in our data structures course, but it is imperative understand their implementations from an AI perspective as well.

Some applications of MST include:

Network design

The standard application is to a problem like phone network design. You have a business with several offices; you want to lease phone lines to connect them up with each other; and the phone company charges different amounts of money to connect different pairs of cities. You want a set of lines that connects all your offices with a minimum total cost. It should be a spanning tree, since if a network isn't a tree you can always remove some edges and save money.

Approximation algorithms for NP-hard problems.

A less obvious application is that the minimum spanning tree can be used to approximately solve the traveling salesman problem. A convenient formal way of defining this problem is to find the shortest path that visits each point at least once.

On the other hand, if you draw a path tracing around the minimum spanning tree, you trace each edge twice and visit all points, so the TSP weight is less than twice the MST weight. Therefore, this tour is within a factor of two of optimal.

Cluster analysis

A clustering problem can be viewed as finding an MST and deleting the $k-1$ most expensive edges.

2) Solved Lab Activities (Allocated Time 1 Hour)

Below are the steps for finding MST using a method known as Kruskal's algorithm which we should have studied in detail for our data structure course. However, in this exercise we will look at implementation of minimum spanning trees to solve AI problems of cyclic paths. In cyclic paths if we get stuck then it is impossible to reach a goal state.

Activity 1:

A disjoint-set data structure is a data structure that keeps track of a set of elements partitioned into several disjoint (non-overlapping) subsets. A union-find algorithm is an algorithm that performs two useful operations on such a data structure:

Find: Determine which subset a particular element is in. This can be used for determining if two elements are in the same subset.

Union: Join two subsets into a single subset. Here first we have to check if the two subsets belong to same set. If no, then we cannot perform union.

For this activity, the application of Disjoint Set Data Structure will be discussed. The application is to check whether a given graph contains a cycle or not.

The Union-Find Algorithm can be used to check whether an undirected graph contains cycle or not. This method assumes that the graph doesn't contain any self-loops. Consider Figure for a subset of graph given below:

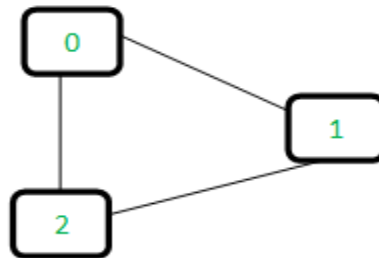


Figure: Graph for Union Find Algorithm

For each edge, make subsets using both the vertices of the edge. If both the vertices are in the same subset, a cycle is found.

The implementation can be given as:

```
# Python Program for union-find algorithm to detect cycle in a undirected graph
# we have one egde for any two vertex i.e 1-2 is either 1-2 or 2-1 but not both
```

```
from collections import defaultdict
```

```
#This class represents a undirected graph using adjacency list representation
```

```
class Graph:
```

```
    def __init__(self,vertices):
        self.V= vertices #No. of vertices
        self.graph = defaultdict(list) # default dictionary to store graph
```

```
    # function to add an edge to graph
```

```
    def addEdge(self,u,v):
        self.graph[u].append(v)
```

```
    # A utility function to find the subset of an element i
```

```
    def find_parent(self, parent,i):
        if parent[i] == -1:
            return i
        if parent[i] != -1:
            return self.find_parent(parent,parent[i])
```

```

# A utility function to do union of two subsets
def union(self,parent,x,y):
    parent[x] = y

# The main function to check whether a given graph
# contains cycle or not
def isCyclic(self):

    # Allocate memory for creating V subsets and
    # Initialize all subsets as single element sets
    parent = [-1]*(self.V)

    # Iterate through all edges of graph, find subset of both
    # vertices of every edge, if both subsets are same, then
    # there is cycle in graph.
    for i in self.graph:
        for j in self.graph[i]:
            x = self.find_parent(parent, i)
            y = self.find_parent(parent, j)
            if x == y:
                return True
            self.union(parent,x,y)

# Create a graph given in the above diagram
g = Graph(3)
g.addEdge(0, 1)
g.addEdge(1, 2)
g.addEdge(2, 0)

if g.isCyclic():
    print ("Graph contains cycle")
else :
    print ("Graph does not contain cycle ")

```

The output is given as:

Graph contains cycle

Activity 2:

The above union() and find() are naive and the worst case time complexity is linear. The trees created to represent subsets can be skewed and can become like a linked list.

Following is an example worst case scenario.

Let there be 4 elements 0, 1, 2, 3

Initially, all elements are single element subsets.

0 1 2 3

Do Union(0, 1)

1 2 3

/

0

Do Union(1, 2)

2 3

/

1

/

0

Do Union(2, 3)

3

/

2

/

1

/

0

The second optimization to naive method is Path Compression. The idea is to flatten the tree when find() is called. When find() is called for an element x, root of the tree is returned. The find() operation traverses up from x to find root. The idea of path compression is to make the found root as parent of x so that we don't have to traverse all intermediate nodes again.

If x is root of a subtree, then path (to root) from all nodes under x also compresses.

```
# A union by rank and path compression based
# program to detect cycle in a graph
from collections import defaultdict

# a structure to represent a graph

class Graph:

    def __init__(self, num_of_v):
        self.num_of_v = num_of_v
        self.edges = defaultdict(list)

    # graph is represented as an
    # array of edges
    def add_edge(self, u, v):
        self.edges[u].append(v)


class Subset:

    def __init__(self, parent, rank):
        self.parent = parent
        self.rank = rank

# A utility function to find set of an element
# node(uses path compression technique)

def find(subsets, node):
    if subsets[node].parent != node:
        subsets[node].parent = find(subsets, subsets[node].parent)
    return subsets[node].parent

# A function that does union of two sets
# of u and v(uses union by rank)

def union(subsets, u, v):

    # Attach smaller rank tree under root
    # of high rank tree(Union by Rank)
    if subsets[u].rank > subsets[v].rank:
```

```

        subsets[v].parent = u
    elif subsets[v].rank > subsets[u].rank:
        subsets[u].parent = v

    # If ranks are same, then make one as
    # root and increment its rank by one
    else:
        subsets[v].parent = u
        subsets[u].rank += 1

# The main function to check whether a given
# graph contains cycle or not

def isCycle(graph):

    # Allocate memory for creating sets
    subsets = []

    for u in range(graph.num_of_v):
        subsets.append(Subset(u, 0))

    # Iterate through all edges of graph,
    # find sets of both vertices of every
    # edge, if sets are same, then there
    # is cycle in graph.
    for u in graph.edges:
        u_rep = find(subsets, u)

        for v in graph.edges[u]:
            v_rep = find(subsets, v)

            if u_rep == v_rep:
                return True
            else:
                union(subsets, u_rep, v_rep)

# Driver Code
g = Graph(3)

```

```

# add edge 0-1
g.add_edge(0, 1)

# add edge 1-2
g.add_edge(1, 2)

# add edge 0-2
g.add_edge(0, 2)

if isCycle(g):
    print('Graph contains cycle')
else:
    print('Graph does not contain cycle')

```

The output would be the same, however try running Tracealloc to see the difference in terms of computation.

Graph contains cycle

Activity 3:

Now let us move to implementation the Kruskal's Minimum Spanning Tree Algorithm.

The steps are as follows:

1. Sort all the edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If cycle is not formed, include this edge. Else, discard it.
3. Repeat step#2 until there are $(V-1)$ edges in the spanning tree.

In the first instance we need to sort out the vertices of the graph as per their weights. In this case let us consider the graph given in Figure below:

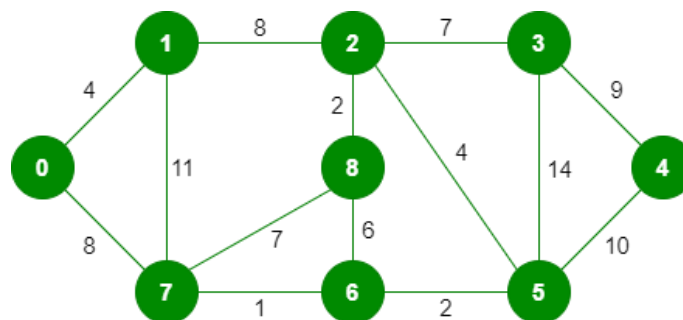


Figure: Graph for Kruskal's Algorithm

The graph contains 9 vertices and 14 edges. So, the minimum spanning tree formed will be having $(9 - 1)$ = 8 edges.

After applying sorting we get:

Weight	Src	Dest
--------	-----	------

1	7	6
---	---	---

2	8	2
---	---	---

2	6	5
---	---	---

4	0	1
---	---	---

4	2	5
---	---	---

6	8	6
---	---	---

7	2	3
---	---	---

7	7	8
---	---	---

8	0	7
---	---	---

8	1	2
---	---	---

9	3	4
---	---	---

10	5	4
----	---	---

11	1	7
----	---	---

14	3	5
----	---	---

Now apply following implementation of Kruskal's algorithm:

```
# Python program for Kruskal's algorithm to find
# Minimum Spanning Tree of a given connected,
# undirected and weighted graph
```

```
from collections import defaultdict
```

```
# Class to represent a graph
```

```
class Graph:
```

```
    def __init__(self, vertices):
        self.V = vertices # No. of vertices
        self.graph = [] # default dictionary
                          # to store graph
```

```
    # function to add an edge to graph
```

```
    def addEdge(self, u, v, w):
        self.graph.append([u, v, w])
```

```

# A utility function to find set of an element i
# (uses path compression technique)
def find(self, parent, i):
    if parent[i] == i:
        return i
    return self.find(parent, parent[i])

# A function that does union of two sets of x and y
# (uses union by rank)
def union(self, parent, rank, x, y):
    xroot = self.find(parent, x)
    yroot = self.find(parent, y)

    # Attach smaller rank tree under root of
    # high rank tree (Union by Rank)
    if rank[xroot] < rank[yroot]:
        parent[xroot] = yroot
    elif rank[xroot] > rank[yroot]:
        parent[yroot] = xroot

    # If ranks are same, then make one as root
    # and increment its rank by one
    else:
        parent[yroot] = xroot
        rank[xroot] += 1

# The main function to construct MST using Kruskal's
# algorithm
def KruskalMST(self):

    result = [] # This will store the resultant MST

    # An index variable, used for sorted edges
    i = 0

    # An index variable, used for result[]
    e = 0

    # Step 1: Sort all the edges in
    # non-decreasing order of their
    # weight. If we are not allowed to change the
    # given graph, we can create a copy of graph

```

```

self.graph = sorted(self.graph,
                    key=lambda item: item[2])

parent = []
rank = []

# Create V subsets with single elements
for node in range(self.V):
    parent.append(node)
    rank.append(0)

# Number of edges to be taken is equal to V-1
while e < self.V - 1:

    # Step 2: Pick the smallest edge and increment
    # the index for next iteration
    u, v, w = self.graph[i]
    i = i + 1
    x = self.find(parent, u)
    y = self.find(parent, v)

    # If including this edge doesn't
    # cause cycle, include it in result
    # and increment the index of result
    # for next edge
    if x != y:
        e = e + 1
        result.append([u, v, w])
        self.union(parent, rank, x, y)
    # Else discard the edge

minimumCost = 0
print ("Edges in the constructed MST")
for u, v, weight in result:
    minimumCost += weight
    print("%d -- %d == %d" % (u, v, weight))
print("Minimum Spanning Tree" , minimumCost)

# Driver code
g = Graph(4)
g.addEdge(0, 1, 10)
g.addEdge(0, 2, 6)

```

```
g.addEdge(0, 3, 5)
g.addEdge(1, 3, 15)
g.addEdge(2, 3, 4)

# Function call
g.KruskalMST()
```

The output would be given as:

Following are the edges in the constructed MST

```
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
```

Minimum Cost Spanning Tree: 19

Activity 4

Similar to Kruskal's algorithm, Prim's algorithm is also a Greedy algorithm. It starts with an empty spanning tree. The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, the other set contains the vertices not yet included. At every step, it considers all the edges that connect the two sets, and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.

A group of edges that connects two set of vertices in a graph is called cut in graph theory. So, at every step of Prim's algorithm, we find a cut (of two sets, one contains the vertices already included in MST and other contains rest of the vertices), pick the minimum weight edge from the cut and include this vertex to MST Set (the set that contains already included vertices).

The idea behind Prim's algorithm is simple, a spanning tree means all vertices must be connected. So the two disjoint subsets (discussed above) of vertices must be connected to make a Spanning Tree. And they must be connected with the minimum weight edge to make it a Minimum Spanning Tree.

- 1) Create a set mstSet that keeps track of vertices already included in MST.
- 2) Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign key value as 0 for the first vertex so that it is picked first.
- 3) While mstSet doesn't include all vertices
 -a) Pick a vertex u which is not there in mstSet and has minimum key value.
 -b) Include u to mstSet.

....c) Update key value of all adjacent vertices of u. To update the key values, iterate through all adjacent vertices. For every adjacent vertex v, if weight of edge u-v is less than the previous key value of v, update the key value as weight of u-v

The idea of using key values is to pick the minimum weight edge from cut. The key values are used only for vertices which are not yet included in MST, the key value for these vertices indicate the minimum weight edges connecting them to the set of vertices included in MST.

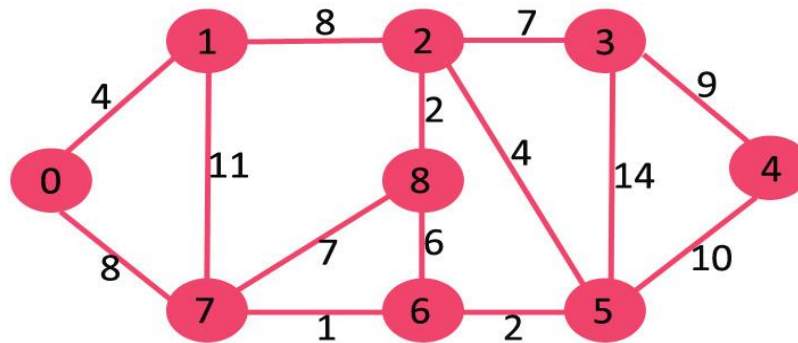


Figure: Graph for Prim's Algorithm

The implementation of the Prim is given below:

```
# A Python program for Prim's Minimum Spanning Tree (MST) algorithm.
# The program is for adjacency matrix representation of the graph

import sys # Library for INT_MAX

class Graph():

    def __init__(self, vertices):
        self.V = vertices
        self.graph = [[0 for column in range(vertices)]
                       for row in range(vertices)]

    # A utility function to print the constructed MST stored in parent[]
    def printMST(self, parent):
        print("Edge \tWeight")
        for i in range(1, self.V):
            print(parent[i], "-", i, "\t", self.graph[i][parent[i]])

    # A utility function to find the vertex with
    # minimum distance value, from the set of vertices
    # not yet included in shortest path tree
    def minKey(self, key, mstSet):
```

```

# Initialize min value
min = sys.maxsize

for v in range(self.V):
    if key[v] < min and mstSet[v] == False:
        min = key[v]
        min_index = v

return min_index

# Function to construct and print MST for a graph
# represented using adjacency matrix representation
def primMST(self):

    # Key values used to pick minimum weight edge in cut
    key = [sys.maxsize] * self.V
    parent = [None] * self.V # Array to store constructed MST
    # Make key 0 so that this vertex is picked as first vertex
    key[0] = 0
    mstSet = [False] * self.V

    parent[0] = -1 # First node is always the root of

    for cout in range(self.V):

        # Pick the minimum distance vertex from
        # the set of vertices not yet processed.
        # u is always equal to src in first iteration
        u = self.minKey(key, mstSet)

        # Put the minimum distance vertex in
        # the shortest path tree
        mstSet[u] = True

        # Update dist value of the adjacent vertices
        # of the picked vertex only if the current
        # distance is greater than new distance and
        # the vertex is not in the shortest path tree
        for v in range(self.V):

            # graph[u][v] is non zero only for adjacent vertices of u
            # mstSet[v] is false for vertices not yet included in MST

```

```

        # Update the key only if graph[u][v] is smaller than key[v]
        if self.graph[u][v] > 0 and mstSet[v] == False and key[v] >
self.graph[u][v]:
            key[v] = self.graph[u][v]
            parent[v] = u

    self.printMST(parent)

g = Graph(5)
g.graph = [ [0, 2, 0, 6, 0],
            [2, 0, 3, 8, 5],
            [0, 3, 0, 0, 7],
            [6, 8, 0, 0, 9],
            [0, 5, 7, 9, 0]]

g.primMST();

```

The output is given as:

Edge	Weight
0 - 1	2
1 - 2	3
0 - 3	6
1 - 4	5

Activity 5

A Hamiltonian path is defined as the path in a directed or undirected graph which visits only once the each vertex of the graph.

Given an adjacency matrix `adj[][]` of an undirected graph consisting of `N` vertices, the task is to find whether the graph contains a Hamiltonian Path or not. If found to be true, then print “Yes”. Otherwise, print “No”.

For example:

Input: $adj[][] = \{\{0, 1, 1, 1, 0\}, \{1, 0, 1, 0, 1\}, \{1, 1, 0, 1, 1\}, \{1, 0, 1, 0, 0\}\}$

Output: Yes

Explanation:

There exists a Hamiltonian Path for the given graph as shown in the figure below:

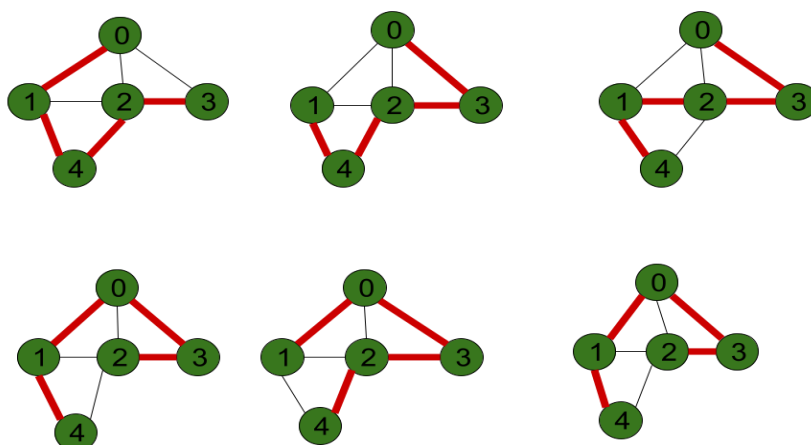


Figure: Graph for Hamiltonian Path Algorithm

Input: $adj[][] = \{\{0, 1, 0, 0\}, \{1, 0, 1, 1\}, \{0, 1, 0, 0\}, \{0, 1, 0, 0\}\}$

Output: No

The simplest approach to solve the given problem is to generate all the possible permutations of N vertices. For each permutation, check if it is a valid Hamiltonian path by checking if there is an edge between adjacent vertices or not. If found to be true, then print “Yes”. Otherwise, print “No”.

However, the above approach can be optimized by using Dynamic Programming and Bit Masking which is based on the following observations:

- The idea is such that for every subset S of vertices, check whether there is a hamiltonian path in the subset S that ends at vertex v where $v \in S$.
- If v has a neighbor u , where $u \in S - \{v\}$, therefore, there exists a Hamiltonian path that ends at vertex u .
- The problem can be solved by generalizing the subset of vertices and the ending vertex of the Hamiltonian path.

The implementation can be given as:

```
# Python3 program for the above approach

# Function to check whether there
# exists a Hamiltonian Path or not
def Hamiltonian_path(adj, N):

    dp = [[False for i in range(1 << N)]
           for j in range(N)]

    # Set all dp[i][(1 << i)] to
    # true
    for i in range(N):
        dp[i][1 << i] = True

    # Iterate over each subset
    # of nodes
    for i in range(1 << N):
        for j in range(N):

            # If the jth nodes is included
            # in the current subset
            if ((i & (1 << j)) != 0):

                # Find K, neighbour of j
                # also present in the
                # current subset
                for k in range(N):
                    if ((i & (1 << k)) != 0 and
                        adj[k][j] == 1 and
                        j != k and
                        dp[k][i ^ (1 << j)]):

                        # Update dp[j][i]
                        # to true
                        dp[j][i] = True
                        break

    # Traverse the vertices
    for i in range(N):

        # Hamiltonian Path exists
```

```

        if (dp[i][(1 << N) - 1]):
            return True

    # Otherwise, return false
    return False

# Driver Code
adj = [ [ 0, 1, 1, 1, 0 ] ,
        [ 1, 0, 1, 0, 1 ],
        [ 1, 1, 0, 1, 1 ],
        [ 1, 0, 1, 0, 0 ] ]

N = len(adj)

if (Hamiltonian_path(adj, N)):
    print("YES")
else:
    print("NO")

```

The output in this case would be:

YES

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

Let us assume there are n cities and there are roads in between some of the cities. Somehow all the roads are damaged simultaneously. We have to repair the roads to connect the cities again. There is a fixed cost to repair a particular road. Find out the minimum cost to connect all the cities by repairing roads. Input is in matrix(city) form, if $city[i][j] = 0$ then there is not any road between city i and city j , if $city[i][j] = a > 0$ then the cost to rebuild the path between city i and city j is a . Print out the minimum cost to connect all the cities.

It is assumed that all cities were connected before the roads were damaged. Use Figure below for reference:

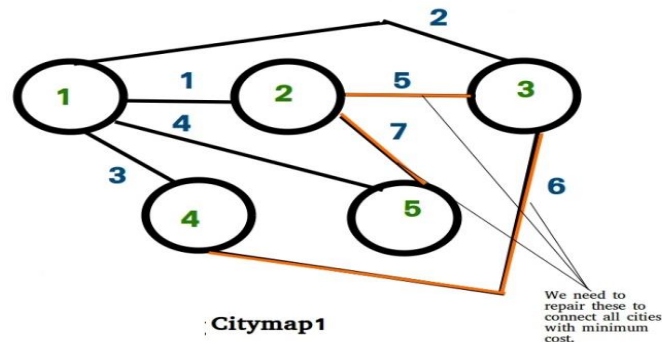


Figure: Graph for connecting cities

Apply MST based approaches to find the solution.

Lab Task 2:

Solve the travelling salesman problem where given a set of cities and distances between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point using MST and also tell how the Travelling Salesman Problem is different from Hamiltonian path.

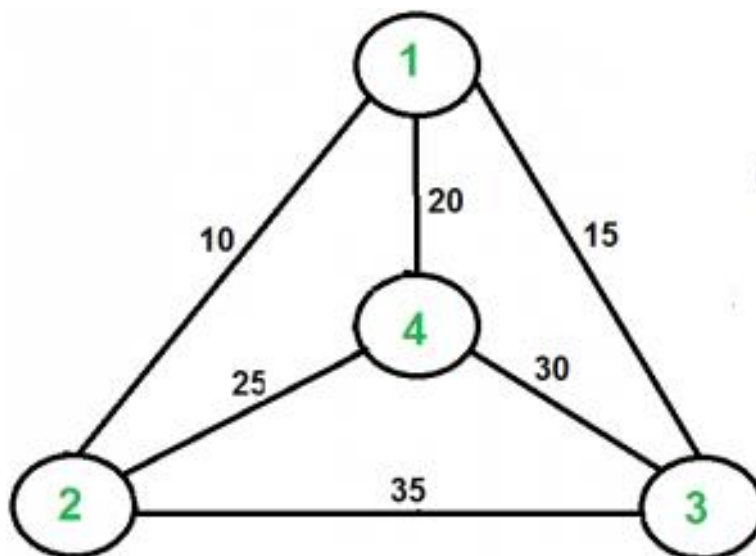


Figure: Graph for Travelling Salesman Problem

Lab 08

Informed Searching Algorithms: A* Search

Objective:

This lab will introduce students to heuristic search methods. In particular we will implement a particular variant of heuristic search known as A* search. Students will also be introduced to different approaches to optimize heuristically inclined search problems.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand different types of heuristically inclined methods
- Calculate Heuristics
- Implement A* Search

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 30 Minutes)

The A* Search algorithm is one of the best and popular technique used in path-finding and graph traversals.

The A* Search algorithms, unlike other traversal techniques uses intelligence. And it is also worth mentioning that many games and web-based maps use this algorithm to find the shortest path very efficiently (approximation).

Consider a square grid having many obstacles and we are given a starting cell and a target cell. We want to reach the target cell (if possible) from the starting cell as quickly as possible.

What A* Search Algorithm does is that at each step it picks the node according to a value- f which is a parameter equal to the sum of two other parameters – g and h . At each step it picks the node/cell having the lowest f , and process that node/cell.

We define g and h as below:

g = the movement cost to move from the starting point to a given square on the grid, following the path generated to get there.

h = the estimated movement cost to move from that given square on the grid to the final destination. This is often referred to as the heuristic, which is nothing but a kind of smart guess. We really don't know the actual distance until we find the path, because all sorts of things can be in the way (walls, water, etc.). There can be many ways to calculate this h which are discussed in the later sections.

We can understand the working of A* search by the Algorithm shown in Figure below:

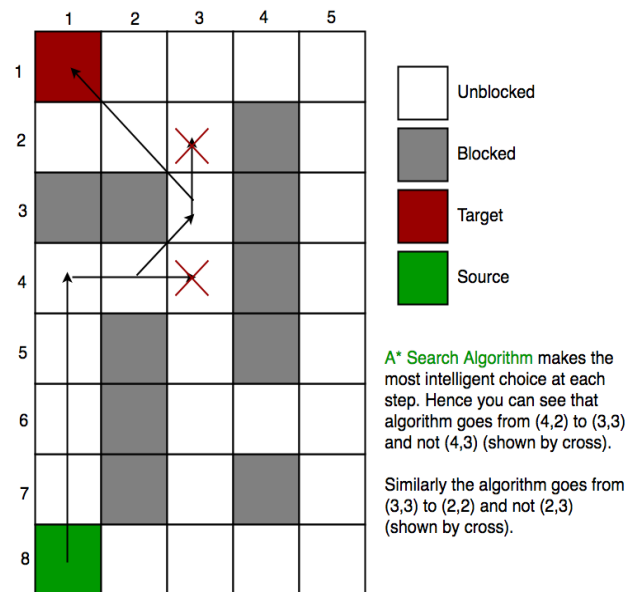


Figure: Graph for Travelling Salesman Problem

To understand the implementation we create two lists:

// A* Search Algorithm

1. Initialize the open list
2. Initialize the closed list put the starting node on the open list (you can leave its f at zero)
3. while the open list is not empty
 - a) find the node with the least f on the open list, call it "q"
 - b) pop q off the open list
 - c) generate q's 8 successors and set their parents to q
 - d) for each successor
 - i) if successor is the goal, stop search
 - ii) else, compute both g and h for successor

$$\text{successor.g} = \text{q.g} + \text{distance between successor and q}$$

$$\text{successor.h} = \text{distance from goal to}$$

successor (This can be done using many ways, we will discuss three heuristics- Manhattan, Diagonal and Euclidean Heuristics)

$\text{successor.f} = \text{successor.g} + \text{successor.h}$

iii) if a node with the same position as successor is in the OPEN list which has a lower f than successor, skip this successor

iv) if a node with the same position as successor is in the CLOSED list which has a lower f than successor, skip this successor otherwise, add the node to the open list
end (for loop)

e) push q on the closed list
end (while loop)

Try dry running the above algorithm on the example given in the figure above.

2) Solved Lab Activities (Allotted Time 1 Hour)

Activity 1

Since A* is a heuristically inclined search approach so we need to calculate Heuristics to start our search. So from the above algorithm we can calculate g but how to calculate h?

We can do the following steps:

- Either calculate the exact value of h (Exact Heuristics).
- Approximate the value of h using some heuristics (Approximation Heuristics).

We will discuss both methods:

A) Exact Heuristics –

We can find exact values of h, but that is generally very time consuming.

Below are some of the methods to calculate the exact value of h.

- 1) Pre-compute the distance between each pair of cells before running the A* Search Algorithm.
- 2) If there are no blocked cells/obstacles then we can just find the exact value of h without any pre-computation using the distance formula/Euclidean Distance.

B) Approximation Heuristics

There are generally three approximation heuristics to calculate h –

1) Manhattan Distance

It is nothing but the sum of absolute values of differences in the goal's x and y coordinates and the current cell's x and y coordinates respectively.

```
# Python code for the above approach
import math as Math

# Code to calculate Manhattan distance
def manhattanDist(M, N, X1, Y1, X2, Y2):
    dist = Math.fabs(X2 - X1) + Math.fabs(Y2 - Y1)
    return (int)(dist)

# Driver code

# Define size of 2-D array
M = 5
N = 5

# First point
X1 = 1
Y1 = 2

# Second point
X2 = 3
Y2 = 3

print(manhattanDist(M, N, X1, Y1, X2, Y2))
```

Output would be 3 in this case.

2) Diagonal Distance

It is nothing but the maximum of absolute values of differences in the goal's x and y coordinates and the current cell's x and y coordinates respectively.

```
# Python3 program for the above approach

from math import sqrt

# Function to find the shortest
# distance between the diagonal of a
# cube and an edge skew to it
```

```
def diagonalLength(a):

    # Stores the required distance
    L = a / sqrt(2)

    # Print the required distance
    print(L)

# Given side of the cube
a = 2

# Function call to find the shortest
# distance between the diagonal of a
# cube and an edge skew to it
diagonalLength(a)
```

Try guessing the output prior to running the program.

3) Euclidean Distance

As it is clear from its name, it is nothing but the distance between the current cell and the goal cell using the distance formula. There are many distance metrics that are used in various Machine Learning Algorithms. One of them is Euclidean Distance. Euclidean distance is the most used distance metric and it is simply a straight line distance between two points.

We might need to import special libraries to calculate for series.

```
import pandas as pd
import numpy as np

# create pandas series
x = pd.Series([1, 2, 3, 4, 5])
y = pd.Series([6, 7, 8, 9, 10])

# here we are computing every thing
# step by step
p1 = np.sum([(a * a) for a in x])
p2 = np.sum([(b * b) for b in y])

# using zip() function to create an
# iterator which aggregates elements
# from two or more iterables
p3 = -1 * np.sum([(2 * a*b) for (a, b) in zip(x, y)])
```

```
dist = np.sqrt(np.sum(p1 + p2 + p3))

print("Series 1:", x)
print("Series 2:", y)
print("Euclidean distance between two series is:", dist)
```

The output in this case would be:

```
Series 1:
0    1
1    2
2    3
3    4
4    5
dtype: int64
Series 2:
0     6
1     7
2     8
3     9
4    10
dtype: int64
Euclidean distance between two series is: 11.180339887498949
```

Activity 2

If in a given Cartesian plane, there are N points. The task is to find the Number of Pairs of points(A, B) such that Point A and Point B do not coincide.

Manhattan Distance and the Euclidean Distance between the points should be equal.

This problem can be used when we have to compare two points in search problem.

```
# Python3 implementation of the
# above approach
from collections import defaultdict

# Function to return the number of
# non coincident pairs of points with
# manhattan distance equal to
# euclidean distance
def findManhattanEuclidPair(arr, n):

    # To store frequency of all distinct Xi
    X = defaultdict(lambda:0)

    # To store Frequency of all distinct Yi
```

```
Y = defaultdict(lambda:0)

# To store Frequency of all distinct
# points (Xi, Yi)
XY = defaultdict(lambda:0)

for i in range(0, n):
    xi = arr[i][0]
    yi = arr[i][1]

    # Hash xi coordinate
    X[xi] += 1

    # Hash yi coordinate
    Y[yi] += 1

    # Hash the point (xi, yi)
    XY[tuple(arr[i])] += 1

xAns, yAns, xyAns = 0, 0, 0

# find pairs with same Xi
for xCoordinatePair in X:
    xFrequency = X[xCoordinatePair]

    # calculate ((xFrequency) C2)
    sameXPairs = (xFrequency *
                  (xFrequency - 1)) // 2
    xAns += sameXPairs

# find pairs with same Yi
for yCoordinatePair in Y:
    yFrequency = Y[yCoordinatePair]

    # calculate ((yFrequency) C2)
    sameYPairs = (yFrequency *
                  (yFrequency - 1)) // 2
    yAns += sameYPairs

# find pairs with same (Xi, Yi)
for XYPair in XY:
    xyFrequency = XY[XYPair]
```

```

# calculate ((xyFrequency) C2)
samePointPairs = (xyFrequency *
                  (xyFrequency - 1)) // 2
xyAns += samePointPairs

return (xAns + yAns - 2 * xyAns)
# we are subtracting 2 * xyAns because we have counted let say A,B coinciding
points two times
# in xAns and yAns which should not be add to the final answer so we are
subtracting xyAns 2 times.

# Driver Code
if __name__ == "__main__":

    arr = [[1, 2], [1,2], [4, 3], [1, 3]]

    n = len(arr)

    print(findManhattanEuclidPair(arr, n))

```

See what the output is after running the code and try to understand the logic.

Activity 3

Now let us try to implement a basic A* search to generate a traversal path from the graph shown in Figure below:

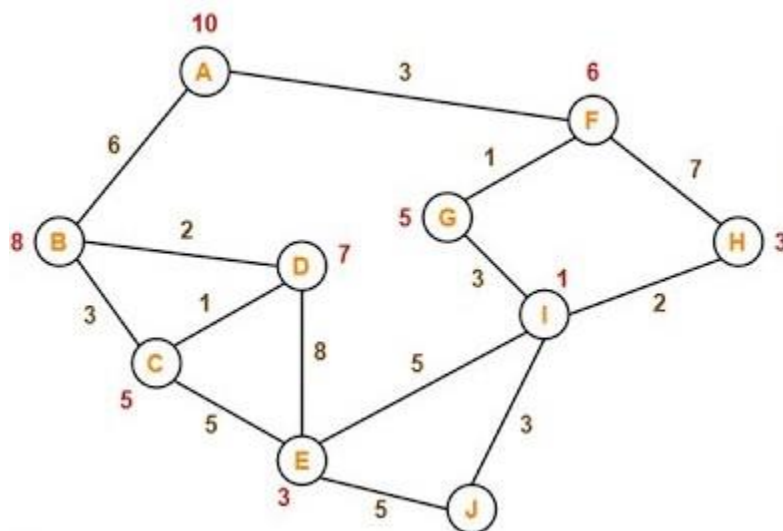


Figure: Graph for A* search problem

Now based on the understanding developed above let us try to create a code to find route from A to J using A* search.

```
def aStarAlgo(start_node, stop_node):
    open_set = set(start_node)
    closed_set = set()
    g = {}          #store distance from starting node
    parents = {}    # parents contains an adjacency map of all nodes
    #distance of starting node from itself is zero
    g[start_node] = 0
    #start_node is root node i.e it has no parent nodes
    #so start_node is set to its own parent node
    parents[start_node] = start_node
    while len(open_set) > 0:
        n = None
        #node with lowest f() is found
        for v in open_set:
            if n == None or g[v] + heuristic(v) < g[n] + heuristic(n):
                n = v
        if n == stop_node or Graph_nodes[n] == None:
            pass
        else:
            for (m, weight) in get_neighbors(n):
                #nodes 'm' not in first and last set are added to first
                #n is set its parent
                if m not in open_set and m not in closed_set:
                    open_set.add(m)
                    parents[m] = n
                    g[m] = g[n] + weight
                #for each node m,compare its distance from start i.e g(m) to the
                #from start through n node
                else:
                    if g[m] > g[n] + weight:
                        #update g(m)
                        g[m] = g[n] + weight
                        #change parent of m to n
                        parents[m] = n
                        #if m in closed set,remove and add to open
                        if m in closed_set:
                            closed_set.remove(m)
                        open_set.add(m)
```

```

    if n == None:
        print('Path does not exist!')
        return None

    # if the current node is the stop_node
    # then we begin reconstructin the path from it to the start_node
    if n == stop_node:
        path = []
        while parents[n] != n:
            path.append(n)
            n = parents[n]
        path.append(start_node)
        path.reverse()
        print('Path found: {}'.format(path))
        return path
    # remove n from the open_list, and add it to closed_list
    # because all of his neighbors were inspected
    open_set.remove(n)
    closed_set.add(n)
    print('Path does not exist!')
    return None

#define fuction to return neighbor and its distance
#from the passed node
def get_neighbors(v):
    if v in Graph_nodes:
        return Graph_nodes[v]
    else:
        return None

#for simplicity we ll consider heuristic distances given
#and this function returns heuristic distance for all nodes
def heuristic(n):
    H_dist = {
        'A': 11,
        'B': 6,
        'C': 5,
        'D': 7,
        'E': 3,
        'F': 6,
        'G': 5,

```

```

        'H': 3,
        'I': 1,
        'J': 0
    }
    return H_dist[n]

#Describe your graph here
Graph_nodes = {
    'A': [('B', 6), ('F', 3)],
    'B': [('A', 6), ('C', 3), ('D', 2)],
    'C': [('B', 3), ('D', 1), ('E', 5)],
    'D': [('B', 2), ('C', 1), ('E', 8)],
    'E': [('C', 5), ('D', 8), ('I', 5), ('J', 5)],
    'F': [('A', 3), ('G', 1), ('H', 7)],
    'G': [('F', 1), ('I', 3)],
    'H': [('F', 7), ('I', 2)],
    'I': [('E', 5), ('G', 3), ('H', 2), ('J', 3)],
}

aStarAlgo('A', 'J')

```

Output would be:

Path found: ['A', 'F', 'G', 'I', 'J']

3) Graded Lab Tasks (Allotted Time 1 Hour)

Lab Task 1:

In the graph given above, change the route of your nodes from A to H and calculate path using the A* search.

Lab Task 2:

Implement the A* search algorithm to find the path from Arad to Hirosawa, show what improvements are evident when using Heuristics based search as compared to uninformed methods.

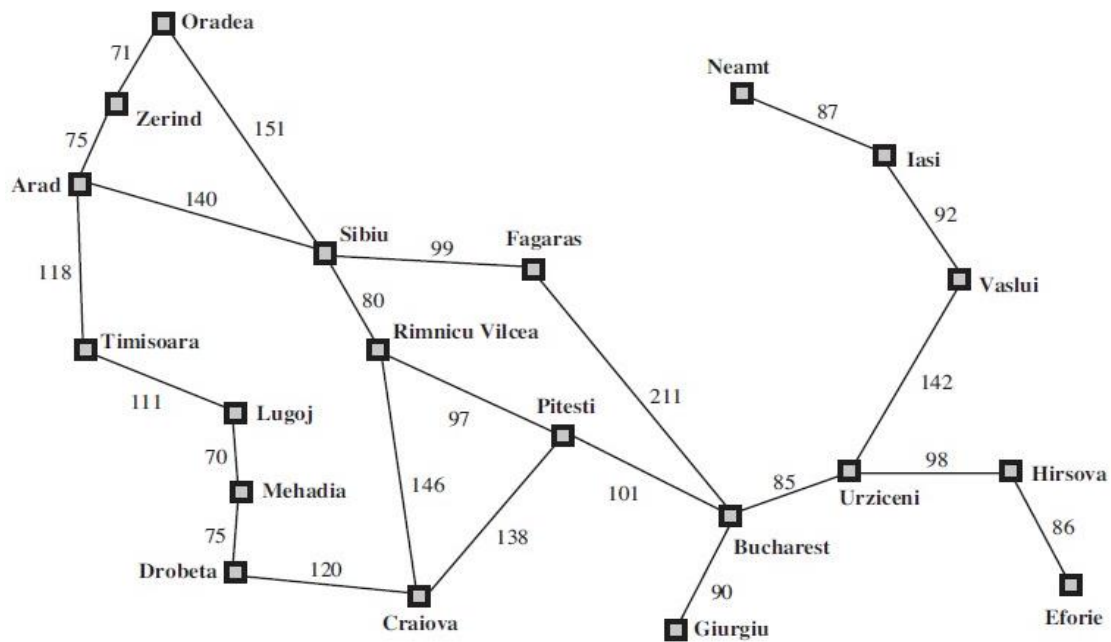


Figure: Graph for A* search problem

Lab 09

Informed Searching Algorithms

Hill Climbing and Best First Search

Objective:

This lab will introduce students to heuristic search methods. In particular we will implement a particular variant of heuristic search known as Hill Climbing search. Students will also be introduced to different approaches to optimize heuristically inclined search problems like best first search.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Implement Hill Climbing Search
- Implement Best First Search

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 15 Minutes)

Hill Climbing

Hill Climbing is a heuristic search used for mathematical optimization problems in the field of Artificial Intelligence.

Given a large set of inputs and a good heuristic function, it tries to find a sufficiently good solution to the problem. This solution may not be the global optimal maximum.

'Heuristic search' means that this search algorithm may not find the optimal solution to the problem. However, it will give a good solution in a reasonable time.

A heuristic function is a function that will rank all the possible alternatives at any branching step in the search algorithm based on the available information. It helps the algorithm to select the best route out of possible routes.

The Features of Hill Climbing are as follow:

- Variant of generate and test algorithm: It is a variant of generating and test algorithm. The generate and test algorithm is as follows :

-
- Generate possible solutions.
 - Test to see if this is the expected solution.
 - If the solution has been found quit else go to step 1.

Hence we call Hill climbing a variant of generating and test algorithm as it takes the feedback from the test procedure. Then this feedback is utilized by the generator in deciding the next move in search space.

It Uses the Greedy approach: At any point in state space, the search moves in that direction only which optimizes the cost of function with the hope of finding the optimal solution at the end.

There are different types of Hill Climbing approaches that can be used

Simple Hill climbing: It examines the neighboring nodes one by one and selects the first neighboring node which optimizes the current cost as the next node.

Steepest-Ascent Hill climbing: It first examines all the neighboring nodes and then selects the node closest to the solution state as of the next node.

Stochastic hill climbing: It does not examine all the neighboring nodes before deciding which node to select. It just selects a neighboring node at random and decides (based on the amount of improvement in that neighbor) whether to move to that neighbor or to examine another.

The state-space diagram is a graphical representation of the set of states our search algorithm can reach vs the value of our objective function(the function which we wish to maximize).

X-axis: denotes the state space i.e. states or configuration our algorithm may reach.

Y-axis: denotes the values of objective function corresponding to a particular state.

The best solution will be that state space where the objective function has a maximum value (global maximum).

Best First Search

In BFS and DFS, when we are at a node, we can consider any of the adjacent as next node. So both BFS and DFS blindly explore paths without considering any cost function. The idea of Best First Search is to use an evaluation function to decide which adjacent is most promising and then explore. Best First Search falls under the category of Heuristic Search or Informed Search. We use a priority queue or heap to store costs of nodes which have lowest evaluation function value. So the implementation is a variation of BFS, we just need to change Queue to PriorityQueue. We will see more of its working in the solve example.

2) Solved Lab Activities (Allotted Time 1 Hour)

Now we will try to learn by solving examples from Hill Climbing and Best First search.

Activity 1

Implementation of hill climbing algorithm in python is given below. In this implementation the algorithm is searching for the most efficient tour to visit a number of cities.

Where Cities are given as A, B, C:

	A	B	C
A	0	3	2
B	3	0	1
C	2	1	0

Figure: Distance between cities for Hill Climbing search

Possible Tours:

A --[3 km]--> B --[1 km]--> C | [4 km]

A --[2 km]--> C --[1 km]--> B | [3 km] (most efficient)

If you change the amount of cities (countCities = x), you have to change the threshold as well. For 20 cities, a threshold between 15-25 is recommended.

For 100 cities, a threshold between 100-175 is recommended. The higher the threshold, the more time the algorithm will need to find an optimum. Check with different variations to find the most optimal path.

The implementation for the above example can be given as:

```
from random import *
import random
import numpy
import copy

countCities = 20;
# 2D Array
cities = numpy.zeros(shape=(20,20))
# tour
hypothesis = [int]*countCities
visitedCities = []
saveState = []

threshold = 25
lastFitness = 0
trials = 0
cityIndex = 1

# calculates fitness based on the difference between the distances
def getFitness(fitness, hypothesis, saveState, cities):
    oldDistance = getDistance(cities, saveState)
    newDistance = getDistance(cities, hypothesis)
    print("Old Distance ",oldDistance,"km")
    print("New Distance ",newDistance,"km")

    if(oldDistance > newDistance):
        fitness += 1
    elif(oldDistance < newDistance):
        fitness -= 1

    return fitness

# choose random City at position cityIndex
def doRandomStep():
    global visitedCities
    global saveState
    global hypothesis
    if(len(visitedCities) >= countCities):
        visitedCities.clear()
        visitedCities.append(0)
    randomNumbers = list(set(saveState) - set(visitedCities))
    randomStep = random.choice(randomNumbers)
    visitedCities.append(randomStep)
```

```

        hypothesis.remove(randomStep)
        hypothesis.insert(cityIndex,randomStep)

# next city
def increment():
    global cityIndex
    global visitedCities
    if (cityIndex < countCities - 2):
        cityIndex += 1
    else:
        visitedCities.clear()
        cityIndex = 1

# calculates distance from tour
def getDistance(cities, hypothesis):
    distance = 0
    for i in range(countCities):
        if (i < countCities-1):
            distance += cities[hypothesis[i]][hypothesis[i+1]]
            print("[",hypothesis[i],"]",distance,"km ",end="")
        else:
            print("[",hypothesis[i],"]")

    return distance

if __name__ == '__main__':

    for i in range(countCities):
        hypothesis[i] = i
        for j in range(countCities):
            if (j > i):
                cities[i][j] = randint(1,100)
            elif(j < i):
                cities[i][j] = cities[j][i]

    print("=== START ===");
    while(lastFitness < threshold):

        print("_____")
        saveState = copy.deepcopy(hypothesis)
        doRandomStep()
        currentFitness = getFitness(lastFitness, hypothesis, saveState, cities)
        print("Old fitness ",lastFitness)
        print("Current fitness ",currentFitness)

        if (currentFitness > lastFitness):

```

```

        lastFitness = currentFitness
    elif(currentFitness < lastFitness):
        hypothesis = copy.deepcopy(saveState)
        if(trials < 3):
            increment()
        else:
            trials = 0
        visitedCities.append(saveState[cityIndex])

```

The output in this case would be:

```

[ 0 ] 30.0 km [ 1 ] 103.0 km [ 3 ] 140.0 km [ 2 ] 158.0 km [ 4 ] 232.0 km [ 5 ] 238.0 km [ 6 ] 269.0 km [ 7 ] 293.0 km [ 8 ] 322.0 km [ 9 ] 356.0 km [ 10 ] 451.0 km [
11 ] 526.0 km [ 12 ] 582.0 km [ 13 ] 609.0 km [ 14 ] 614.0 km [ 15 ] 704.0 km [ 16 ] 709.0 km [ 17 ] 798.0 km [ 18 ] 808.0 km [ 19 ]
[ 0 ] 30.0 km [ 1 ] 93.0 km [ 5 ] 172.0 km [ 3 ] 209.0 km [ 2 ] 227.0 km [ 4 ] 324.0 km [ 6 ] 355.0 km [ 7 ] 379.0 km [ 8 ] 408.0 km [ 9 ] 442.0 km [ 10 ] 537.0 km [ 1
1 ] 612.0 km [ 12 ] 668.0 km [ 13 ] 695.0 km [ 14 ] 700.0 km [ 15 ] 790.0 km [ 16 ] 795.0 km [ 17 ] 884.0 km [ 18 ] 894.0 km [ 19 ]
Old Distance 808.0 km
New Distance 894.0 km
Old fitness 1
Current fitness 0

[ 0 ] 30.0 km [ 1 ] 103.0 km [ 3 ] 140.0 km [ 2 ] 158.0 km [ 4 ] 232.0 km [ 5 ] 238.0 km [ 6 ] 269.0 km [ 7 ] 293.0 km [ 8 ] 322.0 km [ 9 ] 356.0 km [ 10 ] 451.0 km [
11 ] 526.0 km [ 12 ] 582.0 km [ 13 ] 609.0 km [ 14 ] 614.0 km [ 15 ] 704.0 km [ 16 ] 709.0 km [ 17 ] 798.0 km [ 18 ] 808.0 km [ 19 ]
[ 0 ] 30.0 km [ 1 ] 103.0 km [ 3 ] 158.0 km [ 16 ] 257.0 km [ 2 ] 275.0 km [ 4 ] 349.0 km [ 5 ] 355.0 km [ 6 ] 386.0 km [ 7 ] 410.0 km [ 8 ] 439.0 km [ 9 ] 473.0 km [
10 ] 568.0 km [ 11 ] 643.0 km [ 12 ] 699.0 km [ 13 ] 726.0 km [ 14 ] 731.0 km [ 15 ] 786.0 km [ 17 ] 875.0 km [ 18 ]
===== RESTART: Shell =====

```

We can try experimenting with different thresholds to see where the optimum solution is achieved.

Activity 2

This example implements a best-first search algorithm in Python for a grid and a graph. The Best-first search is an informed search algorithm as it uses an heuristic to guide the search, it uses an estimation of the cost to the goal as the heuristic.

In this case Best-first search starts in an initial start node and updates neighbor nodes with an estimation of the cost to the goal node, it selects the neighbor with the lowest cost and continues to expand nodes until it reaches the goal node. Best-first search favors nodes that are close to the goal node, this can be implemented by using a priority queue or by sorting the list of open nodes in ascending order. The heuristic should not overestimate the cost to the goal, a heuristic that is closer to the actual cost is better as long as it doesn't overestimate the cost to the goal.

The Best-first search is complete, and it will find the shortest path to the goal. A good heuristic can make the search fast, but it may take a long time and consume a lot of memory in a large search space.

We will first apply Best First Search to a maze problem, we have created a maze shown in Figure below:

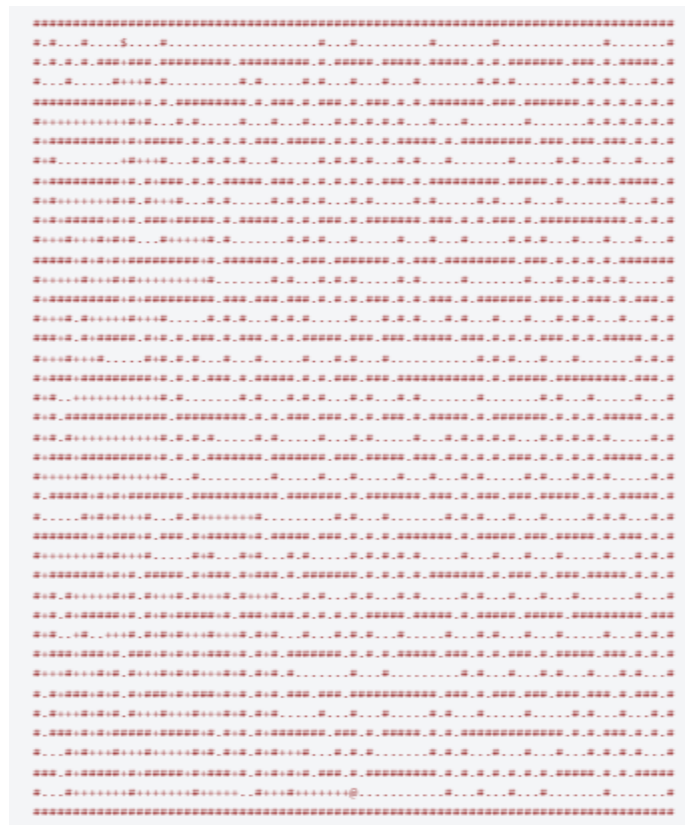


Figure: Maze Grid for Best First Search

The maze data file can be downloaded from:

(https://drive.google.com/file/d/1-WIIL_vmODWPX0PUYVwyjQYxphLxHZaN/view?usp=sharing)

Do make sure to change the path of the file correctly in your code.

The maze has walls, with a start (@) and a goal (\$). The Best-first search is used to find the shortest path from the start node to the goal node by using the distance to the goal node as a heuristic.

This class represents a node

```
1. class Node:
2.
3.     # Initialize the class
4.     def __init__(self, position:(), parent:()):
5.         self.position = position
6.         self.parent = parent
```

```

7.         self.g = 0 # Distance to start node
8.         self.h = 0 # Distance to goal node
9.         self.f = 0 # Total cost
10.
11.         # Compare nodes
12.         def __eq__(self, other):
13.             return self.position == other.position
14.
15.         # Sort nodes
16.         def __lt__(self, other):
17.             return self.f < other.f
18.
19.         # Print node
20.         def __repr__(self):
21.             return '({0},{1})'.format(self.position, self.f)
22.
23.         # Draw a grid
24.         def draw_grid(map, width, height, spacing=2, **kwargs):
25.             for y in range(height):
26.                 for x in range(width):
27.                     print('%%- %ds' % spacing % draw_tile(map, (x, y),
kwargs), end='')
28.                     print()
29.
30.         # Draw a tile
31.         def draw_tile(map, position, kwargs):
32.
33.             # Get the map value
34.             value = map.get(position)
35.
36.             # Check if we should print the path
37.             if 'path' in kwargs and position in kwargs['path']: value =
'+'
38.
39.             # Check if we should print start point
40.             if 'start' in kwargs and position == kwargs['start']: value =
'@'
41.
42.             # Check if we should print the goal point
43.             if 'goal' in kwargs and position == kwargs['goal']: value =
'$'
44.
45.             # Return a tile value
46.             return value
47.
48.         # Best-first search
49.         def best_first_search(map, start, end):
50.
51.             # Create lists for open nodes and closed nodes
52.             open = []

```

```
53.         closed = []
54.
55.         # Create a start node and an goal node
56.         start_node = Node(start, None)
57.         goal_node = Node(end, None)
58.
59.         # Add the start node
60.         open.append(start_node)
61.
62.         # Loop until the open list is empty
63.         while len(open) > 0:
64.
65.             # Sort the open list to get the node with the lowest cost first
66.             open.sort()
67.
68.             # Get the node with the lowest cost
69.             current_node = open.pop(0)
70.
71.             # Add the current node to the closed list
72.             closed.append(current_node)
73.
74.             # Check if we have reached the goal, return the path
75.             if current_node == goal_node:
76.                 path = []
77.                 while current_node != start_node:
78.                     path.append(current_node.position)
79.                     current_node = current_node.parent
80.                 path.append(start)
81.                 # Return reversed path
82.                 return path[::-1]
83.
84.             # Unzip the current node position
85.             (x, y) = current_node.position
86.
87.             # Get neighbors
88.             neighbors = [(x-1, y), (x+1, y), (x, y-1), (x, y+1)]
89.
90.             # Loop neighbors
91.             for next in neighbors:
92.
93.                 # Get value from map
94.                 map_value = map.get(next)
95.
96.                 # Check if the node is a wall
97.                 if (map_value == '#'):
98.                     continue
99.
100.                # Create a neighbor node
101.                neighbor = Node(next, current_node)
102.
```

```

103.             # Check if the neighbor is in the closed list
104.             if(neighbor in closed):
105.                 continue
106.
107.             # Generate heuristics (Manhattan distance)
108.             neighbor.g = abs(neighbor.position[0] -
start_node.position[0]) + abs(neighbor.position[1] -
start_node.position[1])
109.             neighbor.h = abs(neighbor.position[0] -
goal_node.position[0]) + abs(neighbor.position[1] -
goal_node.position[1])
110.             neighbor.f = neighbor.h
111.
112.             # Check if neighbor is in open list and if it has a
lower f value
113.             if(add_to_open(open, neighbor) == True):
114.                 # Everything is green, add neighbor to open list
115.                 open.append(neighbor)
116.
117.             # Return None, no path is found
118.             return None
119.
120.             # Check if a neighbor should be added to open list
121.             def add_to_open(open, neighbor):
122.                 for node in open:
123.                     if (neighbor == node and neighbor.f >= node.f):
124.                         return False
125.                 return True
126.
127.             # The main entry point for this module
128.             def main():
129.
130.                 # Get a map (grid)
131.                 map = {}
132.                 chars = ['c']
133.                 start = None
134.                 end = None
135.                 width = 0
136.                 height = 0
137.
138.                 # Open a file
139.                 fp = open('data\\maze.in', 'r')
140.
141.                 # Loop until there is no more lines
142.                 while len(chars) > 0:
143.
144.                     # Get chars in a line
145.                     chars = [str(i) for i in fp.readline().strip()]
146.
147.                     # Calculate the width

```

```
148.         width = len(chars) if width == 0 else width
149.
150.         # Add chars to map
151.         for x in range(len(chars)):
152.             map[(x, height)] = chars[x]
153.             if(chars[x] == '@'):
154.                 start = (x, height)
155.             elif(chars[x] == '$'):
156.                 end = (x, height)
157.
158.         # Increase the height of the map
159.         if(len(chars) > 0):
160.             height += 1
161.
162.         # Close the file pointer
163.         fp.close()
164.
165.         # Find the closest path from start(@) to end($)
166.         path = best_first_search(map, start, end)
167.         print()
168.         print(path)
169.         print()
170.         draw_grid(map, width, height, spacing=1, path=path,
171.                   start=start, goal=end)
171.         print()
172.         print('Steps to goal: {0}'.format(len(path)))
173.         print()
174.
175.         # Tell python to run main method
176.         if __name__ == "__main__": main()
```

Code file can be accessed at

(https://drive.google.com/file/d/1-WIII_vmODWPX0PUYVWyjQYxphLxHZaN/view?usp=sharing)

The distance to the goal node is calculated as the manhattan distance from a node to the goal node. Also try running the implementation and experiment with changing the start and goal states in the maze.

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

Lab Task 1:

Apply Hill Climbing and Best First search to find the path from Arad to Vaslui, try experimenting with different combinations to get optimal outputs.

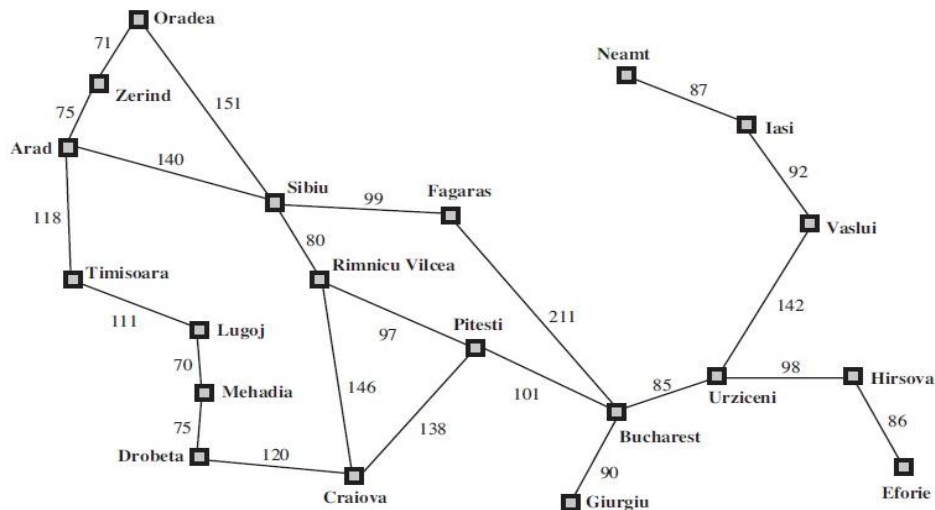


Figure: Graph for Hill Climbing and Best First search problem Arad to Vaslui

In the above definition, mathematical optimization problems imply that hill-climbing solves the problems where we need to maximize or minimize a given real function by choosing values from the given inputs.

Lab Task 2:

Apply Best First search similar to the Grid based problem discussed to the maze given in figure below:

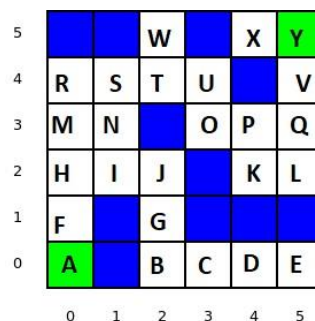


Figure: Maze Problem for Best First Search

Consider a maze as shown below. Each empty tile represents a separate node in the graph, while the walls are represented by blue tiles. Your starting node is A and the goal is to reach Y.

Lab 10

Introduction to Genetic Algorithms

Objective:

This lab will introduce students to genetic algorithms. Students will get the opportunity to get into details of genetic concepts of computation including crossover, mutation, and survivor selection. This lab will also introduce students into different schemes of chromosome encoding. The Genetic Algorithm is a stochastic global search optimization algorithm. It is inspired by the biological theory of evolution by means of natural selection. Specifically, the new synthesis that combines an understanding of genetics with the theory.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand the concept of Genetic Algorithms
- Implement One Max problem solution using genetic algorithms
- Apply genetic algorithms on Travelling Salesman Problem

Instructor Note:

As pre-lab activity, read Chapters 4 from the book (Artificial Intelligence, A Modern Approach by Peter Norvig, 4th edition) to know the basics of genetic algorithms.

1) Useful Concepts (Allocated Time 15 Minutes)

Genetic algorithms work via transformations on populations of chromosomes over some number of generations. Imagine you're playing a card game where your goal is to get the highest possible card after some number of turns. You are initially given five cards and you can choose to keep any number of cards at the end of every turn. In this example, a single card is a chromosome. It represents one solution to your problem. Your entire hand is the population; it's a collection of possible solutions. The changes you make to your hand after every turn are transformations. Finally, every turn represents one generation and one transformation of the population.

The basic structure of a genetic algorithm can be seen from the Figure below:

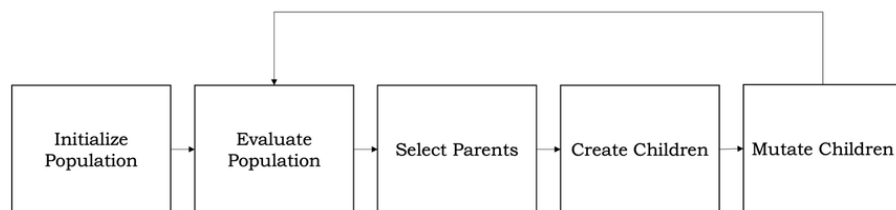


Figure: Genetic Algorithm Example

Each step depicted in the image performs a transformation on the population that brings you closer to finding a solution. The process is repeated until a solution is found. Most genetic algorithms follow a structure like the one in the figure, which is easily translated into equally structured code. Your genetic algorithm will also follow these same steps using a code that mirrors each step in the process.

The algorithm uses analogs of a genetic representation (bitstrings), fitness (function evaluations), genetic recombination (crossover of bitstrings), and mutation (flipping bits).

The algorithm works by first creating a population of a fixed size of random bitstrings. The main loop of the algorithm is repeated for a fixed number of iterations or until no further improvement is seen in the best solution over a given number of iterations.

One iteration of the algorithm is like an evolutionary generation.

First, the population of bitstrings (candidate solutions) are evaluated using the objective function. The objective function evaluation for each candidate solution is taken as the fitness of the solution, which may be minimized or maximized.

Then, parents are selected based on their fitness. A given candidate solution may be used as parent zero or more times. A simple and effective approach to selection involves drawing k candidates from the population randomly and selecting the member from the group with the best fitness. This is called tournament selection where k is a hyperparameter and set to a value such as 3. This simple approach simulates a more costly fitness-proportionate selection scheme.

Parents are used as the basis for generating the next generation of candidate points and one parent for each position in the population is required.

Parents are then taken in pairs and used to create two children. Recombination is performed using a crossover operator. This involves selecting a random split point on the bit string, then creating a child with the bits up to the split point from the first parent and from the split point to the end of the string from the second parent. This process is then inverted for the second child.

For example the two parents:

parent1 = 00000

parent2 = 11111

May result in two cross-over children:

child1 = 00011

child2 = 11100

This is called one point crossover, and there are many other variations of the operator.

Crossover is applied probabilistically for each pair of parents, meaning that in some cases, copies of the parents are taken as the children instead of the recombination operator. Crossover is controlled by a hyperparameter set to a large value, such as 80 percent or 90 percent. The Crossover is the Genetic Algorithm's distinguishing feature. It involves mixing and matching parts of two parents to form children. How you do that mixing and matching depends on the representation of the individuals.

Mutation involves flipping bits in created children candidate solutions. Typically, the mutation rate is set to $1/L$, where L is the length of the bitstring. Each bit in a binary-valued chromosome typically has a small probability of being flipped. For a chromosome with m bits, this mutation rate is typically set to $1/m$, yielding an average of one mutation per child chromosome.

For example, if a problem used a bitstring with 20 bits, then a good default mutation rate would be $(1/20) = 0.05$ or a probability of 5 percent.

This defines the simple genetic algorithm procedure. It is a large field of study, and there are many extensions to the algorithm.

2) Solved Lab Activities (Allotted Time 1 Hour)

In previous lab we implemented hill climbing and saw that it can stick at a local maxima. A possible solution to avoid local maxima is to use genetic algorithms. A flowchart of genetic algorithm is given below:

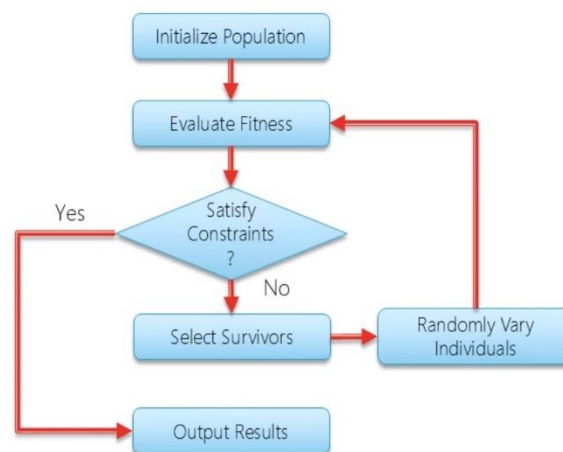


Figure: Genetic Algorithm Flowchart

Activity 1

The One-Max problem is a trivial problem often used to introduce the concept of genetic algorithms. It's incredibly simple, but it's great for introducing many of the critical aspects of a genetic algorithm. The problem boils down to one question: what is the maximum sum of a bitstring (a string consisting of only 1s and 0s) of length N ?

We know that the maximum sum of a bitstring of length N is N . However, if you wanted to prove this using a brute-force search, you'd end up needing to search through 2^N different solutions. As with any search problem, this isn't too difficult with relatively small bitstrings. But what happens if you want to use this technique for bitstrings of length 40? You'd have to search over one trillion possible bitstrings. To avoid this, we can create a genetic algorithm that produces an optimal solution without iterating over every possible solution in the search space.

In this section, we will apply the genetic algorithm to a binary string-based optimization problem.

The problem is called OneMax and evaluates a binary string based on the number of 1s in the string. For example, a bitstring with a length of 20 bits will have a score of 20 for a string of all 1s.

Given we have implemented the genetic algorithm to minimize the objective function, we can add a negative sign to this evaluation so that large positive values become large negative values.

The `onemax()` function below implements this and takes a bitstring of integer values as input and returns the negative sum of the values.

```
# objective function
def onemax(x):
    return -sum(x)
```

Next, we can configure the search. The search will run for 100 iterations, and we will use 20 bits in our candidate solutions, meaning the optimal fitness will be -20.0. The population size will be 100, and we will use a crossover rate of 90 percent and a mutation rate of 5 percent. This configuration was chosen after a little trial and error.

The full implementation of one max using genetic algorithms is given below:

```
from numpy.random import randint
from numpy.random import rand

# objective function
def onemax(x):
    return -sum(x)

# tournament selection
def selection(pop, scores, k=3):
    # first random selection
    selection_ix = randint(len(pop))
    for ix in randint(0, len(pop), k-1):
        # check if better (e.g. perform a tournament)
        if scores[ix] < scores[selection_ix]:
            selection_ix = ix
    return pop[selection_ix]
```

```

# crossover two parents to create two children
def crossover(p1, p2, r_cross):
    # children are copies of parents by default
    c1, c2 = p1.copy(), p2.copy()
    # check for recombination
    if rand() < r_cross:
        # select crossover point that is not on the end of the string
        pt = randint(1, len(p1)-2)
        # perform crossover
        c1 = p1[:pt] + p2[pt:]
        c2 = p2[:pt] + p1[pt:]
    return [c1, c2]

# mutation operator
def mutation(bitstring, r_mut):
    for i in range(len(bitstring)):
        # check for a mutation
        if rand() < r_mut:
            # flip the bit
            bitstring[i] = 1 - bitstring[i]

# genetic algorithm
def genetic_algorithm(objective, n_bits, n_iter, n_pop, r_cross, r_mut):
    # initial population of random bitstring
    pop = [randint(0, 2, n_bits).tolist() for _ in range(n_pop)]
    # keep track of best solution
    best, best_eval = 0, objective(pop[0])
    # enumerate generations
    for gen in range(n_iter):
        # evaluate all candidates in the population
        scores = [objective(c) for c in pop]
        # check for new best solution
        for i in range(n_pop):
            if scores[i] < best_eval:
                best, best_eval = pop[i], scores[i]
                print(">%d, new best f(%s) = %.3f" % (gen, pop[i],
scores[i]))
        # select parents
        selected = [selection(pop, scores) for _ in range(n_pop)]
        # create the next generation
        children = list()
        for i in range(0, n_pop, 2):
            # get selected parents in pairs
            p1, p2 = selected[i], selected[i+1]
            # crossover and mutation

```

```

        for c in crossover(p1, p2, r_cross):
            # mutation
            mutation(c, r_mut)
            # store for next generation
            children.append(c)

    # replace population
    pop = children
    return [best, best_eval]

# define the total iterations
n_iter = 100
# bits
n_bits = 20
# define the population size
n_pop = 100
# crossover rate
r_cross = 0.9
# mutation rate
r_mut = 1.0 / float(n_bits)
# perform the genetic algorithm search
best, score = genetic_algorithm(onemax, n_bits, n_iter, n_pop, r_cross, r_mut)
print('Done!')
print('f(%s) = %f' % (best, score))

```

The output in this case would be:

```

>>>
===== RESTART: D:/AI lectures/Spring 2022-IntrotoAI/LABS CODE/gametree/geneticalgo.py ==
>0, new best f([0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1]) = -11.000
>0, new best f([0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1]) = -13.000
>0, new best f([1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1]) = -15.000
>1, new best f([1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0]) = -16.000
>2, new best f([1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1]) = -17.000
>3, new best f([1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1]) = -18.000
>3, new best f([1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]) = -19.000
>7, new best f([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]) = -20.000
Done!
f([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]) = -20.000000
>>>

```

Running the example will report the best result as it is found along the way, then the final best solution at the end of the search, which we would expect to be the optimal solution.

In this case, we can see that the search found the optimal solution after about seven to eight generations.

Activity 2

Normally, optimizing the OneMax function is not very interesting; we are more likely to want to optimize a continuous function.

For example, we can define the x^2 minimization function that takes input variables and has an optima at $f(0, 0) = 0.0$.

We can minimize this function with a genetic algorithm, first decode the bitstrings to numbers prior to evaluating each with the objective function.

We can achieve this by first decoding each substring to an integer, then scaling the integer to the desired range. This will give a vector of values in the range that can then be provided to the objective function for evaluation.

The *decode()* function below implements this, taking the bounds of the function, the number of bits per variable, and a bitstring as input and returns a list of decoded real values.

The complete example of the genetic algorithm for continuous function optimization is listed below.

```
# genetic algorithm search for continuous function optimization
from numpy.random import randint
from numpy.random import rand

# objective function
def objective(x):
    return x[0]**2.0 + x[1]**2.0

# decode bitstring to numbers
def decode(bounds, n_bits, bitstring):
    decoded = list()
    largest = 2**n_bits
    for i in range(len(bounds)):
        # extract the substring
        start, end = i * n_bits, (i * n_bits)+n_bits
        substring = bitstring[start:end]
        # convert bitstring to a string of chars
        chars = ''.join([str(s) for s in substring])
        # convert string to integer
        integer = int(chars, 2)
        # scale integer to desired range
        value = bounds[i][0] + (integer/largest) * (bounds[i][1] - bounds[i][0])
        # store
        decoded.append(value)
    return decoded

# tournament selection
```

```

def selection(pop, scores, k=3):
    # first random selection
    selection_ix = randint(len(pop))
    for ix in randint(0, len(pop), k-1):
        # check if better (e.g. perform a tournament)
        if scores[ix] < scores[selection_ix]:
            selection_ix = ix
    return pop[selection_ix]

# crossover two parents to create two children
def crossover(p1, p2, r_cross):
    # children are copies of parents by default
    c1, c2 = p1.copy(), p2.copy()
    # check for recombination
    if rand() < r_cross:
        # select crossover point that is not on the end of the string
        pt = randint(1, len(p1)-2)
        # perform crossover
        c1 = p1[:pt] + p2[pt:]
        c2 = p2[:pt] + p1[pt:]
    return [c1, c2]

# mutation operator
def mutation(bitstring, r_mut):
    for i in range(len(bitstring)):
        # check for a mutation
        if rand() < r_mut:
            # flip the bit
            bitstring[i] = 1 - bitstring[i]

# genetic algorithm
def genetic_algorithm(objective, bounds, n_bits, n_iter, n_pop, r_cross, r_mut):
    # initial population of random bitstring
    pop = [randint(0, 2, n_bits*len(bounds)).tolist() for _ in range(n_pop)]
    # keep track of best solution
    best, best_eval = 0, objective(decode(bounds, n_bits, pop[0]))
    # enumerate generations
    for gen in range(n_iter):
        # decode population
        decoded = [decode(bounds, n_bits, p) for p in pop]
        # evaluate all candidates in the population
        scores = [objective(d) for d in decoded]
        # check for new best solution
        for i in range(n_pop):
            if scores[i] < best_eval:
                best, best_eval = pop[i], scores[i]

```

```

        print(">%d, new best f(%s) = %f" % (gen, decoded[i],
scores[i]))
    # select parents
    selected = [selection(pop, scores) for _ in range(n_pop)]
    # create the next generation
    children = list()
    for i in range(0, n_pop, 2):
        # get selected parents in pairs
        p1, p2 = selected[i], selected[i+1]
        # crossover and mutation
        for c in crossover(p1, p2, r_cross):
            # mutation
            mutation(c, r_mut)
            # store for next generation
            children.append(c)
    # replace population
    pop = children
    return [best, best_eval]

# define range for input
bounds = [[-5.0, 5.0], [-5.0, 5.0]]
# define the total iterations
n_iter = 100
# bits per variable
n_bits = 16
# define the population size
n_pop = 100
# crossover rate
r_cross = 0.9
# mutation rate
r_mut = 1.0 / (float(n_bits) * len(bounds))
# perform the genetic algorithm search
best, score = genetic_algorithm(objective, bounds, n_bits, n_iter, n_pop, r_cross,
r_mut)
print('Done!')
decoded = decode(bounds, n_bits, best)
print('f(%s) = %f' % (decoded, score))

```

The output of the code is given as:

```
>>>
===== RESTART: C:/Users/hanzalah/Downloads/geneticalgo2.py =====
>0, new best f([-1.71478271484375, 1.2158203125]) = 4.418699
>0, new best f([-1.225433349609375, -0.8526611328125]) = 2.228718
>0, new best f([0.52032470703125, -0.971527099609375]) = 1.214603
>0, new best f([-0.407562255859375, -0.5322265625]) = 0.449372
>0, new best f([-0.06134033203125, 0.49407958984375]) = 0.247877
>1, new best f([0.1806640625, -0.2313232421875]) = 0.086150
>1, new best f([-0.04730224609375, -0.15106201171875]) = 0.025057
>4, new best f([-0.064849853515625, 0.09490966796875]) = 0.013213
>5, new best f([-0.059967041015625, 0.007781982421875]) = 0.003657
>6, new best f([-0.005950927734375, 0.024566650390625]) = 0.000639
>7, new best f([-0.005950927734375, 0.0006103515625]) = 0.000036
>12, new best f([-0.005340576171875, 0.0]) = 0.000029
>15, new best f([-0.005035400390625, 0.0]) = 0.000025
>15, new best f([-0.000762939453125, 0.0]) = 0.000001
>15, new best f([-0.000457763671875, 0.0]) = 0.000000
>20, new best f([-0.00030517578125, 0.0]) = 0.000000
>29, new best f([-0.000152587890625, 0.0]) = 0.000000
Done!
f([-0.000152587890625, 0.0]) = 0.000000
```

Running the example reports the best decoded results along the way and the best decoded solution at the end of the run. In this case, we can see that the algorithm discovers an input very close to $f(0.0, 0.0) = 0.0$

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

Lab Task 1:

Let us solve the travelling salesman problem using Genetic algorithms. The algorithm is designed to replicate the natural selection process to carry generation, i.e. survival of the fittest of beings. Standard genetic algorithms are divided into five phases which are:

So as already know that a salesman is given a set of cities, he must find the shortest route to visit each city exactly once and return to the starting city.

In this problem, the cities are taken as genes, string generated using these characters is called a chromosome, while a fitness score which is equal to the path length of all the cities mentioned, is used to target a population.

Fitness Score is defined as the length of the path described by the gene. Lesser the path length fitter is the gene. The fittest of all the genes in the gene pool survive the population test and move to the next iteration. The number of iterations depends upon the value of a cooling variable. The value of the cooling variable keeps on decreasing with each iteration and reaches a threshold after a certain number of iterations.

Hint: Use the following pseudo code for your implementation.

```
Initialize procedure GA{
    Set cooling parameter = 0;
    Evaluate population P(t);
```

```

While( Not Done ){
    Parents(t) = Select_Parents(P(t));
    Offspring(t) = Procreate(P(t));
    p(t+1) = Select_Survivors(P(t), Offspring(t));
    t = t + 1;
}
}

```

Find the optimal or near to optimal solution using genetic algorithms on the graph given in Figure below:

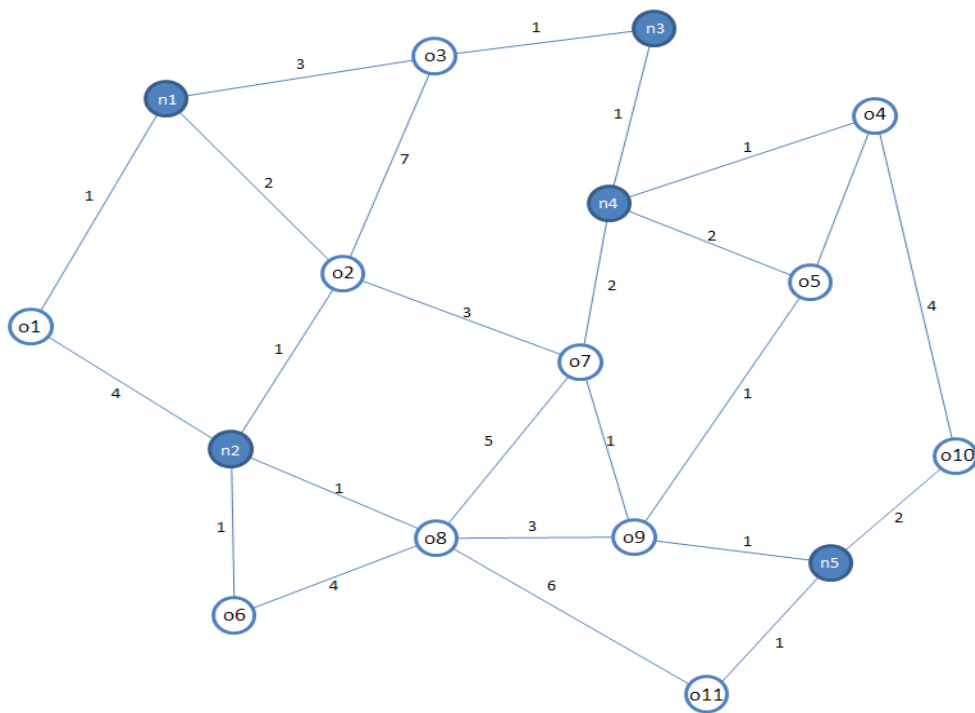


Figure: Graph for Travelling salesman Problem using genetic algorithms

Lab 11

Introduction to Game Theory

Min-Max and Alpha-Beta Pruning

Objective:

This lab will introduce students to genetic algorithms. Students will get the opportunity to get into details of game theory using combinational and min-max logic. They will also get a chance to develop basic logic inference engines for a game like Tic-Tac-Toe.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand the concept of Min-Max Algorithms
- Implement Alpha-Beta Pruning
- Develop a basic game

Instructor Note:

As pre-lab activity, read Chapters 5 from the book (Artificial Intelligence, A Modern Approach by Peter Norvig, 4th edition) to know the basics of Min-Max Algorithms.

1) Useful Concepts (Allocated Time 15 Minutes)

“Game Theory” is an analysis of strategic interaction. It consists of analyzing or modelling strategies and outcomes based on certain rules, in a game of 2 or more players. It has widespread applications and is useful in political scenarios, logical parlour games, business as well as to observe economic behaviour.

The Combinatorial games are two-person games with perfect information and no chance moves (no randomization like coin toss is involved that can effect the game). These games have a win-or-lose or tie outcome and determined by a set of positions, including an initial position, and the player whose turn it is to move. Play moves from one position to another, with the players usually alternating moves, until a terminal position is reached. A terminal position is one from which no moves are possible. Then one of the players is declared the winner and the other the loser. Or there is a tie (Depending on the rules of the combinatorial game, the game could end up in a tie. The only thing that can be stated about the combinatorial game is that the game should end at some point and should not be stuck in a loop. In order to prevent such looping situations in games like chess (consider the case of both the players just moving their queens to-and-fro from one place to the other), there is actually a “50-move rule” according to which the game is considered to be drawn if the last 50 moves by each player have been completed without the movement of any pawn and without any capture.

On the other hand, Game theory in general includes games of chance, games of imperfect knowledge, and games in which players can move simultaneously.

The specialty of Combinatorial Game Theory (CGT) is that the coding part is relatively very small and easy. The key to the Game Theory problems is that hidden observation, which can be sometimes very hard to find.

The commonly played Chess, Game of Nim, Tic-Tac-Toe all comes under the category of Combinatorial Game Theory.

On the other hand, an alternate approach is called Minimax, which is a kind of backtracking algorithm that is used in decision making and game theory to find the optimal move for a player. Assuming that your opponent also plays optimally. It is widely used in two player turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, Chess, etc.

In Minimax the two players are called maximizer and minimizer. The maximizer tries to get the highest score possible while the minimizer tries to do the opposite and get the lowest score possible.

Every board state has a value associated with it. In each state if the maximizer has the upper hand, then, the score of the board will tend to be some positive value. If the minimizer has the upper hand in that board state, then it will tend to be some negative value. The values of the board are calculated by some heuristics which are unique for every type of game.

2) Solved Lab Activities (Allotted Time 1 Hour)

Activity 1:

In the real world when we are creating a program to play Tic-Tac-Toe, Chess, Backgammon, etc. we need to implement a function that calculates the value of the board depending on the placement of pieces on the board. This function is often known as Evaluation Function or a Heuristic Function.

The evaluation function is unique for every type of game. The evaluation function for the game Tic-Tac-Toe is discussed below. The basic idea behind the evaluation function is to give a high value for a board if maximizer's turn or a low value for the board if minimizer's turn.

For this scenario let us consider X as the maximizer and O as the minimizer.

Let us build our evaluation function :

- If X wins on the board we give it a positive value of +10.
- If O wins on the board we give it a negative value of -10
- If no one has won or the game results in a draw then we give a value of +0.

We could have chosen any positive / negative value other than 10. For the sake of simplicity we chose 10 for the sake of simplicity we shall use lower case 'x' and lower case 'o' to represent the players and an

underscore '_' to represent a blank space on the board.

If we represent our board as a 3×3 2D character matrix, like `char board[3][3]`; then we have to check each row, each column and the diagonals to check if either of the players have gotten 3 in a row.

Now let us try implementing for a sample board using the following code:

```
# Python3 program to compute evaluation
# function for Tic Tac Toe Game.

# Returns a value based on who is winning
# b[3][3] is the Tic-Tac-Toe board
def evaluate(b):

    # Checking for Rows for X or O victory.
    for row in range(0, 3):

        if b[row][0] == b[row][1] and b[row][1] == b[row][2]:

            if b[row][0] == 'x':
                return 10
            else if b[row][0] == 'o':
                return -10

    # Checking for Columns for X or O victory.
    for col in range(0, 3):

        if b[0][col] == b[1][col] and b[1][col] == b[2][col]:

            if b[0][col]=='x':
                return 10
            else if b[0][col] == 'o':
                return -10

    # Checking for Diagonals for X or O victory.
    if b[0][0] == b[1][1] and b[1][1] == b[2][2]:

        if b[0][0] == 'x':
            return 10
        else if b[0][0] == 'o':
            return -10

    if b[0][2] == b[1][1] and b[1][1] == b[2][0]:
```

```

    if b[0][2] == 'x':
        return 10
    else if b[0][2] == 'o':
        return -10

# Else if none of them have won then return 0
return 0

# Driver code
if __name__ == "__main__":

    board = [['x', '_', 'o'],
              ['_', 'x', 'o'],
              ['_', '_', 'x']]

    value = evaluate(board)
    print("The value of this board is", value)

```

The output in this case would be 10 as from the board given above, try changing the configuration of the board in code to see how it works.

The task is to understand how to write a simple evaluation function for the game Tic-Tac-Toe. Now we shall see how to combine this evaluation function with the minimax function.

Activity 2:

Let us combine what we have learnt so far about minimax and evaluation function to write a proper Tic-Tac-Toe AI (Artificial Intelligence) that plays a perfect game. This AI will consider all possible scenarios and makes the most optimal move.

We shall be introducing a new function called `findBestMove()`. This function evaluates all the available moves using `minimax()` and then returns the best move the maximizer can make.

To check whether or not the current move is better than the best move we take the help of `minimax()` function which will consider all the possible ways the game can go and returns the best value for that move, assuming the opponent also plays optimally

The code for the maximizer and minimizer in the `minimax()` function is similar to `findBestMove()`

To check whether the game is over and to make sure there are no moves left we use `isMovesLeft()` function. It is a simple straightforward function which checks whether a move is available or not and returns true or false respectively.

The implementation of the above is given as:

```

# Python3 program to find the next optimal move for a player
player, opponent = 'x', 'o'

# This function returns true if there are moves
# remaining on the board. It returns false if
# there are no moves left to play.
def isMovesLeft(board) :

    for i in range(3) :
        for j in range(3) :
            if (board[i][j] == '_') :
                return True
    return False

# This is the evaluation function as discussed
# in the previous article ( http://goo.gl/sJgv68 )
def evaluate(b) :

    # Checking for Rows for X or O victory.
    for row in range(3) :
        if (b[row][0] == b[row][1] and b[row][1] == b[row][2]) :
            if (b[row][0] == player) :
                return 10
            elif (b[row][0] == opponent) :
                return -10

    # Checking for Columns for X or O victory.
    for col in range(3) :

        if (b[0][col] == b[1][col] and b[1][col] == b[2][col]) :

            if (b[0][col] == player) :
                return 10
            elif (b[0][col] == opponent) :
                return -10

    # Checking for Diagonals for X or O victory.
    if (b[0][0] == b[1][1] and b[1][1] == b[2][2]) :

        if (b[0][0] == player) :
            return 10
        elif (b[0][0] == opponent) :
            return -10

```

```

        return -10

    if (b[0][2] == b[1][1] and b[1][1] == b[2][0]) :

        if (b[0][2] == player) :
            return 10
        elif (b[0][2] == opponent) :
            return -10

    # Else if none of them have won then return 0
    return 0

# This is the minimax function. It considers all
# the possible ways the game can go and returns
# the value of the board
def minimax(board, depth, isMax) :
    score = evaluate(board)

    # If Maximizer has won the game return his/her
    # evaluated score
    if (score == 10) :
        return score

    # If Minimizer has won the game return his/her
    # evaluated score
    if (score == -10) :
        return score

    # If there are no more moves and no winner then
    # it is a tie
    if (isMovesLeft(board) == False) :
        return 0

    # If this maximizer's move
    if (isMax) :
        best = -1000

        # Traverse all cells
        for i in range(3) :
            for j in range(3) :

                # Check if cell is empty

```

```

        if (board[i][j]=='_') :

            # Make the move
            board[i][j] = player

            # Call minimax recursively and choose
            # the maximum value
            best = max( best, minimax(board,
                                     depth + 1,
                                     not isMax) )

            # Undo the move
            board[i][j] = '_'
    return best

# If this minimizer's move
else :
    best = 1000

    # Traverse all cells
    for i in range(3) :
        for j in range(3) :

            # Check if cell is empty
            if (board[i][j] == '_' ) :

                # Make the move
                board[i][j] = opponent

                # Call minimax recursively and choose
                # the minimum value
                best = min(best, minimax(board, depth + 1, not isMax))

                # Undo the move
                board[i][j] = '_'
    return best

# This will return the best possible move for the player
def findBestMove(board) :
    bestVal = -1000
    bestMove = (-1, -1)

```

```

# Traverse all cells, evaluate minimax function for
# all empty cells. And return the cell with optimal
# value.
for i in range(3) :
    for j in range(3) :

        # Check if cell is empty
        if (board[i][j] == '_') :

            # Make the move
            board[i][j] = player

            # compute evaluation function for this
            # move.
            moveVal = minimax(board, 0, False)

            # Undo the move
            board[i][j] = '_'

            # If the value of the current move is
            # more than the best value, then update
            # best/
            if (moveVal > bestVal) :
                bestMove = (i, j)
                bestVal = moveVal

    print("The value of the best Move is :", bestVal)
    print()
    return bestMove

# Driver code
board = [
    [ 'x', 'o', 'x' ],
    [ 'o', 'o', 'x' ],
    [ '_', '_', '_' ]
]

bestMove = findBestMove(board)

print("The Optimal Move is :")
print("ROW:", bestMove[0], " COL:", bestMove[1])

```

The output is given as:

The value of the best Move is : 10

The Optimal Move is :

ROW: 2 COL: 2

The Minimax may confuse programmers as it thinks several moves in advance and is very hard to debug at times. Remember this implementation of minimax algorithm can be applied any 2 player board game with some minor changes to the board structure and how we iterate through the moves. Also sometimes it is impossible for minimax to compute every possible game state for complex games like Chess. Hence, we only compute up to a certain depth and use the evaluation function to calculate the value of the board.

about Alpha-Beta pruning that can drastically improve the time taken by minimax to traverse a game tree.

Activity 3:

The Alpha-Beta pruning is an optimization technique for minimax algorithm. It reduces the computation time by a huge factor. This allows us to search much faster and even go into deeper levels in the game tree. It cuts off branches in the game tree which need not be searched because there already exists a better move available. It is called Alpha-Beta pruning because it passes 2 extra parameters in the minimax function, namely alpha and beta.

Let's define the parameters alpha and beta.

- Alpha is the best value that the maximizer currently can guarantee at that level or above.
- Beta is the best value that the minimizer currently can guarantee at that level or above.

So in order to implement Alpha-Beta pruning let us refer to the following tree in Figure below:

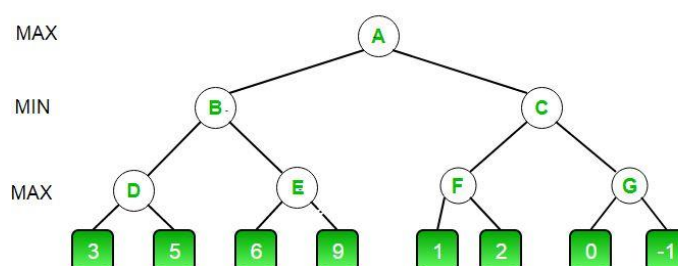


Figure: Alpha Beta Pruning Tree

The Alpha-Beta pruning mechanism is explained below:

- The initial call starts from A. The value of alpha here is **-INFINITY** and the value of beta is **+INFINITY**. These values are passed down to subsequent nodes in the tree. At A the maximizer must choose max of B and C, so A calls B first
- At B it the minimizer must choose min of D and E and hence calls D first.

-
- At **D**, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at **D** is $\max(-\text{INF}, 3)$ which is 3.
 - To decide whether its worth looking at its right node or not, it checks the condition $\beta \leq \alpha$. This is false since $\beta = +\text{INF}$ and $\alpha = 3$. So it continues the search.
 - **D** now looks at its right child which returns a value of 5. At **D**, $\alpha = \max(3, 5)$ which is 5. Now the value of node **D** is 5
 - **D** returns a value of 5 to **B**. At **B**, $\beta = \min(+\text{INF}, 5)$ which is 5. The minimizer is now guaranteed a value of 5 or lesser. **B** now calls **E** to see if he can get a lower value than 5.
 - At **E** the values of alpha and beta is not $-\text{INF}$ and $+\text{INF}$ but instead $-\text{INF}$ and 5 respectively, because the value of beta was changed at **B** and that is what **B** passed down to **E**
 - Now **E** looks at its left child which is 6. At **E**, $\alpha = \max(-\text{INF}, 6)$ which is 6. Here the condition becomes true. β is 5 and α is 6. So $\beta \leq \alpha$ is true. Hence it breaks and **E** returns 6 to **B**
 - Note how it did not matter what the value of **E**'s right child is. It could have been $+\text{INF}$ or $-\text{INF}$, it still wouldn't matter, We never even had to look at it because the minimizer was guaranteed a value of 5 or lesser. So as soon as the maximizer saw the 6 he knew the minimizer would never come this way because he can get a 5 on the left side of **B**. This way we don't have to look at that 9 and hence saved computation time.
 - **E** returns a value of 6 to **B**. At **B**, $\beta = \min(5, 6)$ which is 5. The value of node **B** is also 5
 - **B** returns 5 to **A**. At **A**, $\alpha = \max(-\text{INF}, 5)$ which is 5. Now the maximizer is guaranteed a value of 5 or greater. **A** now calls **C** to see if it can get a higher value than 5.
 - At **C**, $\alpha = 5$ and $\beta = +\text{INF}$. **C** calls **F**
 - At **F**, $\alpha = 5$ and $\beta = +\text{INF}$. **F** looks at its left child which is 1. $\alpha = \max(5, 1)$ which is still 5.
 - **F** looks at its right child which is 2. Hence the best value of this node is 2. α still remains 5
 - **F** returns a value of 2 to **C**. At **C**, $\beta = \min(+\text{INF}, 2)$. The condition $\beta \leq \alpha$ becomes true as $\beta = 2$ and $\alpha = 5$. So it breaks and it does not even have to compute the entire sub-tree of **G**.
 - The intuition behind this break off is that, at **C** the minimizer was guaranteed a value of 2 or lesser. But the maximizer was already guaranteed a value of 5 if he choose **B**. So why would the maximizer ever choose **C** and get a value less than 2 ? Again you can see that it did not matter what those last 2 values were. We also saved a lot of computation by skipping a whole sub tree.
 - **C** now returns a value of 2 to **A**. Therefore the best value at **A** is $\max(5, 2)$ which is 5.
 - Hence the optimal value that the maximizer can get is 5

```
# Python3 program to demonstrate
# working of Alpha-Beta Pruning
```

```
# Initial values of Alpha and Beta
MAX, MIN = 1000, -1000
```

```
# Returns optimal value for current player
#(Initially called for root and maximizer)
def minimax(depth, nodeIndex, maximizingPlayer,
            values, alpha, beta):
```

```
# Terminating condition. i.e
# leaf node is reached
if depth == 3:
    return values[nodeIndex]

if maximizingPlayer:

    best = MIN

    # Recur for left and right children
    for i in range(0, 2):

        val = minimax(depth + 1, nodeIndex * 2 + i,
                      False, values, alpha, beta)
        best = max(best, val)
        alpha = max(alpha, best)

        # Alpha Beta Pruning
        if beta <= alpha:
            break

    return best

else:
    best = MAX

    # Recur for left and
    # right children
    for i in range(0, 2):

        val = minimax(depth + 1, nodeIndex * 2 + i,
                      True, values, alpha, beta)
        best = min(best, val)
        beta = min(beta, best)

        # Alpha Beta Pruning
        if beta <= alpha:
            break

    return best
```

```
# Driver Code
if __name__ == "__main__":

    values = [3, 5, 6, 9, 1, 2, 0, -1]
    print("The optimal value is :", minimax(0, 0, True, values, MIN, MAX))
```

The output in this case would be 5.

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

Lab Task 1:

This problem is just like a puzzle. In the $N \times N$ chessboard, N queens have placed in such a manner no two queens in the same row and same column also not in the same diagonal. This arrangement is the solution to the N Queen problem. Assume the following configuration given in Figure below:

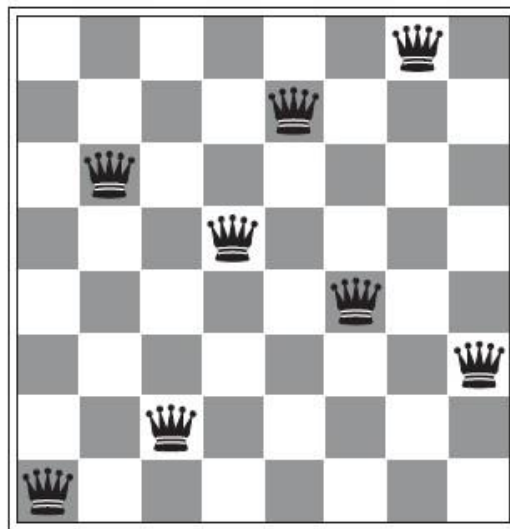


Figure: N-Queens Problem using MinMax Algorithm

Imagine an 8 queen problem, where the goal is to place 8 queens on an 8 X 8 board such that no two queens are on the same row or column or diagonal. Solve the N-Queens problem using Min-Max backtracking algorithms, try generating all possible configurations on the board having non attacking queens. [Refer to lecture slides as well for more details]

Lab Task 2:

Generate the complete game tree for the above N-Queens Problem and apply Alpha-Beta pruning to your implementation. Prove that optimization has taken place after the application of Alpha-Beta pruning.

Lab 12

Linear Regression: Experimenting with different Datasets

Objective:

This lab will strengthen the concepts Machine Learning Algorithms like, Linear Regression, Multiple Linear Regression and Polynomial Regression. Regression is a statistical method used in finance, investing, and other disciplines that attempts to determine the strength and character of the relationship between one dependent variable (usually denoted by Y) and a series of other variables (known as independent variables).

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand the regression techniques.
- Understand how to implement different regression algorithms.

Instructor Note:

As a pre-lab activity, read Chapters 3-5 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 15 Minutes)

The two basic types of regression are simple linear regression and multiple linear regression, although there are non-linear regression methods for more complicated data and analysis. Simple linear regression uses one independent variable to explain or predict the outcome of the dependent variable Y, while multiple linear regression uses two or more independent variables to predict the outcome.

Regression can help finance and investment professionals as well as professionals in other businesses. Regression can also help predict sales for a company based on weather, previous sales, GDP growth, or other types of conditions. The capital asset pricing model (CAPM) is an often-used regression model in finance for pricing assets and discovering costs of capital.

The general form of each type of regression is:

Simple linear regression: $Y = a + bX + u$

Multiple linear regression: $Y = a + b_1X_1 + b_2X_2 + b_3X_3 + \dots + b_tX_t + u$

Where:

Y = the variable that you are trying to predict (dependent variable).

X = the variable that you are using to predict Y (independent variable).

a = the intercept.

b = the slope.

u = the regression residual.

There are two basic types of regression: simple linear regression and multiple linear regression.

Regression takes a group of random variables, thought to be predicting Y, and tries to find a mathematical relationship between them. This relationship is typically in the form of a straight line (linear regression) that best approximates all the individual data points. In multiple regression, the separate variables are differentiated by using subscripts.

KEY TAKEAWAYS

Regression helps investment and financial managers to value assets and understand the relationships between variables

2) Solved Lab Activities (Allocated Time 1 Hour)

Activity 1:

Simple linear regression is an approach for predicting a response using a single feature.

It is assumed that the two variables are linearly related. Hence, we try to find a linear function that predicts the response value(y) as accurately as possible as a function of the feature or independent variable(x).

Let us start to experiment with Simple Linear Regression:
You can find the dataset in the link below:

<https://drive.google.com/drive/folders/1fy6n2K1VIMk2EsDidduF4bwCxf0TngyX?usp=sharing>

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

"""## Importing the dataset"""

dataset = pd.read_csv("D:\Regression\Salary_Data.csv")

X = dataset.iloc[:, :-1].values
```

```

y = dataset.iloc[:, -1].values

"""## Splitting the dataset into the Training set and Test set"""

from sklearn.model_selection import train_test_split
X_train , X_test , y_train , y_test = train_test_split(X,y,test_size = 1/3 ,
random_state = 0)
print(X_train)
print ("-----")
print (y_train)
print ("-----")
print(X_test)
print ("-----")
print (y_test)
print ("_____")

""" Training the Simple Linear Regression model on the Training set"""

from sklearn.linear_model import LinearRegression
regressor = LinearRegression()

regressor.fit(X_train,y_train)

print(X_train)
print ("-----")
print (y_train)
print ("-----")
print(X_test)
print ("-----")
print (y_test)

""" Predicting the Test set results"""

# X_test = np.array([X_test])
# print(X_test)
y_pred = regressor.predict(X_test)

"""## Visualising the Training set results"""

plt.scatter(X_train,y_train,color='r')
plt.plot(X_train , regressor.predict(X_train),color='b')
plt.title('Salary vs Experience (Training set)')
plt.xlabel('Years of Experience')

```

```

plt.ylabel('Salary')
plt.show()

""" Visualising the Test set results"""

plt.scatter(X_test, y_test, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')
plt.title('Salary vs Experience (Test set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()

""" Visualizing Test Results by using X_test and y_pred in plt.plot"""

plt.scatter(X_test, y_test, color = 'red')
plt.plot(X_test, y_pred, color = 'blue')
plt.title('Salary vs Experience (Test set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()

""" Making a single prediction (for example the salary of an employee with 12
years of experience)"""

single_prediction = regressor.predict([[12]])

print (single_prediction)

coefficient = regressor.coef_
intercept = regressor.intercept_

print ("Coefficient is:",coefficient)
print("Slope/Intercept:" , intercept)

""" Extra Practice"""

manual_prediction = 26816.192244031183 + 9345.94244312*15

print ("Manual prediction by using coefficient and intercept" ,
manual_prediction)

auto_prediction = regressor.predict([[15]])

```

```
print("autoprediction:", auto_prediction)
```



Figure: Output Graph of Linear Regression

Output chart is given in the above Figure.

Activity 2:

Multiple linear regression (MLR), also known simply as multiple regression, is a statistical technique that uses several explanatory variables to predict the outcome of a response variable. The goal of multiple linear regression is to model the linear relationship between the explanatory (independent) variables and response (dependent) variables. In essence, multiple regression is the extension of ordinary least-squares (OLS) regression because it involves more than one explanatory variable.

Multiple linear regression (MLR), also known simply as multiple regression, is a statistical technique that uses several explanatory variables to predict the outcome of a response variable.

Multiple regression is an extension of linear (OLS) regression that uses just one explanatory variable.

What Multiple Linear Regression Can Tell You

Simple linear regression is a function that allows an analyst or statistician to make predictions about one variable based on the information that is known about another variable. Linear regression can only be used when one has two continuous variables—an independent variable and a dependent variable. The independent variable is the parameter that is used to calculate the dependent variable or outcome. A multiple regression model extends to several explanatory variables.

The multiple regression model is based on the following assumptions:

- There is a linear relationship between the dependent variables and the independent variables
- The independent variables are not too highly correlated with each other
- y_i observations are selected independently and randomly from the population
- Residuals should be normally distributed with a mean of 0 and variance σ

You can find the dataset in the link mentioned above with Linear regression:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

"""## Importing the dataset"""

dataset = pd.read_csv(r'D:\Regression\50_Startups.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

print(X)

"""## Encoding categorical data"""

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3])],
remainder='passthrough')
X = np.array(ct.fit_transform(X))

print(X)

"""## Splitting the dataset into the Training set and Test set"""

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
random_state = 0)

"""## Training the Multiple Linear Regression model on the Training set"""

from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)

"""## Predicting the Test set results"""
```

```
y_pred = regressor.predict(X_test)
np.set_printoptions(precision=2)
print(np.concatenate((y_pred.reshape(len(y_pred),1),
y_test.reshape(len(y_test),1)),1))
```

Output:

```
[ [103015.2  103282.38]
  [132582.28 144259.4 ]
  [132447.74 146121.95]
  [ 71976.1   77798.83]
  [178537.48 191050.39]
  [116161.24 105008.31]
  [ 67851.69  81229.06]
  [ 98791.73  97483.56]
  [113969.44 110352.25]
  [167921.07 166187.94]]
```

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

In simple linear regression algorithm only works when the relationship between the data is linear. But suppose if we have non-linear data then Linear regression will not be capable to draw a best-fit line and it fails in such conditions. Consider the below diagram which has a non-linear relationship and you can see the Linear regression results on it, which does not perform well means which do not come close to reality. Hence, we introduce polynomial regression to overcome this problem, which helps identify the curvilinear relationship between independent and dependent variables.

Lab Task 1:

How Polynomial Regression Overcomes the problem of Non-Linear data?

Polynomial regression is a form of Linear regression where only due to the Non-linear relationship between dependent and independent variables we add some polynomial terms to linear regression to convert it into Polynomial regression. Suppose we have X as Independent data and Y as dependent data. Before feeding data to a model in preprocessing stage we convert the input variables into polynomial terms using some degree.

Apply Polynomial Regression Technique using “Position_salaries.csv” dataset given in Google Drive (link mentioned above linear regression code) and display the results.

Lab 13

Introduction to Game engines using Python

Objective:

The Games are played with a strategy. Every player or team would make a strategy before starting the game and they must change or build new strategy according to the current situation(s) in the game. We will have to consider computer games also use the same strategy as above. We will apply to concepts learned like Search Algorithms and optimizations problems to figure out the strategy in computer games.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand the basics of strategy making in games
- Understand how to implement different algorithms to make games
- Use advanced python libraries to make games like Snake etc.

Instructor Note:

As a pre-lab activity, read Chapters 7 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 15 Minutes)

The goal of search algorithms is to find the optimal set of moves so that they can reach at the final destination and win. These algorithms use the winning set of conditions, different for every game, to find the best moves.

Let us visualize a computer game as the tree. We know that tree has nodes. Starting from the root, we can come to the final winning node, but with optimal moves. That is the work of search algorithms. Every node in such tree represents a future state. The search algorithms search through this tree to make decisions at each step or node of the game.

We can use a combinational logic approach in games. The major disadvantage of using search algorithms is that they are exhaustive in nature, which is why they explore the entire search space to find the solution that leads to wastage of resources. It would be more cumbersome if these algorithms need to search the whole search space for finding the final solution.

To eliminate such kind of problem, we can use combinational search which uses the heuristic to explore the search space and reduces its size by eliminating the possible wrong moves. Hence, such algorithms can save the resources. Some of the algorithms that use heuristic to search the space and save the resources are discussed here –

The Min-Max is the strategy used by combinational search that uses heuristic to speed up the search strategy. The concept of Minimax strategy can be understood with the example of two player games, in

which each player tries to predict the next move of the opponent and tries to minimize that function. Also, to win, the player always try to maximize its own function based on the current situation.

Heuristic plays an important role in such kind of strategies like Minimax. Every node of the tree would have a heuristic function associated with it. Based on that heuristic, it will take the decision to make a move towards the node that would benefit them the most.

However, a major issue with Minimax algorithm is that it can explore those parts of the tree that are irrelevant, leads to the wastage of resources. Hence there must be a strategy to decide which part of the tree is relevant and which is irrelevant and leave the irrelevant part unexplored. Alpha-Beta pruning is one such kind of strategy.

The main goal of Alpha-Beta pruning algorithm is to avoid the searching those parts of the tree that do not have any solution. The main concept of Alpha-Beta pruning is to use two bounds named Alpha, the maximum lower bound, and Beta, the minimum upper bound. These two parameters are the values that restrict the set of possible solutions. It compares the value of the current node with the value of alpha and beta parameters, so that it can move to the part of the tree that has the solution and discard the rest.

The Negamax algorithm is not different from Minimax algorithm, but it has a more elegant implementation. The main disadvantage of using Minimax algorithm is that we need to define two different heuristic functions. The connection between these heuristics is that, the better a state of a game is for one player, the worse it is for the other player. In Negamax algorithm, the same work of two heuristic functions is done with the help of a single heuristic function.

2) Solved Lab Activities (Allocated Time 1 Hour)

Game development includes mathematics, logic, physics, AI, and much more and it can be amazingly fun. In python, game programming is done using a library in python called “pygame” and it is considered one of the most popular approaches.

Activity 1:

The best way to install pygame is with the pip tool (which python uses to install packages). Note, this comes with python in recent versions. We use the `--user` flag to tell it to install into the home directory, rather than globally.

```
python3 -m pip install -U pygame --user
```

Once you have installed pygame, you’re ready to create your very first pygame instance.

Let us consider the following code:

```
# import the pygame module
import pygame

# import pygame.locals for easier
# access to key coordinates
```

```
from pygame.locals import *

# Define our square object and call super to
# give it all the properties and methods of pygame.sprite.Sprite
# Define the class for our square objects
class Square(pygame.sprite.Sprite):
    def __init__(self):
        super(Square, self).__init__()

        # Define the dimension of the surface
        # Here we are making squares of side 25px
        self.surf = pygame.Surface((25, 25))

        # Define the color of the surface using RGB color coding.
        self.surf.fill((0, 200, 255))
        self.rect = self.surf.get_rect()

# initialize pygame
pygame.init()

# Define the dimensions of screen object
screen = pygame.display.set_mode((800, 600))

# instantiate all square objects
square1 = Square()
square2 = Square()
square3 = Square()
square4 = Square()

# Variable to keep our game loop running
gameOn = True

# Our game loop
while gameOn:
    # for loop through the event queue
    for event in pygame.event.get():

        # Check for KEYDOWN event
        if event.type == KEYDOWN:

            # If the Backspace key has been pressed set
            # running to false to exit the main loop
```

```

if event.key == K_BACKSPACE:
    gameOn = False

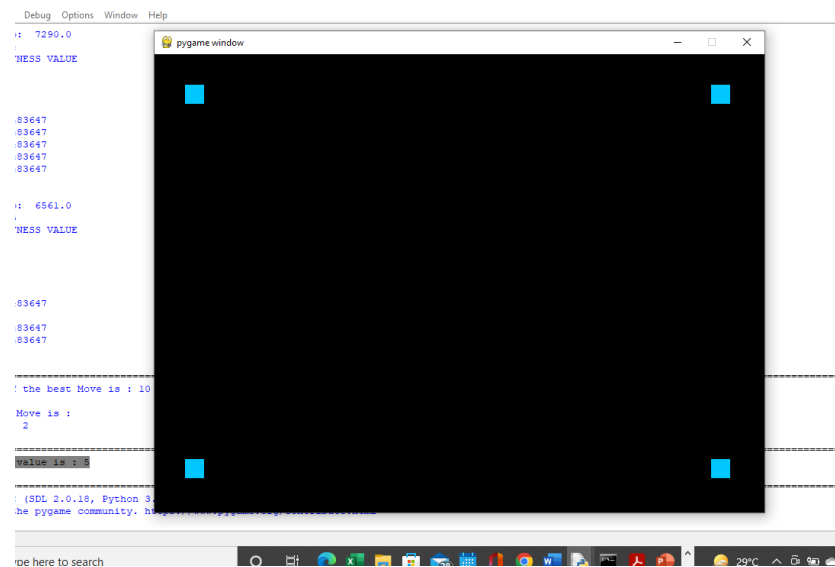
# Check for QUIT event
elif event.type == QUIT:
    gameOn = False

# Define where the squares will appear on the screen
# Use blit to draw them on the screen surface
screen.blit(square1.surf, (40, 40))
screen.blit(square2.surf, (40, 530))
screen.blit(square3.surf, (730, 40))
screen.blit(square4.surf, (730, 530))

# Update the display using flip
pygame.display.flip()

```

The output of the this code would be four cyan squares on the screen.



Activity 2:

Since Pygame is a cross-platform set of Python modules designed for writing video games. It includes computer graphics and sound libraries designed to be used with the Python programming language. Now, it's up to the imagination or necessity of the developer, what type of game we want to develop using this toolkit.

There are 7-basic steps to displaying Text on the pygame window :

- Create a display surface object using `display.set_mode()` method of pygame.
- Create a Font object using `font.Font()` method of pygame.
- Create a Text surface object i.e.surface object in which Text is drawn on it, using `render()` method of pygame font object.
- Create a rectangular object for the text surface object using `get_rect()` method of pygame text surface object.
- Set the position of the Rectangular object by setting the value of the center property of pygame rectangular object.
- Copying the Text surface object to the display surface object using `blit()` method of pygame display surface object.
- Show the display surface object on the pygame window using the `display.update()` method of pygame.

We can also add different animations to the text. By scrolling the text in 6 different ways on the pygame window.

- Scrolling the text on top of the Screen.
- Scrolling the text at the bottom of the screen.
- Scrolling the text on the left side of the Screen
- Scrolling the text on the right side of the Screen
- Scrolling the text in diagonal from left to right side of the Screen
- Scrolling the text in diagonal from right side to left side of the Screen.

Use the Code below to display and move text , you can implement your own pattern as well.

Below is the Implementation

```
# import pygame module in this program
import pygame

# activate the pygame library
# initiate pygame and give permission
# to use pygame's functionality.
pygame.init()

# create the display surface object
# (x, y) is the height and width of pygame window
win=pygame.display.set_mode((500, 500))

# set the pygame window name
pygame.display.set_caption("Scrolling Text")

# setting the pygame font style(1st parameter)
# and size of font(2nd parameter)
Font=pygame.font.SysFont('timesnewroman', 30)

# define the RGB value for white,
# green, yellow, orange colour
white=(255, 255, 255)
yellow=(255, 255, 0)
green=(0, 255, 255)
orange=(255, 100, 0)
done=False

# Split the text into letters
# 3rd parameter is font colour and
# 4th parameter is Font background
letter1=Font.render("C", False, orange, yellow)
letter2=Font.render("O", False, orange, green)
letter3=Font.render("M", False, orange, yellow)
letter4=Font.render("S", False, orange, green)
letter5=Font.render("A", False, orange, yellow)
letter6=Font.render("T", False, orange, green)
letter7=Font.render("S", False, orange, yellow)

# assigning values to
```

```
# i and c variable
i=0
c=1

# infinite loop
while not done:
    if(i>=820):
        i=0
        c+=1
        pygame.time.wait(500)

# completely fill the surface object
# with white color
win.fill(white)
if(c%6==0):
    # Scrolling the text in diagonal
    # on right side of the Screen.
    # copying the text surface object
    # to the display surface object
    # at the center coordinate.
    win.blit(letter1, (662-i, -162+i))
    win.blit(letter2, (639-i, -139+i))
    win.blit(letter3, (608-i, -108+i))
    win.blit(letter4, (579-i, -79+i))
    win.blit(letter5, (552-i, -52+i))
    win.blit(letter6, (529-i, -29+i))
    win.blit(letter7, (500 -i, 0 + i))
    i+=80
if(c%6==5):
    # Scrolling the text in diagonal on
    # left side of the Screen.
    win.blit(letter1, (-162+i, -162+i))
    win.blit(letter2, (-135+i, -135+i))
    win.blit(letter3, (-110+i, -110+i))
    win.blit(letter4, (-79+i, -79+i))
    win.blit(letter5, (-52+i, -52+i))
    win.blit(letter6, (-27+i, -27+i))
    win.blit(letter7, (0+i, 0+i))

    # Decides the speed of
    # the text on screen
    i+=80
```

```
if(c%6==4):

    # Scrolling the text in
    # right side of the Screen.
    win.blit(letter1, (480, -180+i))
    win.blit(letter2, (480, -150+i))
    win.blit(letter3, (480, -120+i))
    win.blit(letter4, (480, -90+i))
    win.blit(letter5, (480, -60+i))
    win.blit(letter6, (480, -30+i))
    win.blit(letter7, (480, 0+i))

    # Decides the speed of
    # the text on screen
    i +=80
if(c%6==3):
    # Scrolling the text in left
    # side of the Screen.
    win.blit(letter1, (0, -180+i))
    win.blit(letter2, (0, -150+i))
    win.blit(letter3, (0, -120+i))
    win.blit(letter4, (0, -90+i))
    win.blit(letter5, (0, -60+i))
    win.blit(letter6, (0, -30+i))
    win.blit(letter7, (0, 0+i))

    # Decides the speed of
    # the text on screen
    i+=80
if(c%6==1):

    win.blit(letter1, (-124+i, 0))
    win.blit(letter2, (-102+i, 0))
    win.blit(letter3, (-82+i, 0))
    win.blit(letter4, (-58+i, 0))
    win.blit(letter5, (-40+i, 0))
    win.blit(letter6, (-19+i, 0))
    win.blit(letter7, (0+i, 0))

    # Decides the speed of
    # the text on screen
    i +=80
```

```

if(c%6==2):

    # Scrolling the text in bottom of the Screen.
    win.blit(letter1, (-124+i, 470))
    win.blit(letter2, (-102+i, 470))
    win.blit(letter3, (-82+i, 470))
    win.blit(letter4, (-58+i, 470))
    win.blit(letter5, (-40+i, 470))
    win.blit(letter6, (-19+i, 470))
    win.blit(letter7, (0+i, 470))

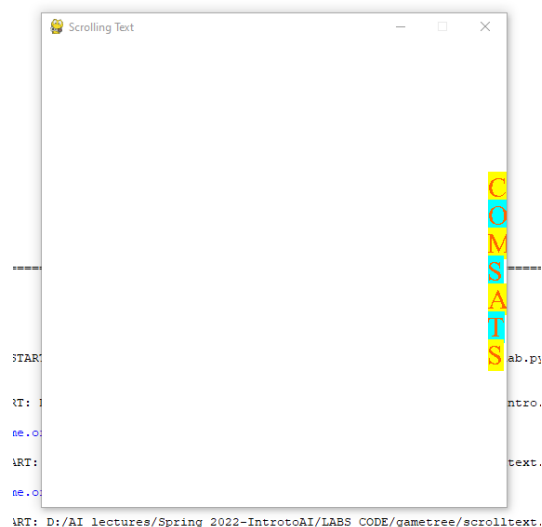
    # Decides the speed
    # of the text on screen
    i+=80

# Draws the surface object to the screen.
pygame.display.update()

# iterate over the list of Event objects
# that was returned by pygame.event.get() method
for event in pygame.event.get():
    if(event.type==pygame.QUIT):
        done=True
#Delay with 5ms
pygame.time.wait(500)
pygame.quit()

```

The output would be a moving text COMSATS around the screen.



Activity 3:

There are four basic steps to displaying images on the pygame window :

- Create a display surface object using `display.set_mode()` method of pygame.
- Create a Image surface object i.e.surface object in which image is drawn on it, using `image.load()` method of pygame.
- Copy the image surface object to the display surface object using `blit()` method of pygame display surface object.
- Show the display surface object on the pygame window using `display.update()` method of pygame.

The implementation is give as:

```
# import pygame module in this program
import pygame

# activate the pygame library .
# initiate pygame and give permission
# to use pygame's functionality.
pygame.init()

# define the RGB value
# for white colour
white = (255, 255, 255)

# assigning values to X and Y variable
X = 400
Y = 400

# create the display surface object
# of specific dimension..e(X, Y).
display_surface = pygame.display.set_mode((X, Y ))

# set the pygame window name
pygame.display.set_caption('Image')

# create a surface object, image is drawn on it.
image = pygame.image.load(r'C:\Users\user\Pictures\geek.jpg')

# infinite loop
while True :

    # completely fill the surface object
```

```
# with white colour
display_surface.fill(white)

# copying the image surface object
# to the display surface object at
# (0, 0) coordinate.
display_surface.blit(image, (0, 0))

# iterate over the list of Event objects
# that was returned by pygame.event.get() method.
for event in pygame.event.get() :

    # if event object type is QUIT
    # then quitting the pygame
    # and program both.
    if event.type == pygame.QUIT :

        # deactivates the pygame library
        pygame.quit()

        # quit the program.
        quit()

    # Draws the surface object to the screen.
    pygame.display.update()
```

Make sure to set the path of the image correctly in the code. It will display an image in a fixed size window for the user.

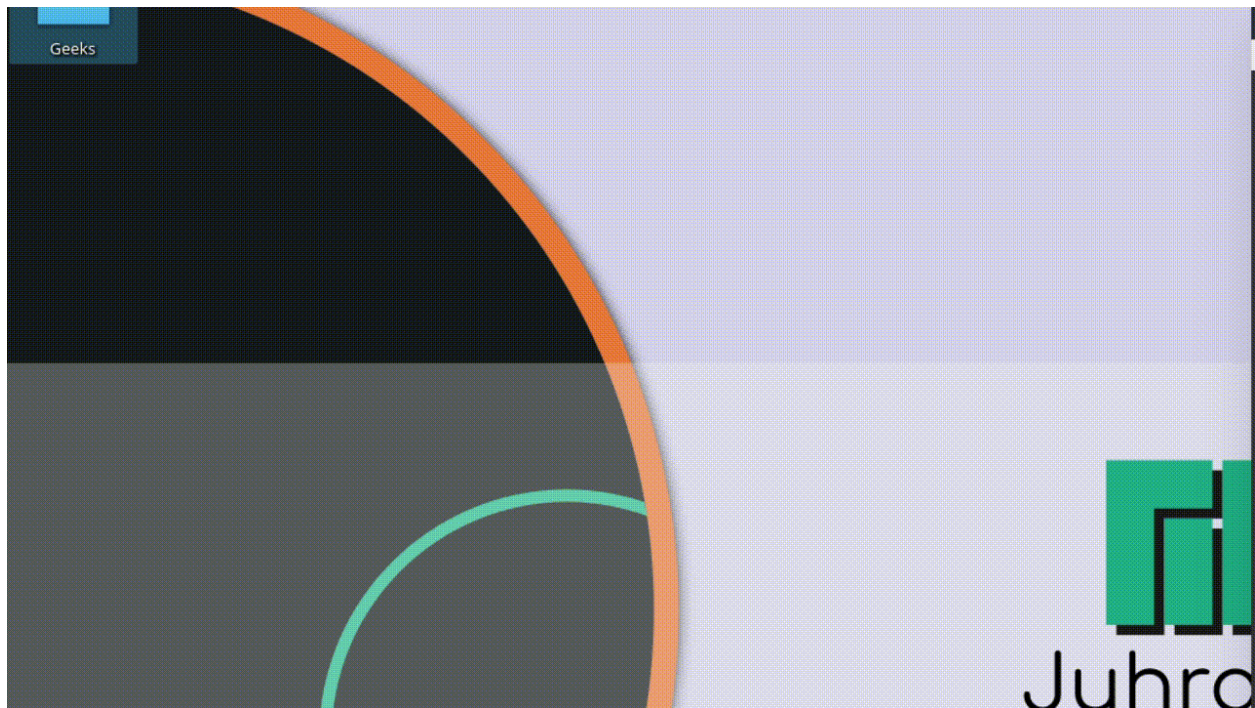
3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Snake game is one of the most popular arcade games of all time. In this game, the main objective of the player is to catch the maximum number of fruits without hitting the wall or itself. Creating a snake game can be taken as a challenge.

Lab Task 1:

Observe the basic implementation of the Snake Game shown below. Use the basic understanding of PyGame Libraries to create the game. In the first instance it should have the following:

1. Basic user controls
2. Scoring mechanism
3. Game Over Function



Lab Task 2:

When the basic game has been created, it is now time to add certain advanced features.

1. When the score exceeds 100 increase the speed two folds.
2. When the score exceeds 150 add additional yellow colour fruit with double marks.
3. Try changing adding obstacles in the existing game frame like walls

Lab 14

Introduction to Expert Systems using Prolog

Objective:

This lab will introduce students to Prolog. Prolog is a programming language, but a rather unusual one. "Prolog" is short for "Programming with Logic", and the link with logic gives Prolog its special character. At the heart of Prolog lies a surprising idea: don't tell the computer what to do. Instead, describe situations of interest, and compute by asking questions. Prolog will logically deduce new facts about the situations and give its deductions back to us as answers.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understand the basics of expert systems
- Understand how to implement an expert system in Prolog

Instructor Note:

As a pre-lab activity, read Chapters 7 from the book (Artificial Intelligence with Python: A Comprehensive Guide to Building Intelligent Apps for Python Beginners and Developers, Prateek Josh, Packt Publishing, 2017) to gain an insight about python programming and its fundamentals.

1) Useful Concepts (Allocated Time 15 Minutes)

Artificial Intelligence is a piece of software that simulates the behaviour and judgement of a human or an organization that has experts in a particular domain is known as an expert system. It does this by acquiring relevant knowledge from its knowledge base and interpreting it according to the user's problem. The data in the knowledge base is added by humans that are expert in a particular domain and this software is used by a non-expert user to acquire some information. It is widely used in many areas such as medical diagnosis, accounting, coding, games etc.

An expert system is AI software that uses knowledge stored in a knowledge base to solve problems that would usually require a human expert thus preserving a human expert's knowledge in its knowledge base. They can advise users as well as provide explanations to them about how they reached a particular conclusion or advice. Knowledge Engineering is the term used to define the process of building an Expert System and its practitioners are called Knowledge Engineers. The primary role of a knowledge engineer is to make sure that the computer possesses all the knowledge required to solve a problem. The knowledge engineer must choose one or more forms in which to represent the required knowledge as a symbolic pattern in the memory of the computer.

There are many examples of expert systems:

-
- MYCIN –
One of the earliest expert systems based on backward chaining. It can identify various bacteria that can cause severe infections and can also recommend drugs based on the person's weight.
 - DENDRAL –
It was an artificial intelligence-based expert system used for chemical analysis. It used a substance's spectrographic data to predict its molecular structure.
 - R1/XCON –
It could select specific software to generate a computer system wished by the user.
 - PXDES –
It could easily determine the type and the degree of lung cancer in a patient based on the data.
 - CaDet –
It is a clinical support system that could identify cancer in its early stages in patients.
 - DXplain –
It was also a clinical support system that could suggest a variety of diseases based on the findings of the doctor.

The Characteristics of an Expert System is based on the that:

- Human experts are perishable, but an expert system is permanent.
- It helps to distribute the expertise of a human.
- One expert system may contain knowledge from more than one human experts thus making the solutions more efficient.
- It decreases the cost of consulting an expert for various domains such as medical diagnosis.
- They use a knowledge base and inference engine.
- Expert systems can solve complex problems by deducing new facts through existing facts of knowledge, represented mostly as if-then rules rather than through conventional procedural code.
- Expert systems were among the first truly successful forms of artificial intelligence (AI) software.

However, they do have their limitations:

- Do not have human-like decision-making power.
- Cannot possess human capabilities.
- Cannot produce correct result from less amount of knowledge.
- Requires excessive training.

The advantages are:

- Low accessibility cost.
- Fast response.
- Not affected by emotions, unlike humans.
- Low error rate.
- Capable of explaining how they reached a solution.

-
- The disadvantages are:
 - The expert system has no emotions.
 - Common sense is the main issue of the expert system.
 - It is developed for a specific domain.
 - It needs to be updated manually. It does not learn itself.
 - Not capable to explain the logic behind the decision.

2) Solved Lab Activities (Allocated Time 1 Hour)

A basic expert system has the following components:

- **Knowledge Base**
The knowledge base represents facts and rules. It consists of knowledge in a particular domain as well as rules to solve a problem, procedures and intrinsic data relevant to the domain.
- **Inference Engine –**
The function of the inference engine is to fetch the relevant knowledge from the knowledge base, interpret it and to find a solution relevant to the user's problem. The inference engine acquires the rules from its knowledge base and applies them to the known facts to infer new facts. Inference engines can also include an explanation and debugging abilities.
- **Knowledge Acquisition and Learning Module –**
The function of this component is to allow the expert system to acquire more and more knowledge from various sources and store it in the knowledge base.
- **User Interface –**
This module makes it possible for a non-expert user to interact with the expert system and find a solution to the problem.
- **Explanation Module –**
This module helps the expert system to give the user an explanation about how the expert system reached a particular conclusion.

The Inference Engine generally uses two strategies for acquiring knowledge from the Knowledge Base, namely:

- Forward Chaining
- Backward Chaining

Activity 1

In the first instance we will learn how to install Prolog.

Go to <http://www.swi-prolog.org/> and download the standalone installer for your platform. After installing, go to /bin directory and open SWI-Prolog which will show the command prompt like interface as shown below,



Once the installation is complete we will get the following window, make sure to install the latest version.

Activity 2

How to create a simple Knowledge Base?

Go to the File menu and click on New. Type the filenames like test and click OK. This will create a new file name test.pl (every file in prolog is saved with .pl extension).

Activity 3

How to add facts in a Knowledge Base?

Knowledge Base 1 (KB1) is simply a collection of facts. Facts are used to state things that are unconditionally true of the domain of interest. For example, we can state that Mia, Jody, and Yolanda are women, and that Jody plays air guitar, using the following four facts:

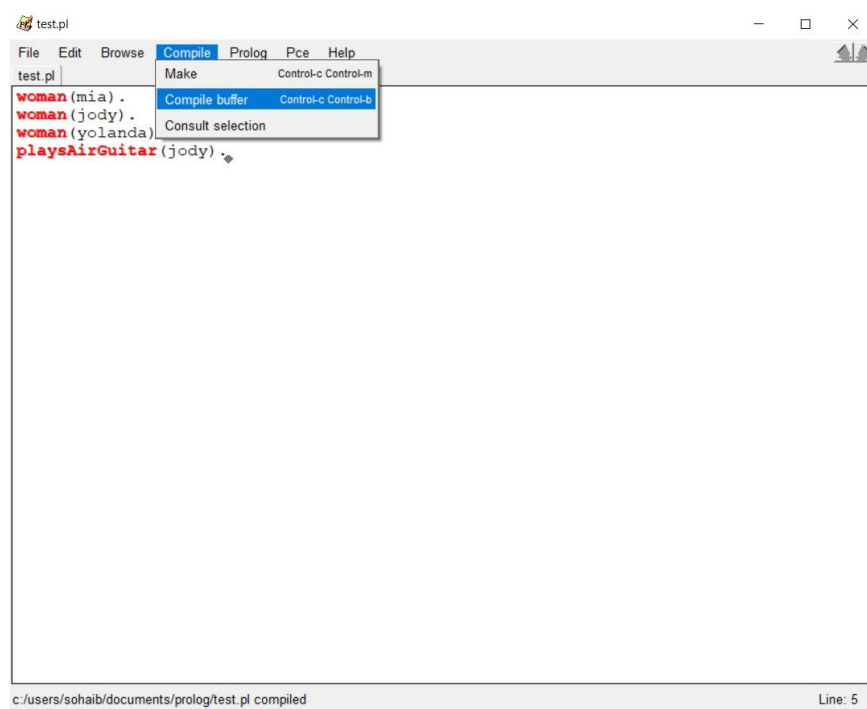
```
woman(mia). woman(jody). woman(yolanda). playsAirGuitar(jody).
```

Do not forget to enter a sentence with a period (.).

Activity 4

How to run a query within a Knowledge Base?

How can we use KB1? By posing queries. That is, by asking questions about the information KB1 contains. But first compile the program using compile option as shown in the following figure,



Now we can ask Prolog whether Mia is a woman by posing the query in the command prompt.

?- woman(mia).

Prolog will answer true for the obvious reason that this is one of the facts explicitly recorded in KB1. Incidentally, we don't type in the ?-. This symbol (or something like it, depending on the implementation of Prolog you are using) is the prompt symbol that the Prolog interpreter displays when it is waiting to evaluate a query. We just type in the actual query (for example `woman(mia)`) followed by `.(` (a full stop).

Activity 5

How to add rules alongside facts in a Knowledge Base?

Here is KB2, our second knowledge base:

```
listensToMusic(mia). happy(yolanda). playsAirGuitar(mia) :- listensToMusic(mia).  
playsAirGuitar(yolanda) :- listensToMusic(yolanda). listensToMusic(yolanda):- happy(yolanda).
```

KB2 contains two facts, `listensToMusic(mia)` and `happy(yolanda)`.

The last three items are rules.

Rules state information that is *conditionally* true of the domain of interest. For example, the first rule says that Mia plays air guitar *if* she listens to music, and the last rule says that Yolanda listens to music *if* she is happy. More generally, the `:-` should be read as “if”, or “is implied by”. The part on the left hand side of the `:-` is called the head of the rule, the part on the right hand side is called the body. So in general rules say: *if* the body of the rule is true, *then* the head of the rule is true too. And now for the key point: *if a knowledge base contains a rule head :- body, and Prolog knows that body follows from the information in the knowledge base, then Prolog can infer head.*

This fundamental deduction step is what logicians call modus ponens.

Let’s consider an example. We will ask Prolog whether Mia plays air guitar:
`?- playsAirGuitar(mia).`

Prolog will respond “yes”.

Why? Well, although `playsAirGuitar(mia)` is not a fact explicitly recorded in KB2, KB2 does contain the rule `playsAirGuitar(mia) :- listensToMusic(mia)`. Moreover, KB2 also contains the fact `listensToMusic(mia)`. Hence Prolog can use modus ponens to deduce that `playsAirGuitar(mia)`.

Our next example shows that Prolog can chain together uses of modus ponens. Suppose we ask: `?- playsAirGuitar(yolanda)`. Prolog would respond “yes”. Why? Well, using the fact `happy(yolanda)` and the rule `listensToMusic(yolanda):- happy(yolanda)`, Prolog can deduce the new fact `listensToMusic(yolanda)`.

This new fact is not explicitly recorded in the knowledge base — it is only *implicitly* present (it is *inferred* knowledge). Nonetheless, Prolog can then use it just like an explicitly recorded fact. Thus, together with the rule `playsAirGuitar(yolanda) :- listensToMusic(yolanda)` it can deduce that `playsAirGuitar(yolanda)`, which is what we asked it. Summing up: any fact produced by an application of modus ponens can be used as input to further rules. By chaining together applications of modus ponens in this way, Prolog is able to retrieve information that logically follows from the rules and facts recorded in the knowledge base.

The facts and rules contained in a knowledge base are called clauses. Thus KB2 contains five clauses, namely three rules and two facts. Another way of looking at KB2 is to say that it consists of three predicates (or procedures).

The three predicates are:
listensToMusic happy playsAirGuitar

The `happy` predicate is defined using a single clause (a fact). The `listensToMusic` and `playsAirGuitar` predicates are each defined using two clauses (in both cases, two rules). It is a good idea to think about Prolog programs in terms of the predicates they contain. In essence, the predicates are the concepts we find important, and the various clauses we write down concerning them are our attempts to pin down what they mean and how they are inter-related.

One final remark. We can view a fact as a rule with an empty body. That is, we can think of facts as “conditionals that do not have any antecedent conditions”, or “degenerate rules”

Activity 6

Use conjunction based rules in a Knowledge Base?

The KB3, our third knowledge base, consists of five clauses:

```
happy(vincent). listensToMusic(butch). playsAirGuitar(vincent):- listensToMusic(vincent), happy(vincent).  
playsAirGuitar(butch):- happy(butch).  
playsAirGuitar(butch):- listensToMusic(butch).
```

There are two facts, namely `happy(vincent)` and `listensToMusic(butch)`, and three rules.

KB3 defines the same three predicates as KB2 (namely `happy`, `listensToMusic`, and `playsAirGuitar`) but it defines them differently. In particular, the three rules that define the `playsAirGuitar` predicate introduce some new ideas. First, note that the rule,

```
playsAirGuitar(vincent):- listensToMusic(vincent), happy(vincent).
```

It has *two* items in its body, or (to use the standard terminology) two goals. What does this rule mean? The important thing to note is the comma `,` that separates the goal `listensToMusic(vincent)` and the goal `happy(vincent)` in the rule’s body. This is the way logical conjunction is expressed in Prolog (that is, the comma means *and*). So this rule says: “Vincent plays air guitar if he listens to music and he is happy”.

Thus, if we posed the query

```
?- playsAirGuitar(vincent).
```

Prolog would answer “no”. This is because while KB3 contains `happy(vincent)`, it does *not* explicitly contain the information `listensToMusic(vincent)`, and this fact cannot be deduced either. So KB3 only fulfils one of the two preconditions needed to establish `playsAirGuitar(vincent)`, and our query fails. Incidentally, the spacing used in this rule is irrelevant.

For example, we could have written it as

```
playsAirGuitar(vincent):- happy(vincent),listensToMusic(vincent).
```

and yet it would have meant exactly the same thing. Prolog offers us a lot of freedom in the way we set out knowledge bases, and we can take advantage of this to keep our code readable. Next, note that KB3 contains two rules with *exactly* the same head, namely:

```
playsAirGuitar(butch):- happy(butch).
playsAirGuitar(butch):- listensToMusic(butch).
```

This is a way of stating that Butch plays air guitar if *either* he listens to music, *or* if he is happy. That is, listing multiple rules with the same head is a way of expressing logical disjunction (that is, it is a way of saying *or*). So if we posed the query `?- playsAirGuitar(butch).` Prolog would answer “yes”. For although the first of these rules will not help (KB3 does not allow Prolog to conclude that `happy(butch)`), KB3 *does* contain `listensToMusic(butch)` and this means Prolog can apply modus ponens using the rule `playsAirGuitar(butch):- listensToMusic(butch).` to conclude that `playsAirGuitar(butch)`. There is another way of expressing disjunction in Prolog. We could replace the pair of rules given above by the single rule `playsAirGuitar(butch):- happy(butch); listensToMusic(butch).`

That is, the semicolon `;` is the Prolog symbol for *or*, so this single rule means exactly the same thing as the previous pair of rules. But Prolog programmers usually write multiple rules, as extensive use of semicolon can make Prolog code hard to read. It should now be clear that Prolog has something to do with logic: after all, the `:-` means implication, the `,` means conjunction, and the `;` means disjunction.

Moreover, we have seen that a standard logical proof rule (modus ponens) plays an important role in Prolog programming. And in fact “Prolog” is short for “Programming in logic”.

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

Create a knowledge base which defines your family tree and make a query that uses application of modus ponens to derive a fact which is not explicitly elaborated in the knowledge base. Use a range of queries to complete the family tree.

The following can be used as a base template to start creating your own tree <https://swish.swi-prolog.org/p/prolog-family-tree.pl>